Java Notes for Senior Engineer Interviews

Eight years of experience

These notes prepare an engineer with eight years of experience to explain Java guarantees, reconstruct correct code and defend a diagnosis or design decision. The contents and physical body are the default interview reading sequence: complete the Core sections first, then Applied depth, Awareness and Lookup. Code, distinct examples and follow-up answers stay with their owning topic.

Java 17 and Java 21 are the baseline. Java 24 and Java 25 differences are called out where they change behavior. Confirm the deployed JDK before making version-dependent claims. Examples include focused fragments and explicitly unsafe alternatives; identify the required contracts before reconstructing them.

Preparation depth

Priority	What to demonstrate
🔥 **CORE — MUST KNOW**	Explain the mechanism and guarantees, reconstruct an example, identify a failure trigger, and handle two follow-ups.
★ **APPLIED — HIGH ROI**	Defend one useful implementation or diagnosis, its tradeoff, and its behavior under failure or load.
○ **AWARENESS — KNOW CONCEPTUALLY**	Explain what it is, why it exists, when it matters, and one limitation. Look up detailed internals as needed.
△ **LOOKUP — LOOKUP ONLY**	Recognize the recipe or detail and find its topic. Practice it only when a task or recall gap makes it necessary.

Readiness check

- Give the main answer without notes in 20 to 60 seconds, then explain the mechanism and its limits.
- For a coding topic, reconstruct a useful example from a blank page and check a boundary or failure case.
- For a diagnostic topic, choose evidence that tests the hypothesis and say what would disprove it.
- Connect the answer to correctness, memory, resource limits, latency, or a production design decision.

Finish a topic when you can demonstrate its assigned depth in a cold attempt. Record a failed attempt as a specific repair task. After a repair, retry without notes. Once Core topics and the selected Applied scenarios pass, maintain them with mixed practice; add depth when a demonstrated gap or role requirement needs it.

Each numbered topic is its one main location. The ranked reading order, priority index and timed routes provide links; explanations, distinct examples, and follow-ups stay with the topic. Study time and completed reading do not establish readiness. Future updates follow 9.25.

Interview priority index

These are preparation depths for senior Java backend work, not predictions of interview questions. The contents, nested section numbers and physical body follow the same interview reading order. Each topic has one location. This category index groups topics by subject and links to their current section numbers. Complete Core sections 1 to 8 before Applied, Awareness and Lookup. Atomic risk reservations at 3.12 are Core because they test hands-on invariant reasoning.

Category	🔥 Core	★ Applied	○ Awareness	△ Lookup
Java language foundations	2.1, 2.5, 6.4	9.1	10.1	11.1
Object design and immutability	2.2, 2.3, 2.4, 2.6	None	None	None
Collections and data structures	2.7, 2.8, 2.9	9.2, 9.3	None	11.2
Functional programming Streams and collectors	5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 5.10, 5.11, 5.12	9.4, 9.5, 9.6	10.2	11.3
Exceptions serialization and resource management	6.1, 6.2, 6.3	None	10.3, 10.4	None
JVM execution and memory	4.1, 4.2	9.7, 9.8, 9.9	10.5, 10.6	None
Garbage collection and memory investigation	4.3, 4.4, 4.5, 4.6	9.10, 9.11, 9.12	10.7	None
Concurrency correctness and the Java Memory Model	3.1, 3.2, 3.3, 3.4, 3.5, 3.6	9.13, 9.14	10.8, 10.9	None
Concurrency utilities and asynchronous execution	3.7, 3.8, 3.9, 3.10, 3.11, 3.13	9.15, 9.16, 9.17, 9.18	None	None
Practical Java patterns and trading scenarios	3.12	9.19, 9.20, 9.21	None	None
Spring dependency injection proxies and transactions	7.1, 7.2, 7.3	None	None	None

Category	🔥 Core	★ Applied	○ Awareness	△ Lookup
SQL and coordination across processes	7.4, 7.5, 7.6	9.22	None	11.4
Performance diagnostics and engineering	4.7, 4.8, 4.9, 4.10, 4.11	9.23, 9.24	10.10	None
Interview practice and readiness	1.1, 8.1	9.25	None	None

Learning sequence

Read Core sections 1 to 8 in physical order: answer construction; Java contracts and collections; concurrency and bounded execution; JVM memory and performance diagnosis; Streams and collectors; exceptions, resources and numeric boundaries; Spring and shared authority; timed reconstruction and defense.

Then continue through Applied section 9, Awareness section 10 and Lookup section 11. References follow in section 12. If a role emphasizes a particular mechanism, use the timed routes for additional rehearsal after the default pass. Visible section numbers always match the physical hierarchy; internal bookmarks keep links stable.

A Lookup classification applies to the reference detail in that topic, not to the ability to use Java entry points, arrays, SQL, or string joining. Core contracts still have to work in the code you write. The linked contents that follow supply the full topic names.

Rehearsal routes

Choose a route for revision of material already studied. Attempt before reading; use the linked topic only to repair a failure, then retry. The time limit selects a sample of work and is not an interview-readiness guarantee. Stop at the scheduled boundary and record unfinished repairs.

30 minutes for a focused check

0 to 8 minutes explain equality and mutable map keys. 8 to 16 minutes explain one compound-action race and bounded executor capacity. 16 to 24 minutes diagnose a suspected leak or frozen batch and choose the first evidence. 24 to 30 minutes repair the weakest answer and repeat it without notes.

Topic addresses 2.3, 2.8, 3.3, 3.8, 4.4, 4.8

Complete with three recorded answers, a repeated answer after repair, and a specific next task.

2 hours for core reconstruction

0 to 30 minutes sample object contracts, immutability, and PECS. 30 to 65 minutes code grouping with a tie policy and test empty and duplicate input. 65 to 95 minutes code a bounded concurrent reservation and test rejection without mutation. 95 to 115 minutes diagnose a slow or blocked service. 115 to 120 minutes record repairs.

Topic addresses 2.4, 2.3, 2.5, 5.9, 5.10, 3.5, 3.12, 3.13, 4.7

Complete with two checked implementations, one evidence-based diagnosis, and a repair list.

4 hours for a senior rehearsal

0 to 40 minutes repair the weakest Core answers. 40 to 100 minutes complete the two coding tasks from the 2 hour route. 100 to 145 minutes defend transaction boundaries and an invariant across three JVMs. 145 to 185 minutes diagnose a performance regression. 185 to 205 minutes take a break. 205 to 240 minutes run a mixed oral mock and retry its weakest answer.

Topic addresses 7.2, 7.3, 7.4, 7.5, 3.12, 4.7, 8.1

Complete with checked code, a defended shared-state design, a tested diagnostic hypothesis, and one repaired mock answer.

8 hours for broad revision

0 to 90 minutes sample Core answers and repair failures, reserving the last 10 minutes for a break. 90 to 180 minutes reconstruct and check two coding examples. 180 to 240 minutes take a 40 minute meal break and spend 20 minutes on one Applied gap. 240 to 330 minutes

defend Spring, shared-state, and trading scenarios. 330 to 420 minutes take a 10 minute break, then diagnose failures and answer version-sensitive follow-ups. 420 to 480 minutes run a 35 minute mixed mock and spend 25 minutes on its most consequential repairs.

Topic addresses 3.12, 9.21, 7.3, 7.6, 3.10, 9.16, 9.12, 4.8, 8.1

Complete with a sampled readiness record, checked examples, defended scenarios, and a ranked repair list. Untested topics remain untested.

Ranked interview reading order 1 of 4

Ranks 1 to 24. These ranks match the physical body and refreshed contents. Core occupies ranks 1 to 57, Applied 58 to 82, Awareness 83 to 92 and Lookup 93 to 96. Read Core sections 1 to 8 straight through. Rank is the position among all 96 topics; a section number such as 3.12 identifies a topic within its parent section.

Rank	Depth	Canonical topic
1	🔥 Core	1.1 Answer construction and senior reasoning
2	🔥 Core	2.1 Values references parameter passing and dispatch
3	🔥 Core	2.2 Interfaces abstract classes inheritance and composition
4	🔥 Core	2.3 Equality hashing and ordering contracts
5	🔥 Core	2.4 Immutable objects and defensive copies
6	🔥 Core	2.5 Generics erasure and PECS
7	🔥 Core	2.6 Strings the pool interning and mutable builders
8	🔥 Core	2.7 Collection selection complexity and concurrent access
9	🔥 Core	2.8 HashMap internals and mutable keys
10	🔥 Core	2.9 PriorityQueue Deque and comparator safety
11	🔥 Core	3.1 Atomicity visibility ordering and happens before
12	🔥 Core	3.2 Synchronized monitors and reentrancy
13	🔥 Core	3.3 Races compound actions and state ownership
14	🔥 Core	3.4 Safe publication final fields and construction
15	🔥 Core	3.5 Atomic counters and LongAdder
16	🔥 Core	3.6 Deadlock livelock starvation and lock ordering
17	🔥 Core	3.7 Thread lifecycle wait notify and interruption
18	🔥 Core	3.8 Executors capacity queues and rejection
19	🔥 Core	3.9 Callable Future and completion stages
20	🔥 Core	3.10 CompletableFuture composition errors and cancellation
21	🔥 Core	3.11 ThreadLocal cleanup and context propagation
22	🔥 Core	3.12 Risk limits atomic reservations and partition ownership
23	🔥 Core	3.13 Testing concurrency and shutdown behavior
24	🔥 Core	4.1 JVM bytecode and execution model

Ranked interview reading order 2 of 4

Ranks 25 to 48. Continue in rank order. Skip a topic only after a cold demonstration passes its assigned depth; return to a linked prerequisite if an explanation fails.

Rank	Depth	Canonical topic
25	🔥 Core	4.2 Runtime memory areas and stack frames
26	🔥 Core	4.3 Reachability roots cycles and collection eligibility
27	🔥 Core	4.4 Memory leak patterns and retained references
28	🔥 Core	4.5 Heap dumps retained size and dominators
29	🔥 Core	4.6 GC diagnosis allocation rate and heap sizing
30	🔥 Core	4.7 Performance regressions CPU waiting and bottlenecks
31	🔥 Core	4.8 JFR thread dumps and diagnostic evidence
32	🔥 Core	4.9 Throughput tail latency and allocation cost
33	🔥 Core	4.10 Backpressure queue growth timeouts and Little law
34	🔥 Core	4.11 Optimization workflow and representative verification
35	🔥 Core	5.1 Functional interfaces lambdas and identity
36	🔥 Core	5.2 Stream pipelines laziness and single use
37	🔥 Core	5.3 Filtering mapping and flattening
38	🔥 Core	5.4 Distinct values equality and duplicate detection
39	🔥 Core	5.5 Sorting ranking limiting and skipping
40	🔥 Core	5.6 Finding matching extrema and Optional
41	🔥 Core	5.7 Reduction versus mutable collection
42	🔥 Core	5.8 Collectors and result mutability
43	🔥 Core	5.9 Lookup maps duplicate keys and merge policies
44	🔥 Core	5.10 Grouping downstream aggregation and partitioning
45	🔥 Core	5.11 Side effects peek and encounter order
46	🔥 Core	5.12 Parallel streams and execution constraints
47	🔥 Core	6.1 Throwable exceptions and errors
48	🔥 Core	6.2 Resource lifetime and deterministic cleanup

Ranked interview reading order 3 of 4

Ranks 49 to 72. Continue in rank order. Skip a topic only after a cold demonstration passes its assigned depth; return to a linked prerequisite if an explanation fails.

Rank	Depth	Canonical topic
49	🔥 Core	6.3 Exception policies catch ordering and translation
50	🔥 Core	6.4 Numeric precision overflow and time
51	🔥 Core	7.1 Container bean discovery injection lifecycle and scopes
52	🔥 Core	7.2 AOP proxies and self invocation
53	🔥 Core	7.3 Transaction propagation isolation and rollback
54	🔥 Core	7.4 Local coordination versus shared authority
55	🔥 Core	7.5 Database atomicity optimistic and pessimistic locking
56	🔥 Core	7.6 Idempotency retries and effect guarantees
57	🔥 Core	8.1 Timed coding debugging and defense practice
58	★ Applied	9.1 Modern Java features and version boundaries
59	★ Applied	9.2 Bounded caches FIFO LRU and deduplication
60	★ Applied	9.3 Sorted concurrent maps and skip lists
61	★ Applied	9.4 Primitive streams boxing and array results
62	★ Applied	9.5 Streams loops and algorithmic complexity
63	★ Applied	9.6 Stream coding practice and reconstruction
64	★ Applied	9.7 Allocation TLABs escape analysis and boxing
65	★ Applied	9.8 Class loading initialization and identity
66	★ Applied	9.9 JIT inlining deoptimization and code cache
67	★ Applied	9.10 Generational collection and pause semantics
68	★ Applied	9.11 G1 regions phases and humongous allocations
69	★ Applied	9.12 Collector choice G1 Parallel and ZGC
70	★ Applied	9.13 Explicit locks read write locks and StampedLock
71	★ Applied	9.14 CAS ABA and compound state transitions
72	★ Applied	9.15 ForkJoinPool and work stealing

Ranked interview reading order 4 of 4

Ranks 73 to 96. Continue in rank order. Skip a topic only after a cold demonstration passes its assigned depth; return to a linked prerequisite if an explanation fails.

Rank	Depth	Canonical topic
73	★ Applied	9.16 Virtual threads carrier threads and version specific pinning
74	★ Applied	9.17 Semaphores and downstream admission limits
75	★ Applied	9.18 CountDownLatch CyclicBarrier and coordination
76	★ Applied	9.19 Singleton initialization and ways to break assumptions
77	★ Applied	9.20 Object pools ownership and reference counting
78	★ Applied	9.21 Price time ordering and partial execution
79	★ Applied	9.22 Distributed locks leases and fencing
80	★ Applied	9.23 Lock contention and reducing shared mutation
81	★ Applied	9.24 JVM metrics and service observability
82	★ Applied	9.25 Maintaining and extending these notes
83	○ Awareness	10.1 Annotations reflection and module boundaries
84	○ Awareness	10.2 Collector internals and advanced composition
85	○ Awareness	10.3 Serializable Externalizable and safe wire formats
86	○ Awareness	10.4 Files paths character encodings and NIO
87	○ Awareness	10.5 Object layout compressed references and footprint
88	○ Awareness	10.6 Native memory JNI and process limits
89	○ Awareness	10.7 Reference strengths and weak maps
90	○ Awareness	10.8 False sharing and contention awareness
91	○ Awareness	10.9 Memory fences VarHandle and AQS
92	○ Awareness	10.10 JMH and microbenchmark traps
93	△ Lookup	11.1 Java startup and main methods
94	△ Lookup	11.2 Arrays lists copying and conversions
95	△ Lookup	11.3 Joining strings
96	△ Lookup	11.4 Joins row counts and data definition operations

Contents

1 Core Answering senior Java interview questions....13
1.1 Answer construction and senior reasoning....13
2 Core Java contracts and collection correctness....15
2.1 Values references parameter passing and dispatch....15
2.2 Interfaces abstract classes inheritance and composition....15
2.3 Equality hashing and ordering contracts....18
2.4 Immutable objects and defensive copies....19
2.5 Generics erasure and PECS....23
2.6 Strings the pool interning and mutable builders....24
2.7 Collection selection complexity and concurrent access....25
2.8 HashMap internals and mutable keys....29
2.9 PriorityQueue Deque and comparator safety....32
3 Core Concurrency and bounded execution....34
3.1 Atomicity visibility ordering and happens before....34
3.2 Synchronized monitors and reentrancy....37
3.3 Races compound actions and state ownership....41
3.4 Safe publication final fields and construction....43
3.5 Atomic counters and LongAdder....44
3.6 Deadlock livelock starvation and lock ordering....48
3.7 Thread lifecycle wait notify and interruption....51
3.8 Executors capacity queues and rejection....53
3.9 Callable Future and completion stages....58
3.10 CompletableFuture composition errors and cancellation....59
3.11 ThreadLocal cleanup and context propagation....62
3.12 Risk limits atomic reservations and partition ownership....65
3.13 Testing concurrency and shutdown behavior....68
4 Core JVM memory and performance diagnosis....69
4.1 JVM bytecode and execution model....69
4.2 Runtime memory areas and stack frames....73
4.3 Reachability roots cycles and collection eligibility....75
4.4 Memory leak patterns and retained references....78
4.5 Heap dumps retained size and dominators....82
4.6 GC diagnosis allocation rate and heap sizing....83
4.7 Performance regressions CPU waiting and bottlenecks....85
4.8 JFR thread dumps and diagnostic evidence....88
4.9 Throughput tail latency and allocation cost....90
4.10 Backpressure queue growth timeouts and Little law....91
4.11 Optimization workflow and representative verification....93
5 Core Streams and deterministic collection....95
5.1 Functional interfaces lambdas and identity....95

5.2 Stream pipelines laziness and single use..... 96
5.3 Filtering mapping and flattening.....99
5.4 Distinct values equality and duplicate detection..... 104
5.5 Sorting ranking limiting and skipping..... 105
5.6 Finding matching extrema and Optional..... 107
5.7 Reduction versus mutable collection..... 109
5.8 Collectors and result mutability..... 110
5.9 Lookup maps duplicate keys and merge policies..... 111
5.10 Grouping downstream aggregation and partitioning..... 115
5.11 Side effects peek and encounter order..... 119
5.12 Parallel streams and execution constraints..... 120
6 Core Exceptions resources and numeric boundaries..... 122
6.1 Throwable exceptions and errors..... 122
6.2 Resource lifetime and deterministic cleanup..... 125
6.3 Exception policies catch ordering and translation..... 126
6.4 Numeric precision overflow and time..... 127
7 Core Spring transactions and shared authority..... 129
7.1 Container bean discovery injection lifecycle and scopes..... 129
7.2 AOP proxies and self invocation..... 134
7.3 Transaction propagation isolation and rollback..... 136
7.4 Local coordination versus shared authority..... 137
7.5 Database atomicity optimistic and pessimistic locking..... 139
7.6 Idempotency retries and effect guarantees..... 140
8 Core Timed reconstruction and defense..... 142
8.1 Timed coding debugging and defense practice..... 142
9 Applied Java depth and production scenarios..... 147
9.1 Modern Java features and version boundaries..... 147
9.2 Bounded caches FIFO LRU and deduplication..... 148
9.3 Sorted concurrent maps and skip lists..... 149
9.4 Primitive streams boxing and array results..... 151
9.5 Streams loops and algorithmic complexity..... 153
9.6 Stream coding practice and reconstruction..... 155
9.7 Allocation TLABs escape analysis and boxing..... 157
9.8 Class loading initialization and identity..... 160
9.9 JIT inlining deoptimization and code cache..... 162
9.10 Generational collection and pause semantics..... 164
9.11 G1 regions phases and humongous allocations..... 165
9.12 Collector choice G1 Parallel and ZGC..... 167
9.13 Explicit locks read write locks and StampedLock..... 167
9.14 CAS ABA and compound state transitions..... 170
9.15 ForkJoinPool and work stealing..... 174

9.16 Virtual threads carrier threads and version specific pinning..... 175
9.17 Semaphores and downstream admission limits..... 177
9.18 CountDownLatch CyclicBarrier and coordination..... 178
9.19 Singleton initialization and ways to break assumptions..... 179
9.20 Object pools ownership and reference counting..... 185
9.21 Price time ordering and partial execution..... 187
9.22 Distributed locks leases and fencing..... 188
9.23 Lock contention and reducing shared mutation..... 189
9.24 JVM metrics and service observability..... 190
9.25 Maintaining and extending these notes..... 191
10 Awareness Java internals and version details..... 194
10.1 Annotations reflection and module boundaries..... 194
10.2 Collector internals and advanced composition..... 195
10.3 Serializable Externalizable and safe wire formats..... 196
10.4 Files paths character encodings and NIO..... 198
10.5 Object layout compressed references and footprint..... 200
10.6 Native memory JNI and process limits..... 201
10.7 Reference strengths and weak maps..... 202
10.8 False sharing and contention awareness..... 203
10.9 Memory fences VarHandle and AQS..... 203
10.10 JMH and microbenchmark traps..... 205
11 Lookup Java and SQL recipes..... 206
11.1 Java startup and main methods..... 206
11.2 Arrays lists copying and conversions..... 206
11.3 Joining strings..... 207
11.4 Joins row counts and data definition operations..... 207
12 Official references..... 210

1 Core Answering senior Java interview questions

Begin with the answer method, guarantees, failure boundaries and evidence expected from a senior engineer.

1.1 Answer construction and senior reasoning

What these questions test

The related questions form a technical map.

Use it to locate the state, guarantee and failure boundary.

The technical map connects language and object contracts to JVM execution, concurrency and Spring behavior.

Apply those mechanisms to production diagnosis, performance and observability, then to database locking, idempotency and coordination across JVMs. A question about `volatile` starts with its exact guarantee and can lead to a shared-state design decision.

Explain what happens when two cores observe shared state, and justify the visibility and atomicity guarantees.

For GC, explain both collection eligibility and the evidence behind a memory-growth diagnosis.

Identify the live root path, retained graph and lifecycle responsible for growth.

For `synchronized`, state the monitor discipline and its happens-before relationship.

Then explain why a JVM monitor cannot enforce an invariant across three application instances.

That's the transition from Java programmer → senior backend engineer.

Final interview mental model

When an interviewer gives you any JVM/concurrency problem, don't immediately name a class.

Walk through: 1. WHAT STATE EXISTS?

2. WHO OWNS IT?

3. IS IT SHARED?

4. IS IT MUTABLE?

5. WHAT INVARIANT MUST HOLD?

6. WHAT CAN HAPPEN CONCURRENTLY?

7. DO I NEED: atomicity?

visibility?

ordering?

8. CAN I REMOVE SHARING?

9. CAN I PARTITION OWNERSHIP?

10. IF NOT: `volatile`?

atomic?

lock?

concurrent collection?

11. WHAT HAPPENS UNDER CONTENTION?

12. WHAT HAPPENS UNDER OVERLOAD?

13. IS THE QUEUE BOUNDED?

14. WHAT IS THE BOTTLENECK?

15. WHAT HAPPENS TO p99?

16. HOW WOULD I OBSERVE IT?

17. HOW WOULD I PROVE THE FIX?

That is the transition from knowing Java syntax to thinking like a senior JVM/backend engineer.

The interview strategy

For senior interviews, don't begin with everything you know.

Use: 1. DIRECT ANSWER

```
        ↓
```

2. WHY

```
        ↓
```

3. SMALL EXAMPLE

```
        ↓
```

4. EDGE CASE / FAILURE MODE

```
        ↓
```

5. PRODUCTION IMPLICATION; Example: Q: When is an object eligible for GC?

Bad: Java has heap memory and garbage collectors such as G1...

Better: An object becomes eligible for garbage collection when it is no longer strongly reachable from any GC root. Cycles don't prevent collection because Java GC is reachability-based, not reference-counting-based.

Then allow the interviewer to drill deeper.

2 Core Java contracts and collection correctness

Establish value and reference behavior, object contracts and collection invariants before reasoning about shared mutable state.

2.1 Values references parameter passing and dispatch

Core contract and interview points

Java passes arguments by value. For an object argument, the copied value is a reference: the callee can mutate the referenced object but reassigning its local parameter does not reassign the caller variable. `Arrays` are objects and follow the same rule.

Overloading is selected from compile-time types and applicable method signatures. Overridden instance methods dispatch using the runtime receiver type. Static methods are hidden rather than overridden. Explain `null` overload ambiguity and why casting can change overload selection without changing the object.

private limits direct access, final constrains reassignment or overriding depending on its target, and static associates a member with a class. A `final` reference can still point to a mutable object. Encapsulation is a contract over all access paths, including exposed collections.

Interview challenge: explain the result when a method changes `list.add(x)`, then assigns list = new `ArrayList`<>(); explain which change the caller observes.

Official reference Java Language Specification execution and memory model

Follow up question Does reassigning a method parameter change the caller's variable?

Answer: No. The callee receives a copied reference value. Mutating the shared list is visible through the caller's alias; assigning a new list to the local parameter is not. Draw the two reference slots before and after each statement.

Follow up question Can a cast change which implementation of an overridden method executes?

Answer: A cast can change compile-time overload selection, but virtual dispatch still uses the runtime receiver for the selected overridden instance method. Static method selection follows the qualifying compile-time type; null receivers cannot execute instance methods.

2.2 Interfaces abstract classes inheritance and composition

Interface vs Abstract Classes

A Java class has one direct superclass and may implement multiple interfaces. Conflicting inherited interface defaults can require an explicit override; multiple interface inheritance is allowed.

So a class can not extend multiple abstract classes but can implement multiple interfaces .

Use interfaces for substitutable contracts and abstract classes for a shared implementation or state hierarchy. The noun versus verb mnemonic is only a rough analogy and does not determine the design.

Interfaces are for when you want to say "I don't care how you do it, but here's what you need to get done." Abstract classes are for when you want to say "I know what you should do, and I know how you should do it in some/many of the cases." Abstract classes have some serious drawbacks. For example:

Intentional compile failure: Java does not allow a class to extend both House and Boat. The following interface alternative illustrates multiple contracts.

```
class House { }
class Boat { }
class HouseBoat extends House, Boat { }
```

You can get through via an interface:

```
interface Liveable { }
interface Floatable { }
class HouseBoat implements Liveable, Floatable { }
```

When to Use an Interface

Let's look at some scenarios when one should go with an interface:

- If the problem needs to be solved using multiple inheritances and is composed of different class hierarchies
- When unrelated classes implement our interface. For example, `Comparable` provides the `compareTo()` method that can be overridden to compare two objects
- When application functionalities have to be defined as a contract, but not concerned about who implements the behavior. i.e., third-party vendors need to implement it fully

Consider using the interface when our problem makes the statement "A is capable of [doing this]". For example, "Clonable is capable of cloning an object", "Drawable is capable of drawing a shape", etc.

When to Use an Abstract `Class`

Now, let's see some scenarios when one should use the abstract class:

- When trying to use the inheritance concept in code (share code among many related classes), by providing common base class methods that the subclasses override
- If we have specified requirements and only partial implementation details
- While classes that extend abstract classes have several common fields or methods (that require non-public modifiers)
- If one wants to have non-final or non-static methods to modify the states of an object

Consider using abstract classes and inheritance when our problem makes the evidence "A is a B". For example, "Dog is an Animal", "Lamborghini is a Car", etc.

Points to be Noted:-

An abstract subclass may leave inherited abstract methods unimplemented; a concrete subclass must implement the required methods.

An abstract class may contain zero abstract methods. Declaring it abstract prevents direct instantiation.

A Java class extends at most one class, including an abstract class, and may implement multiple interfaces.

Final method can't be overridden. Thus, an abstract function can't be final.

Inheritance (Is - A) vs Composition (Has - A)

Inheritance Some examples would be:

- A person is a human.
- A cat is an animal.
- A car is a vehicle.

Inheritance should only be used when:

- Both classes are in the same logical domain
- The subclass is a proper subtype of the superclass
- The superclass's implementation is necessary or appropriate for the subclass
- The enhancements made by the subclass are primarily additive.

Composition

- A car has a battery (a battery is part of a car).
- The car has a steering wheel.
- A person has a heart (a heart is part of a person).
- A house has a living room (a living room is part of a house).

They're different in purpose too.

- Inheritance: To design a class on what it is.
- Composition: build behavior by delegating to collaborators. Use inheritance only when the subtype can honor the parent contract; an is-a analogy alone does not establish substitutability.

Official reference Java language classes inheritance and records

Follow up question When would you choose composition over inheritance?

Answer: Use composition when behavior should vary independently or a collaborator does not satisfy the parent’s behavioral contract. Inheritance requires substitutability, including invariants and failure behavior; code reuse alone is insufficient justification.

Follow up question What if two inherited interface defaults conflict?

Answer: An applicable concrete class method takes precedence over interface defaults. A more specific interface default takes precedence over a less specific one; otherwise resolve the unrelated-default conflict explicitly in the implementing class.

2.3 Equality hashing and ordering contracts

Core contract and interview points

equals must be reflexive, symmetric, transitive, consistent, and false for `null`. Equal objects must have equal hash codes; unequal objects may collide. Choose identity fields that remain stable while the object is used as a hashed key.

Comparable defines a natural order; Comparator defines an external order. Sorted collections treat a zero comparison as an ordering equivalence, so consistency with equals matters for predictable Set and Map semantics. Tie breakers must be deliberate and transitive.

Use `Integer.compare`, `Long.compare`, or `Comparator.comparing` rather than subtraction. Decide how `null` values and ties should be handled, and avoid changing a priority or comparison field while an element remains inside a sorted collection or heap.

Interview question Why must `equals()` and `hashCode()` agree?

Contract: If two objects are equal according to equals, they must produce the same hash code.

`HashMap` conceptually: hashCode

```
   ↓
```

bucket

```
   ↓
```

candidate entries

```
   ↓
```

equals If equal objects have different hashes, lookup may search a different bucket and fail.

Interview question What happens if a `HashMap` key mutates?

Suppose:

```
map.put(user, value);
```

The hash is based on: user.id; Then:

```
user.id = differentValue;
```

Now: object physically remains in old bucket; new hash points elsewhere; Lookup may fail.

Therefore: `HashMap` keys should generally have stable equality/hash state while stored.

Immutable keys are ideal.

Official reference Java 25 HashMap API

2.4 Immutable objects and defensive copies

Define an immutable class. one field is a list.

What Does Immutable Mean?

After construction: The externally observable state of the object cannot change.

Example:

```
String name = "Java";
```

Operations don't mutate the existing `String`.

They produce another value/object as needed.

Typical Immutable Class

```
public final class Employee {

    private final long id;
    private final String name;

    public Employee(long id, String name) {
        this.id = id;
        this.name = name;
    }

    public long getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

Common principles:

Use private state, initialize it completely, prevent uncontrolled subclass mutation and avoid mutators or mutable aliases.

A final class and final fields help enforce the design, but mutable reachable state still needs an explicit copy and ownership policy.

The `List` Trap

Wrong:

```
public final class Employee {

    private final List<String> skills;

    public Employee(List<String> skills) {
        this.skills = skills;
```

```
    }

    public List<String> getSkills() {
        return skills;
    }
}
```

Why?

```
List<String> skills = new ArrayList<>();

skills.add("Java");

Employee e = new Employee(skills);

skills.add("Python");
```

Employee now effectively observes: Java; Python; even though nobody called a setter on Employee.

Defensive Copy

Modern Java:

```
public final class Employee {

    private final List<String> skills;

    public Employee(List<String> skills) {
        this.skills = List.copyOf(skills);
    }

    public List<String> getSkills() {
        return skills;
    }
}
```

Now the internal list cannot be modified through the original list reference, and the returned list itself is unmodifiable.

`Collections.unmodifiableList()` Trap

Consider:

```
this.skills = Collections.unmodifiableList(skills);
```

This creates an unmodifiable view, not necessarily an independent copy.

Original reference:

```
List<String> original = new ArrayList<>();

original.add("Java");

Employee employee = new Employee(original);
```

```
original.add("Python");
```

If the object's field wraps original, the underlying list changed.

Contrast: `List.copyOf(original)`; which creates an unmodifiable copy where required.

Shallow vs Deep Immutability

Suppose:

```
List<Address> addresses;
```

and Address itself is mutable.

Even:

```
this.addresses = List.copyOf(addresses);
```

protects the list structure, but does not magically clone each mutable Address.

An unmodifiable list can still refer to mutable Address instances. Protecting the list structure does not freeze the elements.

For deep immutability: immutable container + immutable elements; or defensive copies of mutable elements.

Why Immutability Helps Concurrency

Immutable objects are naturally easier to share.

If state never changes: `Thread` A ----\; `Thread` B ----- > immutable object; `Thread` C ----/; there are no concurrent writes to coordinate.

Correct construction and publication still matter, but immutability dramatically simplifies reasoning.

`final` fields have special initialization guarantees when construction obeys the Java Memory Model; do not leak `this` before construction completes.

Answer to rehearse

An immutable object cannot have its observable state changed after construction. I make its state private, initialize it completely in the constructor, avoid mutators, prevent inappropriate subclass-based mutation, and never expose mutable internal state. For a `List` field I make a defensive copy, for example `List.copyOf()`. If the list contains mutable objects, I must also consider deep immutability because copying the list alone only protects the container.

Interview question Why are immutable objects concurrency-friendly?

Readers do not coordinate value mutations when no such mutations are possible. Correct construction, publication and replacement of a shared reference still need their own discipline.

Follow up question How do you create an immutable Java class?

Answer: Prevent uncontrolled subclass mutation, validate constructor inputs, assign state once, avoid leaking this during construction, and defensively copy mutable inputs and outputs. The whole reachable state relevant to the value must obey the contract; a `final` class with `final` fields alone is insufficient.

Follow up question Why isn't this immutable?

Answer: The Employee example exposes or retains the caller's mutable skills list. Mutating either alias changes the apparent value. Copy the list at construction and return an unmodifiable snapshot; also copy mutable elements when the value contract requires their contents to remain fixed.

Follow up question Can immutable objects be shared safely?

Answer: A correctly constructed immutable value can be read concurrently without locks for changes to its value. Final-field initialization guarantees require construction discipline. Publication of the object reference, replacement of a shared reference, and any surrounding mutable state still need their own correct synchronization protocol.

Follow up question Does final make an object immutable?

Answer: No. A `final` reference cannot be reassigned after initialization, but the referenced list, array or other object may still mutate. Final primitive fields are fixed; deep value immutability requires controlling mutable reachable state and aliases.

```
final List<String> list = new ArrayList<>();
list.add("x");
```

`List.copyOf` rejects a `null` collection or `null` elements. It may reuse a suitable unmodifiable list rather than allocating a new container, so depend on the unmodifiable snapshot contract and not a fresh object identity. Choose and test the `null` policy explicitly.

List.copyOf contract

Checked reconstruction: immutable list value

Java 17+ record, placed as a nested or top-level record. `String` elements are immutable; `List.copyOf` takes an unmodifiable snapshot and rejects `null` elements. A list of mutable element types would require an additional element-copy policy.

```
record SkillsProfile(String name, java.util.List<String> skills) {
    SkillsProfile {
        java.util.Objects.requireNonNull(name, "name");
        skills = java.util.List.copyOf(skills);
    }
}
```

Check caller-list mutation leaves the profile unchanged, accessor mutation is rejected, `null` input is rejected, and equal profiles compare equal. This value is immutable; a shared variable that replaces it still needs publication discipline.

2.5 Generics erasure and PECS

Core contract and interview points

Generics provide compile-time type checking; erasure replaces type parameters with their bounds and can introduce casts and bridge methods. `List<String>` and `List<Integer>` do not become separate runtime classes. Use a `Class<T>` or another explicit type token when runtime type information is required.

Generic collections are invariant: `List<Integer>` is not a `List<Number>`. A producer declared ? extends `Number` can be read as `Number` but cannot safely accept an arbitrary `Number`. A consumer declared ? super `Integer` can accept `Integer` values, while reads have only the `Object` guarantee.

PECS means producer extends and consumer super. For copying values, the source produces T and the destination consumes T. Raw types suppress useful guarantees and can defer a failure to a later cast. `Arrays` are covariant and enforce element type at runtime, so incompatible stores can throw `ArrayStoreException`.

```
static <T> void copy(List<? extends T> source,
                     List<? super T> destination) {
    for (T item : source) destination.add(item);
}
```

Defend: why can you read from `List<? extends Number>` but not add `Integer` to it? The unknown concrete element type might be `Double`.

Official reference Java language types values variables and erasure

Erasure also constrains overloads: methods that differ only by `List<String>` versus `List<Integer>` parameters have the same erased signature and cannot coexist as overloads. Generic type safety does not validate every deserialized or reflectively supplied value; keep unchecked operations at a narrow, checked boundary.

Follow up question Why can't a List<Integer> be passed as a List<Number>?

Answer: That would let the receiving code insert a Double into an Integer list. Use an extends wildcard for reading produced numbers or a super wildcard for a destination that consumes Integers; select the variance from the actual operations.

Follow up question Can you overload methods only by List<String> and List<Integer>?

Answer: No. Both parameters erase to List and produce the same erased signature. Use distinct method names or another distinguishing parameter; erasure is also why an unchecked cast is not element-by-element runtime validation.

2.6 Strings the pool interning and mutable builders

`String` vs `StringBuilder` vs `StringBuffer` in Java

`Objects` of `String` are immutable, and objects of `StringBuffer` and `StringBuilder` are mutable.

StringBuilder is normally preferred for text construction confined to one thread. StringBuffer synchronizes individual operations, but compound operations can still require external coordination. Choose from ownership and atomicity requirements; measure performance when it matters.

The `StringBuilder` class does not have a `contains()` method, but it does have an `indexOf()` method. The `indexOf()` method returns -1 when the substring was not found.

String value contract and design benefits

A `String` has fixed contents after creation. Stable contents support sharing, reliable hash keys, literal reuse and stable interpretation of names and parameters. These are design benefits of the immutable value contract.

Interning of `String`

Further reading

The compiler records string constants in the class file. At runtime, literals and string-valued constant expressions use interned references; the compiler does not search a running JVM pool.

`String.intern()` returns the existing equal pooled `String` when present; otherwise it adds this `String` to the pool and returns it. It does not promise to allocate an exact copy.

Equal Strings have identical references when you retain the results of `intern()`. Calling intern does not rewrite existing variables or remove previously allocated Strings. Memory savings depend on retaining canonical results, releasing redundant objects, and the workload.

```
String s1 = new String("GFG");
```

new `String("GFG")` creates a distinct `String` object. The literal refers to an interned `String` which may already exist. The number of newly allocated objects depends on what has already been loaded and interned; it is not always two.

Use equals for `String` content equality. The == operator checks reference identity, not a portable raw memory address. Equal content does not imply identical references unless the relevant canonicalization contract applies.

Follow up question What is the `String` pool?

Answer: It supplies canonical equal `String` references for literals, string-valued constant expressions, and the results of `intern()`. new `String` can create a distinct equal object.

Calling `intern()` returns the canonical reference; retain that return value if canonical identity is required.

Follow up question Why is `String` immutable?

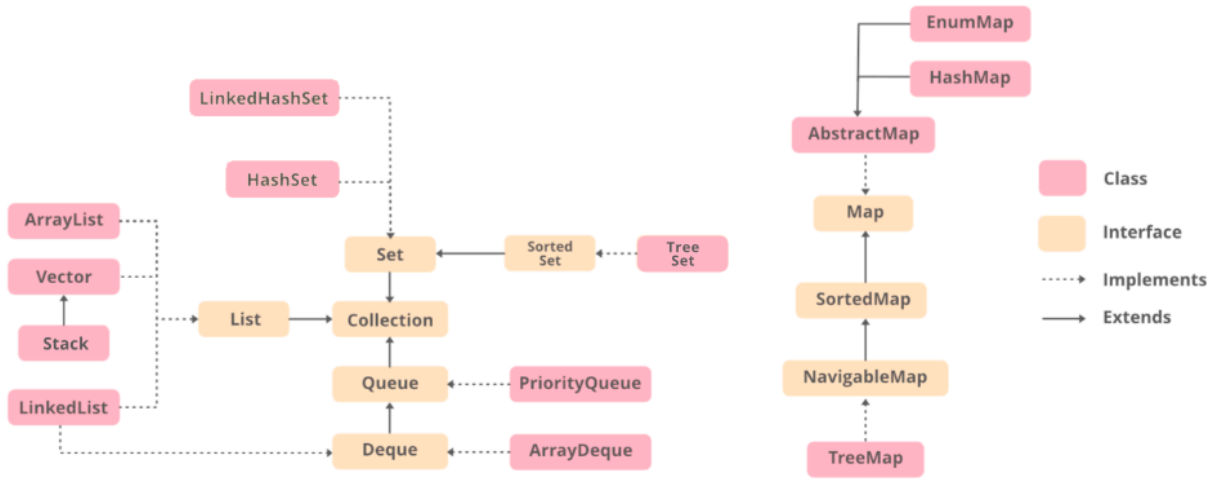
Answer: Stable contents support safe sharing of the value, reliable hash keys, literal reuse and stable interpretation of names and parameters. Immutability is a value contract, not a guarantee that a mutable variable holding the reference is safely published or that secret characters can be erased.

Official reference Java 25 String API

Immutability does not erase secrets. A password in a `String` cannot be cleared by overwriting its characters; choose credential-handling APIs deliberately. A variable holding a String reference can still change and may need synchronization.

2.7 Collection selection complexity and concurrent access

Choose collections from ordering, access pattern, atomicity, memory and ownership requirements. The hierarchy identifies relationships; the selection below identifies useful decisions.



Concurrent map contracts

Type	Relationship
`ConcurrentMap`	Extends `Map`
`ConcurrentNavigableMap`	Extends `ConcurrentMap` and NavigableMap
`ConcurrentHashMap`	Implements `ConcurrentMap`; does not promise sorted keys
`ConcurrentSkipListMap`	Implements `ConcurrentNavigableMap`; supports sorted navigation
`Collection` type	**Purpose**

Type	Relationship
HashMap / LinkedHashMap	Hash lookup; use LinkedHashMap when insertion or access order matters. Neither is inherently thread-safe.
TreeMap	Sorted keys and navigation with O(log n) lookup and update; comparator consistency affects key identity.
ConcurrentHashMap	Concurrent hash lookup and documented per-key atomic operations; no multi-key transaction or sorted iteration.
ConcurrentSkipListMap	Concurrent sorted navigation with expected average O(log n) operations; use only when key order is required.
ArrayList	Constant-time indexed access and amortized constant-time append; middle insertion or removal shifts a suffix.
ArrayDeque	Usually the first queue or stack choice for one owner; end operations are generally amortized constant time.
ConcurrentLinkedQueue / Deque	Concurrent unbounded nonblocking queues; thread safety does not provide an admission bound.
CopyOnWriteArrayList	Snapshot iteration for small, read-heavy, rarely changed lists; writes copy the backing array.

Synchronize `HashMap`

Further reading

A monitor allows one owning thread at a time. Mutual exclusion protects a shared invariant only when all relevant accesses use that same monitor; other synchronization mechanisms have their own access rules.

The `synchronizedMap()` method of java.util.`Collections` class is used to return a `synchronized` (thread-safe) map backed by the specified map. In order to guarantee serial access, it is critical that all access to the backing map is accomplished through the returned map.

Choose `Collections.synchronizedMap` when its wrapper monitor and external compound-operation locking fit the requirements. Choose `ConcurrentHashMap` for documented per-key atomic operations and concurrent access. Neither choice alone creates a transaction over several keys or mutable values, and the choice is not determined by a simple reads-versus-writes rule.

`Collections.synchronizedMap` uses a shared monitor for operations through the wrapper. Modern `ConcurrentHashMap` uses a design involving CAS and coordination at bins for updates, with retrievals generally nonblocking. The old segment-locking description

applies to historical implementations and should not be presented as the modern contract. Neither approach automatically protects arbitrary multi-operation or multi-key invariants.

Modern `ConcurrentHashMap` does not have a fixed contract of 16 concurrent updates. Update concurrency depends on the keys, bins, contention, resize activity, and implementation. Use the documented API guarantees rather than a historical segment count.

`ConcurrentHashMap` iterators are weakly consistent and do not throw `ConcurrentModificationException` as a fail-fast response to concurrent map mutation. An iterator itself is intended for use by one thread at a time.

Interview question Why isn't `HashMap` thread-safe?

Concurrent modifications can violate assumptions about its state and operations.

Even simple compound logic:

```
if (!map.containsKey(key)) {
    map.put(key, value);
}
```

is a race: `Thread` A checks absent; `Thread` B checks absent; A inserts; B inserts; Use an appropriate concurrent collection and atomic APIs.

Interview question How does `ConcurrentHashMap` help?

`ConcurrentHashMap` provides thread-safe concurrent access without putting one global lock around every operation.

Modern implementations combine techniques including:

- `volatile` visibility
- CAS
- fine-grained synchronization where needed

Important APIs: `putIfAbsent()`; `computeIfAbsent()`; `compute()`; `merge()`; `replace()`; These allow compound map operations to be expressed atomically at the map-operation level.

Interview question Why is this unsafe even with ConcurrentHashMap?

```
if (!map.containsKey(k)) {
    map.put(k, v);
}
```

Because two operations are individually thread-safe but the sequence isn't atomic.

Use:

```
map.putIfAbsent(k, v);
```

or:

```
map.computeIfAbsent(k, this::create);
```

This principle generalizes: `Thread`-safe components do not automatically make compound application logic thread-safe.

Interview question Is computeIfAbsent always harmless?

No.

The mapping function should generally be:

- reasonably short
- side-effect controlled
- not recursively modifying the same map in problematic ways
- not doing unexpectedly expensive blocking work

Senior point: Atomic APIs protect state transitions, but putting slow I/O inside them can create contention and latency problems.

Interview question `CopyOnWriteArrayList` - when?

Every modification creates a new backing-array representation.

Therefore:

```
reads → cheap
writes → expensive
```

Good: configuration listeners; small subscriber sets; rare writes; many reads; Bad: large list; frequent writes

Interview question Is `HashMap.get()` safe concurrently if nobody writes?

If the map has been safely published and is never mutated afterward, concurrent reads are generally fine.

The missing piece in many answers is safe publication.

construct

```
↓
```

publish safely

```
↓
```

read-only access Immutability/read-only state greatly simplifies concurrency.

Interview question Can `ConcurrentHashMap` solve all concurrent state problems?

No.

Business invariants may span multiple keys/objects/operations.

Interview question Is `Collections.synchronizedMap()` identical to `ConcurrentHashMap`?

No.

Their concurrency designs and iteration/compound-operation semantics differ significantly.

Interview question Can `ConcurrentModificationException` be relied upon for thread safety?

Absolutely not.

It's a fail-fast/best-effort behavior of certain iterators, not a synchronization mechanism.

Official reference Java 25 ConcurrentHashMap API

`Set` selection and iteration

`HashSet` uses hashing with expected constant-time basic operations under suitable hash distribution; `TreeSet` provides sorted operations with logarithmic cost; `LinkedHashSet` preserves encounter order. Stable equality or comparison fields matter just as for map keys. `EnumSet` and `EnumMap` specialize storage for one enum type. None of these choices alone makes arbitrary concurrent mutation safe.

When traversing a `Collections.synchronizedMap` view, synchronize on the returned map for the complete traversal, or take a protected snapshot. Locking an entrySet object is not the documented wrapper monitor. `ConcurrentHashMap` iteration is weakly consistent, not a transactionally consistent snapshot; mutable values still need their own discipline.

`ConcurrentHashMap` rejects `null` keys and values. Its computeIfAbsent computation is atomic for that map operation, but it is not a once-ever side-effect guarantee: a `null` result, an exception, removal or a later absence can allow another computation. Keep callbacks short and avoid modifying that same map recursively. A returned mutable value is not protected from all future races.

Synchronized-map traversal contract

HashSet operation and synchronization contract

2.8 HashMap internals and mutable keys

Hashing in Java

- `Object.hashCode()`: use to get the hashcode of a single, non-`null` object
- `Objects.hashCode()`: use to get the hashcode of a single object that might be `null`
- `Objects.hash()`: use to get a hashcode for multiple objects

`HashMap` methods

The `keySet()` method returns the `Set` of keys contained in the `Map`.

```
Set<String> actualValues = map.keySet();
```

The `entrySet()` method returns the set of key-value mappings.

```
Set<Map.Entry<String, Integer>> actualValues = map.entrySet();
```

The `values()` method returns the `Collection` of values contained in the `Map`.

```
Collection<Integer> actualValues = map.values();
```

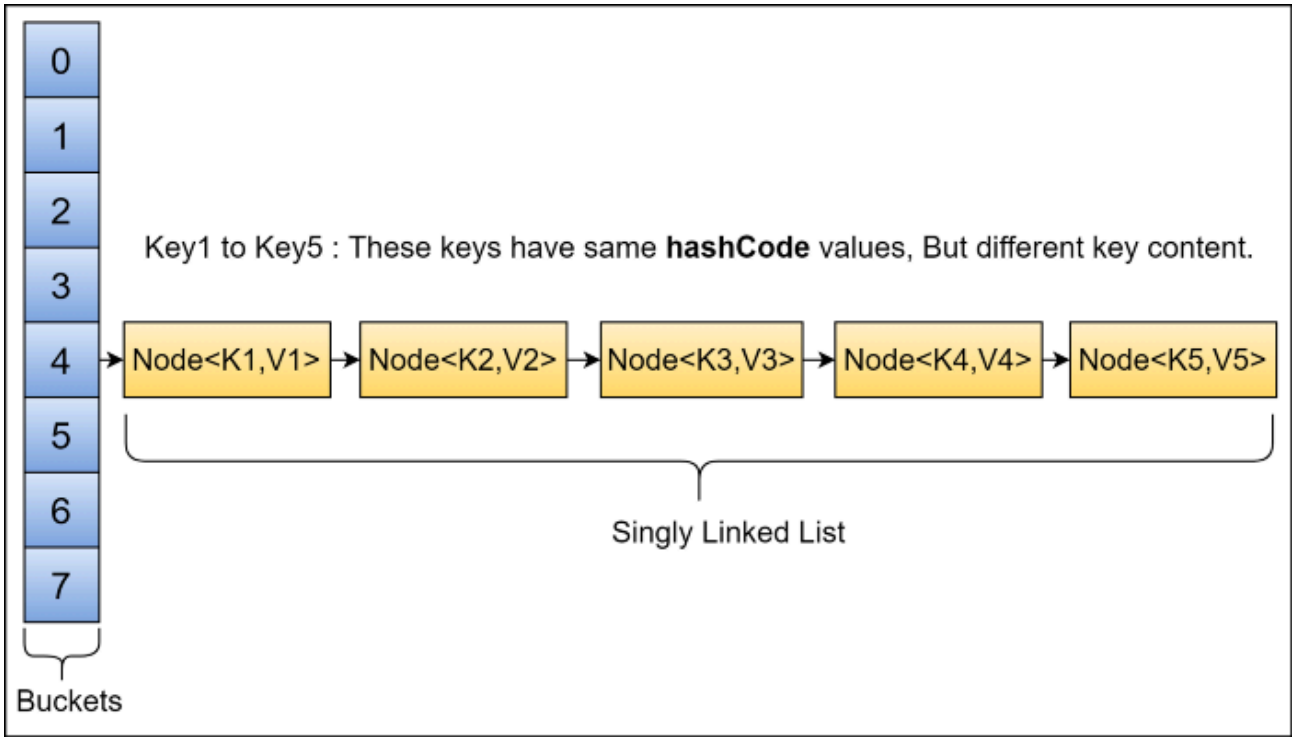
How to Create a New Entry in a `Map`

Using AbstractMap.SimpleEntry

The SimpleEntry class is a static nested class in the AbstractMap class. It provides two different constructors to initialize an instance:

```
AbstractMap.SimpleEntry<String, String> firstEntry = new
AbstractMap.SimpleEntry<>("key1", "value1");
AbstractMap.SimpleEntry<String, String> secondEntry = new
AbstractMap.SimpleEntry<>("key2", "value2");
AbstractMap.SimpleEntry<String, String> thirdEntry = new
AbstractMap.SimpleEntry<>(firstEntry);
```

HashMap Internal Working



Few important points about `HashMap` :

- `HashMap` uses its static inner class Node<K,V> for storing the entries into the map.
- `HashMap` allows at most one `null` key and multiple `null` values.
- The `HashMap` class does not preserve the order of insertion of entries into the map.
- `HashMap` distributes entries across bins using key hashes. A bin may be empty, list-based, or tree-based in common OpenJDK implementations. Capacity is the number of bins; size is the number of mappings. Resizing is based on a size threshold derived from capacity and load factor, not on how many different bins have been used.
- `HashMap` is almost similar to `Hashtable` except that it's unsynchronized and allows at max one `null` key and multiple `null` values.
- `HashMap` uses `hashCode()` and `equals()` methods on keys for the get and put operations. So `HashMap` key objects should provide a good implementation of these methods.

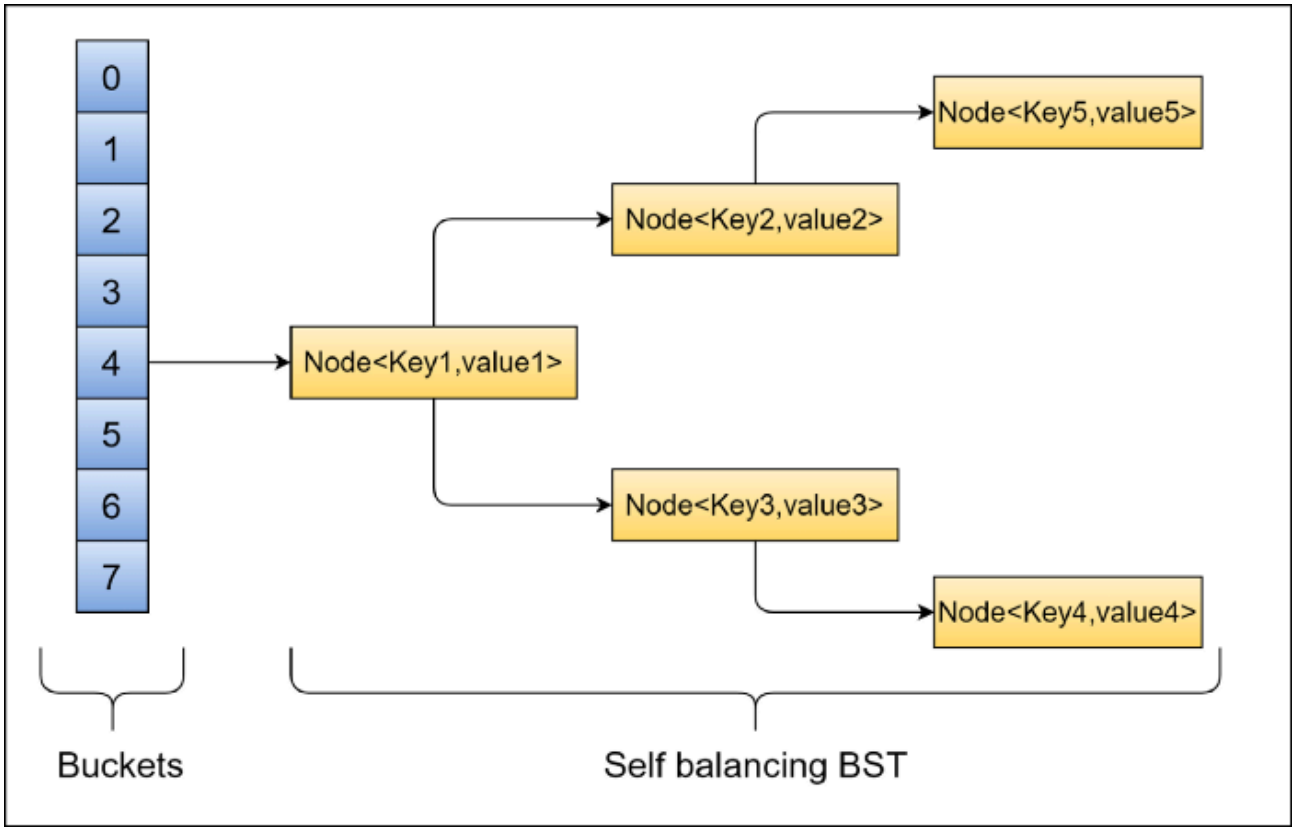
- That's why the Wrapper classes like `Integer` and `String` classes are a good choice for keys for `HashMap` as they are immutable and their object state won't change over the course of the execution of the program.

HashMap is not synchronized. Concurrent mutation requires a suitable synchronization design. A fully constructed, safely published map that is never mutated may be read concurrently; do not present every use inside a multithreaded application as forbidden.

Each Node Has the Following Structure:

```
final int hash;
final K key;
V value;
Node<K,V> next;
```

Common OpenJDK implementations use red-black tree bins for sufficiently large collision chains when table capacity is also sufficient. The familiar thresholds are implementation details, not the `Map` API contract. Tree bins improve many collision-heavy workloads, but do not claim a universal logarithmic worst case for every adversarial key and comparison pattern. Expected constant-time lookup assumes suitable hash distribution.



Official reference Java 25 HashMap API

Follow up question Does a HashMap collision mean two keys are equal?

Answer: No. A bucket can contain unequal keys. Hashing locates candidate storage and equals distinguishes the key. Test equal keys, unequal colliding keys and mutation of equality-relevant fields; do not rely on bucket structure as the API contract.

Follow up question Does treeification give every HashMap operation a guaranteed logarithmic bound?

Answer: No universal API guarantee follows from a tree-bin implementation detail. State the JDK and key behavior, distinguish expected hash lookup from collision handling, and avoid promising a worst-case bound for arbitrary adversarial key types.

2.9 PriorityQueue Deque and comparator safety

Priority Queue in Java

We use the `PriorityQueue` class to implement Heaps in Java. By default Min Heap is implemented by this class.

The default `PriorityQueue` is implemented with Min-Heap, that is the top element is the minimum one in the heap.

you can create a min-heap and max-heap in the following way:

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<Integer>(10, new
Comparator<Integer>() {
    @Override
    public int compare(Integer a, Integer b) {
        return Integer.compare(b, a);
    }
});

PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>();
```

Deque in Java

The `Deque` interface present in java.util package is a subtype of the queue interface. The `Deque` is related to the double-ended queue that supports addition or removal of elements from either end of the data structure. It can either be used as a `queue(first-in-first-out/FIFO)` or as a `stack(last-in-first-out/LIFO)`. `Deque` is the acronym for double ended queue.

Further reading

`Deque` is an interface extending `Queue`. Instantiate an implementation such as `ArrayDeque` or `LinkedList`; a class implements an interface rather than extending it as a class.

Prefer a `Deque` implementation for stack operations. For capacity rejection, `Queue.offer` returns false while add throws `IllegalStateException`; other input violations can still throw from offer. Select the failure-reporting style that fits the caller contract.

poll - This method is used to retrieve and remove the element at the head of the deque. This method returns `null` if the deque is empty.

`add()` vs `offer()`

The add versus offer distinction concerns capacity rejection, not immunity from every exception. Use the documented contract of the chosen `Queue` implementation.

For a capacity-restricted `Queue`, add reports inability to add with `IllegalStateException` while offer returns false. offer can still throw for other violations, such as an unsupported `null` element. Do not depend on a particular implementation of add delegating to offer.

Use offer when capacity rejection is an expected result and add when rejecting insertion should raise an exception.

`PriorityQueue` iteration is not sorted traversal. poll repeatedly obtains the head according to the queue comparator; ties need a deliberate policy when deterministic order matters. The queue rejects `null`, is not thread-safe, and `remove(Object)` or contains may scan linearly. Changing a comparison field in place does not reheapify the queue.

Follow up question Why is subtracting two priorities in a comparator dangerous?

Answer: The subtraction can overflow and reverse the ordering. Use Integer.compare, Long.compare or comparator builders. Add a deterministic tie rule where business order matters, and do not mutate ordering fields while an element remains in the heap.

Follow up question Does iterating a PriorityQueue return priority order?

Answer: No. Polling repeatedly returns head order according to its comparator; iteration does not promise sorted order. Copy and poll or explicitly sort for ordered output, and account for the additional time and memory.

3 Core Concurrency and bounded execution

Explain visibility and atomicity, reconstruct safe concurrent transitions, then defend admission, interruption, failure and shutdown behavior.

3.1 Atomicity visibility ordering and happens before

Core contract and interview points

A concurrency answer needs a specific relationship: a monitor unlock happens-before a subsequent lock on the same monitor, and a `volatile` write happens-before a subsequent read of that field. Publication through documented concurrent utilities also has memory-consistency guarantees. Sleeping is not a publication mechanism.

`volatile` does not make check-then-act or read-modify-write expressions atomic. Protect the entire business transition, not only each individual field access. `Long` and double atomicity questions require the distinction between the language specification and observations on a particular JVM; prefer explicit documented synchronization guarantees.

Volatile Keyword

A `volatile` write happens-before subsequent reads of that same field and constrains permitted reordering through the JMM. It does not disable all compiler optimizations or freeze every instruction around the access. It does not make a compound expression atomic.

Q: Does `volatile` make `count++` thread-safe?

No.

```
volatile int count;
count++;
```

is still: read; modify; write; Multiple threads can lose updates.

Use appropriate atomic operations or locking.

Q: What is happens-before?

A formal JMM ordering relation.

If action A happens-before action B, effects of A are guaranteed to be visible to B according to the JMM.

Examples include relationships created through:

```
monitor unlock → subsequent lock
volatile write → subsequent read
```

`Thread.start()`; `Thread.join()`; proper class initialization

Interview question What is the Java Memory Model?

This is one of the most important senior Java questions.

The JMM defines the legal behavior of multithreaded Java programs, particularly around:

- visibility
- ordering
- synchronization
- happens-before relationships

Consider:

```
boolean ready = false;
int value = 0;
```

`Thread` A:

```
value = 42;
ready = true;
```

`Thread` B:

```
if (ready) {
    System.out.println(value);
}
```

Without synchronization, you cannot reason simply: A wrote 42, therefore B must see it.

Threads, compilers and hardware operate under memory-ordering rules.

The JMM provides the contract under which cross-thread communication becomes predictable.

Interview question What are the three big concurrency concepts?

Memorize: ATOMICITY; VISIBILITY; ORDERING

Atomicity

Can an operation appear indivisible?

Visibility

When one thread writes something, when is another guaranteed to observe it?

Ordering

What ordering relationships between operations are guaranteed?

Most Java concurrency questions reduce to these three.

Interview question What is happens-before?

A happens-before relationship guarantees that: Effects of one action are visible to and ordered before another action according to the JMM.

Important examples include:

Program order

Earlier actions within one thread happen-before later actions according to the rule.

Monitor lock

An unlock on a monitor happens-before a subsequent successful lock on that same monitor.

Volatile

A write to a `volatile` variable happens-before a subsequent read of that same `volatile` variable.

`Thread` start

Actions before calling `Thread.start()` happen-before actions in the started thread.

`Thread` termination/join

Actions in a thread happen-before another thread successfully returns from joining it.

There are additional rules, but these cover enormous interview ROI.

Interview question When is `volatile` appropriate?

Good cases include state where updates don't require compound invariants.

Examples:

```
volatile boolean shutdown;
volatile Config currentConfig;
```

For example, immutable configuration publication:

```
volatile Config config;
```

Writer:

```
config = new Config(...);
```

Readers retrieve the latest published reference.

Bad case:

```
volatile int balance;

balance -= amount;
if correctness depends on a compound check/update.
```

Interview question What is instruction reordering?

Compilers and CPUs may reorder operations where permitted while preserving required semantics.

Suppose:

```
x = 1;
ready = true;
```

In a multithreaded program without proper synchronization, you cannot simply assume another thread sees all writes in source-code order.

Synchronization constructs establish ordering constraints.

Important: Reordering is not "the CPU randomly executes Java incorrectly."; The JVM and hardware may optimize while respecting the JMM.

Interview question Is `volatile` long needed because long reads are otherwise torn?

For modern Java, don't use outdated folklore as the primary reason for `volatile`.

Use `volatile` when you need its: visibility + ordering semantics The current JMM specifies atomicity properties for `volatile` long/double accesses, and modern practical Java should be reasoned about using the specification rather than old platform anecdotes.

Follow up question What does volatile guarantee?

Answer: A write to a `volatile` field happens-before subsequent reads of that same field, providing the required visibility and ordering for that relationship. Reads and writes of the `volatile` field are atomic, including `volatile` long and double. Compound actions such as increment or check-then-act remain separate operations.

Follow up question Does `volatile` make operations atomic?

Answer: It makes an individual `volatile` read or write atomic; it does not make a sequence atomic. Two `volatile` reads followed by a write do not form a transaction. Use CAS for a suitable single-value transition or one lock for a broader invariant.

Follow up question Is `i++` atomic for an int?

Answer: No. It combines a read, addition and write, so concurrent increments can overwrite each other. An int read or write being atomic does not make the compound expression atomic. Use `AtomicInteger.incrementAndGet` or the same lock around all relevant accesses.

Follow up question Does `volatile` provide mutual exclusion?

Answer: No. Multiple threads can enter the same code simultaneously. Volatile can publish a state change; it cannot protect a multi-step critical section from interleaving.

Official reference Java language threads locks and memory model

3.2 Synchronized monitors and reentrancy

`Thread` Synchronization

`Thread` interference and memory consistency errors can be avoided by ensuring the following two things-

- For monitor-based protection, competing accesses to the invariant must acquire the same monitor. One thread owns it at a time, so the protected critical sections do not interleave. Reads and writes outside that discipline remain a separate correctness problem.
- An ordinary shared-variable write does not automatically establish happens-before with another thread. A monitor unlock happens-before a subsequent lock on that same

monitor. Use that relationship, a `volatile` relationship, or a documented concurrent-utility guarantee to justify visibility.

Luckily, Java has a `synchronized` keyword using which you can synchronize access to any shared resource, thereby avoiding both kinds of errors.

```
    public synchronized void increment() {
        count = count + 1;
    }
```

`synchronized` (Protecting Shared Data)

In a multi-threaded application, two threads might try to modify the same object at the same time.

Without protection, the data can become inconsistent.

PTR uses Java's `synchronized` keyword to ensure that only one thread at a time enters critical sections of code.

Imagine two supervisors both try to modify the same PTLG simultaneously.

`Thread` A

```
↓
```

Update PTLG

`Thread` B

```
↓
```

Update PTLG Without synchronization:

- both threads read old data
- both modify it
- one update overwrites the other

This is called a race condition.

PTR protects important methods like:

```
public synchronized Result<String> createParticipant(...)
```

When one thread enters this method, Other threads trying to acquire the same monitor wait while it is held. Threads using a different monitor do not wait for this reason.

`Thread` A

```
↓
```

Acquire `Lock`

```
↓
```

Execute

```
↓
```

Release `Lock`

```
↓
```

`Thread` B starts Only one update happens at a time.

What lock is used?

For instance methods,; `synchronized`; locks the current object (this).

All `synchronized` instance methods on the same WebUIApiService instance use that instance monitor. Different instances have different monitors.

Example: `createParticipant()`

`deleteParticipant()`

`enableGroup()`

`importTradeLimits()`

`exportTradeLimits()` These `synchronized` instance methods exclude one another when invoked on the same instance, assuming the calls do acquire that monitor.

Advantage

```
✔ Easy to understand
```

Protects the chosen invariant only when all relevant access follows the same locking discipline.

```
✔ Simple implementation
```

Disadvantage

Suppose: Supervisor A creates PTLG A.

Supervisor B deletes PTLG Z.

These operations are unrelated.

Yet `Thread` B still waits because both methods use the same lock.

This reduces concurrency.

The notes identify this as a design trade-off and suggest that a more granular locking strategy, such as one lock per PTLG, could improve parallelism.

Imagine a bank with one cashier.

Even if two customers need completely different services, only one can be served at a time.

Adding more specialized counters would increase throughput.

Interview question Instance synchronized vs static synchronized?

Instance:

```
synchronized void foo() {}
```

locks the monitor associated with: this; Static:

```
static synchronized void foo() {}
```

locks the monitor associated with the: `Class` object; Therefore:

```
a.foo();
b.foo();
```

can use different instance monitors if a != b.

But static `synchronized` methods for the same class coordinate on the same class-level monitor.

Interview question What is a `synchronized` block?

Instead of locking an entire method:

```
synchronized void process() {
    // everything
}
```

you can reduce the critical section:

```
void process() {
    prepare();

    synchronized (lock) {
        updateSharedState();
    }

    finish();
}
```

Potential advantage: smaller critical section

```
↓
```

less contention

```
↓
```

better concurrency But correctness comes first.

Interview question Is `synchronized` reentrant?

Yes.

If a thread already owns a monitor, it can acquire that same monitor again.

Example:

```
synchronized void a() {
    b();
}

synchronized void b() {
}
```

Same instance: `Thread` T enters `a()`; T owns monitor; `a()` calls `b()`; T reenters same monitor; Without reentrancy, common nested `synchronized` method calls could deadlock themselves.

Follow up question What does `synchronized` provide?

Answer: Mutual exclusion through ownership of the same monitor, reentrancy for its owner, and the monitor unlock-to-subsequent-lock happens-before relationship. The invariant is protected only if competing accesses follow that monitor discipline.

Follow up question Does synchronized provide visibility?

Answer: Yes, an unlock on a monitor happens-before a subsequent lock on that same monitor. A lock on an unrelated object does not establish that relationship. Unsynchronized readers need a separate valid publication or access protocol.

Follow up question Does `synchronized` provide atomicity for its critical section?

Answer: The critical section is indivisible with respect to competing sections that acquire the same monitor. This is not a database transaction or protection against unrelated accesses, external failures or I/O side effects.

3.3 Races compound actions and state ownership

"make it thread-safe" pattern

Whenever interviewer gives shared mutable state:

```
private int count;
```

Think: 1. What state is shared?

2. What operations mutate it?

3. What must be atomic?

4. What visibility is required?

5. Is there a multi-variable invariant?

6. Can mutable sharing be removed?

Don't automatically reach for: `synchronized`; First ask whether sharing is necessary.

The easiest concurrency bug to solve is the shared state you eliminate.

How to make a method thread safe in java

Further reading

A `synchronized` method protects the chosen state only when all relevant access obeys the same locking discipline. A static `synchronized` method acquires the `Class` object monitor; an instance `synchronized` method acquires that instance monitor.

If the method is not static, then adding `synchronized` keyword will synchronize the instance of the class, not the `Class` object.

In this particular counter example, we actually can make `count++` atomic by using `AtomicInteger` from the package "java.util.concurrent.atomic".

```
import java.util.concurrent.atomic.AtomicInteger;

public class MyCounter {
      private static AtomicInteger counter = new AtomicInteger(0);

      public static int getCount() {
            return counter.getAndIncrement();
      }
}
```

Interview question What is a race condition?

A program's correctness depends on uncontrolled timing/interleaving between concurrent operations.

Example:

```
if (balance >= amount) {
    balance -= amount;
}
```

Two threads can both observe: balance = 100; amount = 80; Then both approve.

Result violates the invariant.

The important issue isn't merely the variable write.

It's the compound business invariant: check + update must be atomic

Interview question What's the difference between a data race and a race condition?

Broadly: A data race concerns conflicting unsynchronized accesses under the memory model.

A race condition is the broader correctness concept where timing/interleaving causes incorrect behavior.

In interviews, focus on the practical principle: Identify the invariant and make the entire state transition atomic - not merely individual reads/writes.

Interview question Are local variables automatically thread-safe?

If they're genuinely confined to one invocation/thread and don't reference shared mutable state, they don't themselves create shared-variable races.

But:

```
List<Order> orders = sharedList;
```

the local reference is private; the referenced object may still be shared.

Interview question Why is ownership often better than locking?

Compare: Shared mutable state

```
        ↓
```

many threads

```
        ↓
```

locks / CAS / races; versus: state

```
↓
```

one owner

```
↓
```

messages/events Single ownership reduces the number of possible interleavings.

That's a deep concurrency principle: The easiest race condition to solve is the shared mutation you eliminate.

Interview question What is your concurrency decision hierarchy?

A very strong answer: 1. Can state be immutable?

```
        ↓ no
```

2. Can state have one owner?

```
        ↓ no
```

3. Can state be partitioned?

```
        ↓ no
```

4. Can one atomic variable encode the transition?

```
        ↓ no
```

5. Use appropriate locking

```
        ↓
```

6. Measure contention and optimize only if necessary; This is generally superior to: "Use `ConcurrentHashMap` + `AtomicLong` because they're concurrent."

3.4 Safe publication final fields and construction

Interview question What is safe publication?

Suppose one thread constructs:

```
Config c = new Config(...);
```

and another thread uses it.

You need a proper happens-before relationship so the second thread sees a correctly initialized object state.

Mechanisms include:

- static initialization
- locking
- `volatile` publication
- concurrent collections
- other JMM-safe constructs

This is called safe publication.

Interview question What special guarantee do `final` fields have?

The JMM provides special initialization guarantees for properly constructed objects containing `final` fields.

If the object doesn't improperly escape during construction and is used according to the JMM, other threads can rely on final-field initialization semantics.

This is one reason immutability is powerful.

But:

```
final reference does not mean referenced object is immutable.
```

Example:

```
final List<String> names = new ArrayList<>();
```

You cannot reassign names, but:

```
names.add("Alice");
```

still mutates the list.

3.5 Atomic counters and LongAdder

Core contract and interview points

Use `AtomicLong` when a linearizable single-value operation or an exact state transition is required. `LongAdder` distributes contention for frequently updated statistics; sum is not an atomic snapshot during concurrent writes. It is unsuitable for check-then-reserve capacity enforcement.

Exact-count assertions and reset examples require quiescent writers. sumThenReset does not define an exact atomic interval while updates continue. Separate approximate telemetry from a business invariant that must never exceed a bound.

Counter trap: synchronizing only increment while checking counter < limit outside the lock permits stale reads and can exceed the limit. The bounded demo below checks the limit and increments under the same monitor, then observes every submitted task through its `Future`.

`AtomicInteger` vs `LongAdder`?

Answer:
AtomicInteger/AtomicLong maintain one atomic value using CAS.

Under very high contention, many threads compete for that same memory location.

```
LongAdder spreads updates across multiple internal cells and combines them when sum() is requested.
```

Therefore:

```
Exact atomic value needed → AtomicLong
High-contention statistics → LongAdder
Example: request counter → LongAdder.
```

Multithreading safe counter

Further reading

Unsafe compound action despite atomic increments

```
while (counter.get() < 1000) {
    counter.getAndIncrement();
}
```

Both operations are individually atomic, but competing threads can pass the limit check before either increments. The final count can exceed the bound. Protect the complete check and update with one lock or a suitable CAS transition.

Correct bounded counter demonstration

```
import java.util.*;
import java.util.concurrent.*;

final class BoundedCounter implements Runnable {
    private static final Object LOCK = new Object();
    private static final int LIMIT = 1000;
    private static int count;
    public void run() {
        while (true) {
            synchronized (LOCK) {
                if (count >= LIMIT) return;
                count++;
            }
        }
    }
    static int total() {
        synchronized (LOCK) { return count; }
    }
}

public class BoundedCounterDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService workers = Executors.newFixedThreadPool(5);
        List<Future<?>> tasks = new ArrayList<>();
        try {
            for (int i = 0; i < 5; i++) {
```

```
                tasks.add(workers.submit(new BoundedCounter()));
            }
            for (Future<?> task : tasks) task.get();
            if (BoundedCounter.total() != 1000) {
                throw new AssertionError("Bound violated");
            }
            System.out.println("PASS bounded counter 1000");
        } finally {
            workers.shutdownNow();
        }
    }
}
```

LongAdder

Let's consider some logic that's incrementing some values very often, where using an `AtomicLong` can be a bottleneck. This uses a compare-and-swap operation, which - under heavy contention - can lead to a lot of wasted CPU cycles.

`LongAdder`, on the other hand, uses a very clever trick to reduce contention between threads, when these are incrementing it.

When we want to increment an instance of the `LongAdder`, we need to call the `increment()` method. That implementation keeps an array of counters that can grow on demand.

Under contention, a `LongAdder` implementation can spread updates across cells and adapt its internal storage. Cell count is not one per caller and is not an API guarantee. sum combines the base and cells; its concurrent snapshot limits are part of the contract above.

Let's create an instance of the `LongAdder` class and update it from multiple threads:

```
LongAdder counter = new LongAdder();
ExecutorService executorService = Executors.newFixedThreadPool(8);

int numberOfThreads = 4;
int numberOfIncrements = 100;

Runnable incrementAction = () -> IntStream
  .range(0, numberOfIncrements)
  .forEach(i -> counter.increment());

for (int i = 0; i < numberOfThreads; i++) {
    executorService.execute(incrementAction);
}
executorService.shutdown();
if (!executorService.awaitTermination(5, TimeUnit.SECONDS)) {
    executorService.shutdownNow();
    throw new IllegalStateException("Writers did not finish");
}
```

`LongAdder.sum()` combines its base and cells, but is not an atomic snapshot during concurrent updates. The following exact-count assertions require all writers to have completed.

```
assertEquals(counter.sum(), numberOfIncrements * numberOfThreads);
```

Sometimes, after we call `sum()`, we want to clear all state that is associated with the instance of the `LongAdder` and start counting from the beginning. We can use the `sumThenReset()` method to achieve that:

```
assertEquals(counter.sumThenReset(), numberOfIncrements *
numberOfThreads);
assertEquals(counter.sum(), 0);
```

The zero result after reset assumes no writer is concurrently incrementing. sumThenReset is not an atomic boundary for an exact measurement interval under active writers.

Interview question `AtomicLong` vs `LongAdder`?

`AtomicLong` maintains one atomic value.

Under heavy write contention: many threads

```
    ↓
```

same CAS location

```
    ↓
```

contention

```
LongAdder distributes updates across internal cells and combines them for
reads.
```

Conceptually: `Thread` updates

```
 ↓
```

[cell0][cell1][cell2][cell3]

```
              ↓
```

sum

Choose `AtomicLong`

when you require one immediately coherent atomic value/update semantics.

Choose `LongAdder`

```
for highly contended statistics such as:
```

request count; metrics; events processed; where scalable increments matter more than treating each instantaneous `sum()` as a linearizable atomic snapshot.

Official reference Java 25 LongAdder API

3.6 Deadlock livelock starvation and lock ordering

Multithreaded batch job is deadlocked. find it.

This question separates: knows Java; from: can operate Java in production

What Is Deadlock?

Example: `Thread` A: owns `Lock` 1; waits for `Lock` 2

`Thread` B: owns `Lock` 2; waits for `Lock` 1; Diagram: A owns L1; A wants L2

```
    ↓
```

B owns L2 B wants L1

```
    ↓
```

A Neither can proceed.

Example

```
Object lockA = new Object();
Object lockB = new Object();
```

`Thread` 1:

```
synchronized (lockA) {
    synchronized (lockB) {
        // work
    }
}
```

`Thread` 2:

```
synchronized (lockB) {
    synchronized (lockA) {
        // work
    }
}
```

Potential deadlock.

Four Coffman Conditions

Classic conditions: 1. Mutual exclusion; 2. Hold and wait; 3. No preemption; 4. Circular wait; All four together permit deadlock.

Breaking one prevents this classical deadlock condition.

For practical Java code, consistent lock ordering is one of the most important techniques.

Production Question: "The Batch Is Frozen. What Do You Do?"

Don't answer: I'll inspect the code.

Start with runtime evidence.

1. Confirm lack of progress; 2. Capture thread dumps; 3. Inspect BLOCKED / waiting threads; 4. Identify owned and awaited monitors/locks; 5. Look for circular dependency; 6. Map stack frames back to code; 7. Fix locking design

Thread Dump

Common tools: jcmd <pid> Thread.print; or: jstack <pid>; Thread dump may reveal something conceptually like: Thread-1; BLOCKED; waiting to lock <Lock-B>; locked <Lock-A>

Thread-2; BLOCKED; waiting to lock <Lock-A>; locked <Lock-B>; That's extremely strong evidence.

Programmatic Detection

Java provides: ThreadMXBean; Example concept: ManagementFactory

```
    .getThreadMXBean()
    .findDeadlockedThreads();
```

Useful for diagnostics/monitoring.

Prevention - Lock Ordering

Bad:

```
Thread A: Account → Ledger
Thread B: Ledger → Account
```

Better: ALL threads:

```
Account → Ledger
```

No circular ordering.

For dynamically selected resources, define deterministic ordering, e.g. by stable ID.

ReentrantLock.tryLock()

Instead of waiting indefinitely:

```
if (lock.tryLock(2, TimeUnit.SECONDS)) {
    try {
        // critical section
    } finally {
        lock.unlock();
    }
}
```

Timeout enables controlled failure/retry strategies.

But tryLock() isn't a substitute for good lock architecture.

Deadlock vs Starvation vs Livelock

Deadlock

Everyone involved waits forever.

Starvation

A thread continually fails to obtain CPU/resource/lock access.

Livelock

Threads remain active but repeatedly react to each other without making useful progress.

Easy analogy: Deadlock: "You go."; "No, you go."; Nobody moves.

Livelock: Both repeatedly move left/right trying; to avoid each other but still cannot pass.

Starvation: Everyone else keeps getting served,; but you never do.

Answer to rehearse

First I would confirm that the batch has stopped making progress and capture multiple thread dumps using jcmd `Thread.print` or jstack. I'd inspect blocked/waiting threads, determine which locks they own and which locks they're waiting for, and look for a circular wait. Then I'd map those stack frames to the locking code. Typical fixes are consistent lock ordering, reducing nested locking, narrowing critical sections, or using timed tryLock where appropriate. I wouldn't restart first because that destroys valuable diagnostic evidence.

That last sentence sounds senior.

Interview question Livelock vs deadlock?

Deadlock

Threads cannot proceed.

Livelock

Threads are actively changing state but repeatedly interfere and make no useful progress.

Analogy: Person A moves left; Person B moves right; Both adjust; Both adjust again; ...

They're active, but nobody passes.

Interview question What is starvation?

A thread is theoretically able to run but repeatedly fails to obtain the resource/CPU opportunity it needs.

Possible causes:

- unfair locking
- priority effects
- monopolized executor
- pathological contention

Follow up question What is deadlock?

Answer: A set of participants cannot progress because each is waiting for resources or completion that another participant in the set must release or produce. `Lock` cycles are a

common form; executor tasks waiting on other queued tasks can also produce a progress failure.

Follow up question Coffman conditions?

Answer: Mutual exclusion, hold-and-wait, no preemption, and circular wait describe the classic resource deadlock conditions. Break a condition with a consistent lock order, acquiring required resources together where practical, or bounded acquisition plus correct release and recovery.

Follow up question How would you diagnose deadlock in production?

Answer: Capture thread dumps and inspect owners, waiters and lock cycles; correlate repeated dumps, JFR and executor state. JVM deadlock detection covers supported monitor and ownable-synchronizer cycles, not every application-level dependency. For virtual threads use the deployed JDK's documented dump facilities.

Follow up question Can `synchronized` methods deadlock?

Answer: Yes. Two threads can own different monitors and acquire the other in opposite order. Reentrancy avoids self-deadlock on the same monitor; it does not remove cycles across multiple monitors.

3.7 Thread lifecycle wait notify and interruption

Core contract and interview points

`Thread` states are NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. RUNNABLE can include a thread eligible to run but waiting for CPU. BLOCKED specifically describes monitor-entry waiting; explicit-lock parking can appear as WAITING.

wait releases the object monitor while waiting and requires ownership of that monitor. sleep does not release an acquired monitor. Check a wait condition in a loop because wakeups can be spurious or another thread can consume the condition before the waiter resumes. notify chooses one waiter; notifyAll wakes all to recheck their conditions.

Interruption is a cooperative cancellation signal. Code should propagate `InterruptedException` when its contract allows, or restore the flag with `Thread.currentThread().interrupt()` when it cannot propagate and then exit or follow a deliberate policy. Do not swallow interruption and continue indefinite work.

Java Concurrency

Java Concurrency / Multithreading Basics | CalliCoder Separate a task from its execution mechanism by using `Runnable` or `Callable` and an appropriate executor. Extending `Thread` is possible but usually couples task logic to thread lifecycle. Use a lambda for a simple task and choose platform or virtual threads from the workload and target JDK.

join waits for a thread to terminate; a timed join may return while that thread is still alive. Check isAlive after a timeout when completion is required. A successful termination

observation establishes the relevant happens-before relationship; simply waiting for a duration does not prove completion.

While it is easy to create one or two threads and run them, it becomes a problem when your application requires creating 20 or 30 threads for running tasks concurrently.

Also, it won't be exaggerating to say that large multi-threaded applications will have hundreds, if not thousands of threads running simultaneously. So, it makes sense to separate thread creation and management from the rest of the application.

Enter `Executors`, A framework for creating and managing threads. `Executors` framework helps you with -

- Thread Creation: It provides various methods for creating threads, more specifically a pool of threads, that your application can use to run tasks concurrently.
- `Thread` Management: It manages the life cycle of the threads in the thread pool. You don't need to worry about whether the threads in the thread pool are active or busy or dead before submitting a task for execution.
- Task submission and execution: `Executors` framework provides methods for submitting tasks for execution in the thread pool, and also gives you the power to decide when the tasks will be executed. For example, You can submit a task to be executed now or schedule them to be executed later or make them execute periodically.

Java Concurrency API defines the following three executor interfaces that covers everything that is needed for creating and managing threads -

- `Executor` - A simple interface that contains a method called `execute()` to launch a task specified by a `Runnable` object.
- `ExecutorService` - A sub-interface of `Executor` that adds functionality to manage the lifecycle of the tasks. It also provides a `submit()` method whose overloaded versions can accept a `Runnable` as well as a `Callable` object. `Callable` objects are similar to `Runnable` except that the task specified by a `Callable` object can also return a value. We'll learn about `Callable` in more detail, in the next blog post.
- `ScheduledExecutorService` - A sub-interface of `ExecutorService`. It adds functionality to schedule the execution of the tasks.

Interview question What is thread interruption?

Interruption is a cooperative cancellation/request mechanism.

```
thread.interrupt();
```

doesn't generally mean: "Kill this thread immediately."; It signals interruption status.

Blocking APIs may react by throwing: `InterruptedException`

Interview question Why is swallowing `InterruptedException` bad?

Bad:

```
try {
```

```
    queue.take();
} catch (InterruptedException e) {
    // ignore
}
```

You may destroy the caller's cancellation semantics.

Often either propagate it or, if you cannot:

```
catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    // handle/exit appropriately
}
```

Re-interrupting restores interruption status.

Exact handling depends on method semantics.

Official reference Java 25 thread states

3.8 Executors capacity queues and rejection

Core contract and interview points

`ThreadPoolExecutor` first tries to use core workers, then queues work, then creates additional workers up to maximumPoolSize when the queue refuses the task, and finally rejects when capacity is exhausted. With an unbounded queue, maximumPoolSize may provide no practical expansion beyond the core under normal queuing.

Select queue capacity, timeout, and rejection behavior from a resource budget. CallerRunsPolicy pushes work onto the submitter and can hurt an event-loop or request-thread contract. shutdown stops acceptance while allowing submitted work to finish; shutdownNow attempts interruption and returns queued tasks. Neither guarantees that tasks cooperate.

A task submitted with submit reports its exception through the returned `Future`; discarding the `Future` can hide a failure from the caller. Observe failures, expose queue delay and rejection metrics, and keep blocking dependency work out of an executor that must remain responsive.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ExecutorsExample {
    public static void main(String[] args) {
        System.out.println("Inside : " +
Thread.currentThread().getName());

        System.out.println("Creating Executor Service with a thread pool
of Size 2");
        ExecutorService executorService = Executors.newFixedThreadPool(2);
```

```
        Runnable task1 = () -> {
            System.out.println("Executing Task1 inside : " +
Thread.currentThread().getName());
            try {
                TimeUnit.SECONDS.sleep(2);
            } catch (InterruptedException ex) {
                Thread.currentThread().interrupt();
                return;
            }
        };

        Runnable task2 = () -> {
            System.out.println("Executing Task2 inside : " +
Thread.currentThread().getName());
            try {
                TimeUnit.SECONDS.sleep(4);
            } catch (InterruptedException ex) {
                Thread.currentThread().interrupt();
                return;
            }
        };

        Runnable task3 = () -> {
            System.out.println("Executing Task3 inside : " +
Thread.currentThread().getName());
            try {
                TimeUnit.SECONDS.sleep(3);
            } catch (InterruptedException ex) {
                Thread.currentThread().interrupt();
                return;
            }
        };

        System.out.println("Submitting the tasks for execution...");
        executorService.submit(task1);
        executorService.submit(task2);
        executorService.submit(task3);

        executorService.shutdown();
    }
}
```

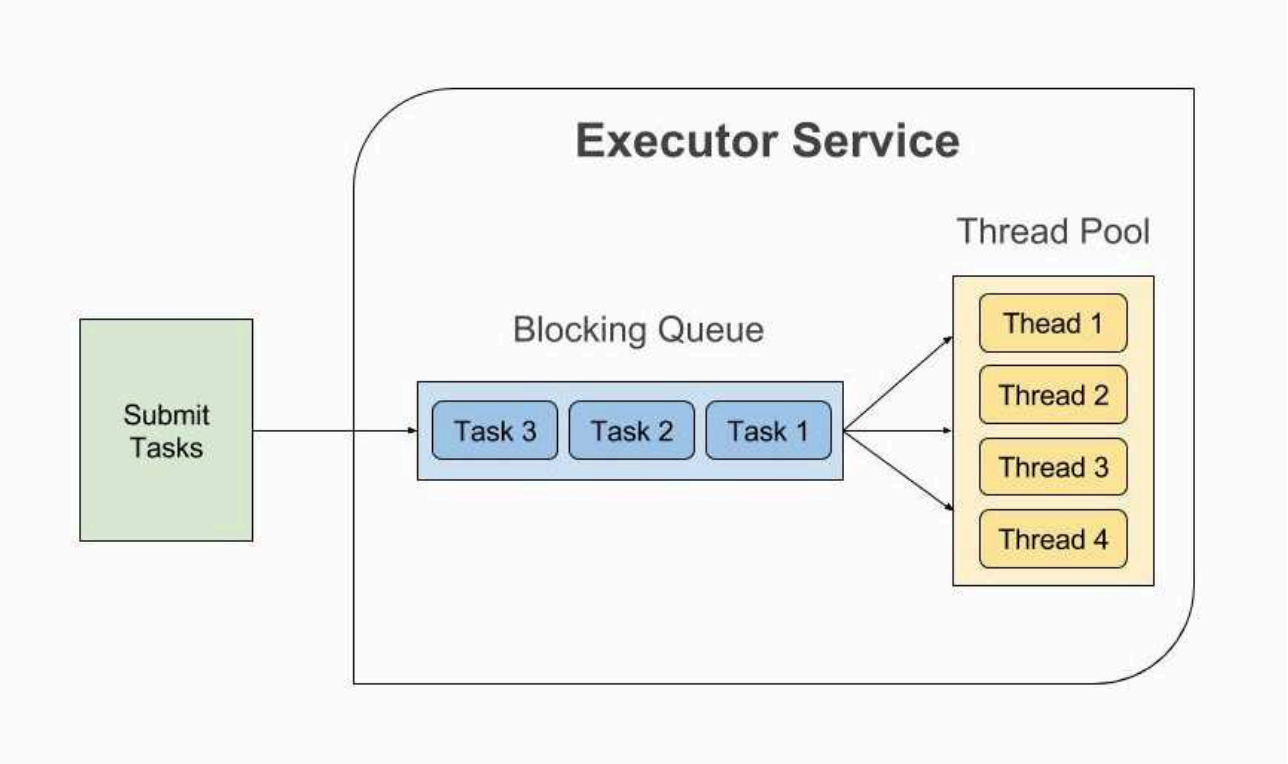
`Thread` Pool

Most of the executor implementations use thread pools to execute tasks. A thread pool is nothing but a bunch of worker threads that exist separately from the `Runnable` or `Callable` tasks and is managed by the executor.

Creating platform threads has meaningful resource cost, so suitable pools can reuse them. Virtual threads have a different cost model and normally represent individual tasks. Choose the execution mechanism and admission limit from the workload; a pool is not created once and then guaranteed never to replace workers.

We already saw an example of a thread pool in the previous section called a fixed thread-pool.

`ThreadPoolExecutor` coordinates workers with its work queue. Whether a task is queued, starts in an additional worker, or is rejected follows the core-size, queue, maximum-size and rejection-policy rules described above. A full queue alone does not imply rejection if another worker may be created.



Interview question Why use a thread pool instead of creating threads per task?

Platform threads consume resources and creation/destruction has overhead.

A pool allows controlled reuse and, critically, bounded concurrency.

Tasks

↓

`Queue`

↓

Worker pool

↓

Execution A thread pool is therefore not merely a performance optimization.

It is also: A resource-governance mechanism.

Interview question Explain `ThreadPoolExecutor`.

Key parameters: corePoolSize; maximumPoolSize; keepAliveTime; workQueue; ThreadFactory; RejectedExecutionHandler; The high-level behavior depends strongly on the queue.

Typical conceptual flow: task arrives

```
   ↓
```

workers < core?

```
   ├── yes → create worker
   └── no
       ↓
```

queue task if possible

```
       ↓ queue full?
       ├── no → wait
       └── yes
           ↓
```

workers < max?

```
       ├── yes → add worker
       └── no → reject
```

This is an excellent senior interview topic.

Interview question Why is an unbounded executor queue dangerous?

Suppose: incoming = 10,000 requests/sec; processing = 5,000 requests/sec; Backlog grows: +5,000 tasks/sec; An unbounded queue can turn overload into: growing queue

```
↓
```

growing latency

```
↓
```

memory consumption

```
↓
```

timeouts

```
↓
```

retry storms

```
↓
```

eventual failure Bounded queues force overload to become visible.

This is backpressure/resource protection.

Interview question What is rejection policy?

When: queue full + max workers reached; the executor must decide what to do.

Policies include behaviors such as:

- abort/reject
- caller runs
- discard
- discard oldest

CallerRunsPolicy can sometimes create natural throttling: producer submits

```
↓ queue full
```

producer executes task itself

```
↓
```

producer slows down But whether that's appropriate depends on the architecture.

Interview question How do you size a thread pool?

There is no universal magic formula.

CPU-bound

Approximately related to available cores.

Too many runnable CPU-bound threads: context switching + cache disruption; without more CPU throughput.

Blocking I/O

More concurrency can help because threads spend time waiting.

But the real limits include:

- DB connections
- HTTP connection pools
- downstream capacity
- memory
- latency targets
- rate limits

Senior answer: Size concurrency based on the bottleneck and downstream capacity, then validate with measurement.

Interview question What happens if your executor has 500 threads but DB pool has 20 connections?

Potentially: 500 application tasks

```
        ↓
```

20 DB connections

```
        ↓
```

480 waiting The 500 threads don't create 500-database-call throughput.

Instead they may create:

- queueing
- memory consumption
- context switching
- timeout amplification

The narrowest downstream resource often determines useful concurrency.

Interview question "More threads = more performance." True?

No.

CPU-bound: cores = 8; threads = 10,000; doesn't give 10,000-way CPU execution.

Blocking workloads may benefit from more concurrency, but downstream resources impose limits.

Performance depends on: useful parallelism; not thread count

Interview question How do thread pools relate to backpressure?

Bounded executor: incoming

```
↓
```

limited workers

```
↓
```

bounded queue

```
↓
```

reject/throttle This prevents unlimited accumulation.

An unbounded queue says: "Accept overload now and suffer later."; A bounded queue forces the system to confront capacity limits early.

Official reference Java 25 ThreadPoolExecutor API

3.9 Callable Future and completion stages

`Callable` and `Future`

While defining tasks using `Runnable` is very convenient, it is limited by the fact that the tasks can not return a result. Well, Java provides a `Callable` interface to define tasks that return a result. A `Callable` is similar to `Runnable` except that it can return a result and throw a checked exception.

Complete Java 17+ example. Save in FutureAndCallableExample.java. Worker cleanup runs on success, task failure or interruption; shutdownNow requests interruption rather than proving termination. Tasks must honor their interruption contract.

```
import java.util.concurrent.*;
```

```
public class FutureAndCallableExample {
    public static void main(String[] args)
            throws InterruptedException, ExecutionException {
        ExecutorService executor = Executors.newSingleThreadExecutor();
        try {
            Callable<String> callable = () -> {
                Thread.sleep(2000);
                return "Hello from Callable";
            };
            Future<String> future = executor.submit(callable);
            System.out.println("Caller can continue before get");
            // get waits here and reports task failure as
ExecutionException.
            System.out.println(future.get());
        } finally {
            executor.shutdownNow();
        }
    }
}
```

Follow up question What if Future.get is interrupted?

Answer: It throws InterruptedException. Propagate it when the signature permits or restore the interrupt flag before translating it. Decide separately whether to cancel the task; interruption of the waiting caller does not establish that the submitted task stopped.

Follow up question Does shutdownNow prove executor termination?

Answer: No. It attempts to interrupt active tasks and returns queued tasks that never started. Await termination with a bounded policy and require tasks to cooperate with interruption; completion of a Future and termination of the executor are different observations.

3.10 CompletableFuture composition errors and cancellation

Execution and waiting contracts

`CompletableFuture` is both a Future and a CompletionStage. It supports explicit completion and dependent-stage composition; ordinary `Future` has no built-in fluent stage-composition API, although several Futures can be coordinated manually or with executor utilities. Both can report failure and cancellation.

`get()` may block and reports computation failure through checked `ExecutionException`; `join()` may block and reports it through unchecked `CompletionException`. Neither asynchronous submission nor composition makes a later blocking wait disappear.

Non-async actions may run on the completing thread or another caller of a completion method. Async methods use the supplied executor, or normally the common ForkJoinPool when none is supplied; the documented fallback creates a thread per async task when

common-pool parallelism is below two. An Executor may run work inline. Choose execution, isolation and admission policies explicitly.

Composition and outcomes

`thenApply` transforms a successful value. `thenCompose` accepts a function returning a CompletionStage and flattens that dependent stage. `thenCombine` combines two successful stage results; it does not itself start independent tasks or guarantee parallel execution. `thenAccept` consumes a value and returns a CompletableFuture<Void>; `thenRun` needs no input value.

`allOf` completes after all supplied futures complete; it returns no collected values and does not start tasks. Read the values in a dependent stage after normal completion. If any input fails, handle the failure policy explicitly. `anyOf` yields the first completion outcome, including failure; it is not a first-success policy.

Use `exceptionally` for failure recovery, `handle` to transform success or failure, and `whenComplete` to observe an outcome. An observer that throws can affect its returned stage; observation is not a guarantee of transparency. Define whether recovery returns a valid fallback or hides a fault.

Timeout and cancellation boundaries

A stage timeout completes a future exceptionally or with a fallback; it does not prove that a socket, JDBC call or task stopped. `CompletableFuture.cancel` records cancellation; mayInterruptIfRunning has no effect in this implementation. Propagate deadlines and cooperative cancellation to the actual operation and release resources at that operation's lifecycle.

Interview question What problem does CompletableFuture solve?

It represents completion and composes dependent outcomes without manually waiting between each step. Execution capacity, context propagation and resource lifetime remain separate responsibilities.

Interview question thenApply vs thenCompose?

Transform T to U with thenApply; use thenCompose for T to CompletionStage<U> when the result should be flattened. Returning a future from thenApply instead creates a nested future.

Interview question thenCombine?

Combine two stage results with a function. Submit independent work separately and define its executors; the combinator itself is a dependency, not an execution scheduler.

Interview question What's dangerous about CompletableFuture.supplyAsync()?

The default common pool may mix unrelated work and be unsuitable for blocking service calls. A chosen executor improves ownership and isolation, but also needs admission limits. Avoid blocking joins inside a pool that must execute the awaited work.

Checked reconstruction of dependent stages

Complete Java 21 example. The inputs are in-memory computations, so it demonstrates composition rather than downstream I/O capacity. The executor owns two workers and is closed. The checks cover value transformation, flattening, combining, collection after allOf, failure recovery, observation and cancellation state.

```
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class CompletionStagesDemo {
    static void check(boolean ok) {
        if (!ok) throw new AssertionError();
    }

    public static void main(String[] args) {
        try (ExecutorService pool = Executors.newFixedThreadPool(2)) {
            var base = CompletableFuture.supplyAsync(() -> 10, pool);
            var bonus = CompletableFuture.supplyAsync(() -> 3, pool);
            var doubled = base.thenApply(n -> n * 2);
            var dependent = base.thenCompose(n ->
                CompletableFuture.supplyAsync(() -> n + 1, pool));
            var total = base.thenCombine(bonus, Integer::sum);
            var values = CompletableFuture.allOf(base, bonus)
                .thenApply(v -> List.of(base.join(), bonus.join()));
            check(doubled.join() == 20);
            check(dependent.join() == 11);
            check(total.join() == 13);
            check(values.join().equals(List.of(10, 3)));

            var failed = CompletableFuture.<Integer>failedFuture(
                new IllegalStateException("unavailable"));
            check(failed.exceptionally(ex -> -1).join() == -1);
            check(failed.handle((v, ex) -> ex == null ? v : -2)
                .join() == -2);
            AtomicInteger observed = new AtomicInteger();
            check(total.whenComplete((v, ex) ->
                observed.incrementAndGet()).join() == 13);
            check(observed.get() == 1);

            CompletableFuture<Integer> pending = new
CompletableFuture<>();
```

```
            check(pending.cancel(true));
            check(pending.isCancelled());
            try {
                pending.join();
                throw new AssertionError("expected cancellation");
            } catch (CancellationException expected) {
                check(pending.isCompletedExceptionally());
            }
        }
        System.out.println("PASS completion stages");
    }
}
```

These checks verify the stored example. They do not prove cancellation interrupts underlying work, or that a real service has bounded I/O, propagated context or correct deadlines.

Official reference Java 25 CompletableFuture API

3.11 ThreadLocal cleanup and context propagation

`ThreadLocal` solution

Store the current user once.

```
ThreadLocal<ClientInfo>
```

Any method running on the same thread can retrieve it.

No need to pass it through every method.

Imagine every worker wears a name badge.

Whenever someone asks,; "Who are you?"; they simply look at their badge.

No one has to keep asking their manager.

Every layer needs to know Current User

Without `ThreadLocal`

```
controller(User user){
service(user);

}
service(User user){
dao(user);

}
dao(User user){
helper(user);

}
```

Every method passes User

↓

User

↓

User

↓

User Called parameter plumbing.

Ugly.

`ThreadLocal` Idea

Instead `Thread`

↓

Storage

↓

User; The thread itself remembers; Current User

Imagine `Thread` 17.

`Thread` 17

↓

ClientInfo = Alice Every method can simply do

```
ClientInfoThreadLocal.get();
```

instead of pass parameter

pass parameter

pass parameter

Internal Working

Every Java `Thread` has something like `Thread`

↓

`ThreadLocalMap`

↓

`ThreadLocal`

↓

Value; Conceptually: `Thread`-1

↓

`Map`

`CurrentUser → Alice`

```
RequestId → 123
Locale → EN
```

`Thread`-2 `Map`

```
CurrentUser → Bob
RequestId → 456
```

Completely independent.

Each thread owns its ThreadLocalMap, but a stored value can still be a shared mutable object. `Thread`-local storage does not automatically make the value thread-safe.

Why `Thread` Safe?

Suppose; 100 HTTP Threads; Each has; Own `Map`; Separate `ThreadLocal` slots avoid sharing the slot itself; check whether the stored objects are actually distinct.

Therefore There is no race on independent thread-local slots, but shared values or asynchronous context handoff require their own correctness rules.

No lock

No `synchronized`

Interview question Why can `ThreadLocal` leak memory with thread pools?

Pool threads are long-lived.

Suppose: request

```
 ↓
ThreadLocal.set(largeObject)
 ↓
```

request finishes

```
 ↓
```

pool thread remains alive

```
 ↓
```

value remains associated Always clean up when appropriate:

```
try {
    context.set(value);
    process();
} finally {
    context.remove();
}
```

This is especially important in server environments.

Interview question Why can `ThreadLocal` also cause logical bugs?

Because thread pools reuse threads.

Request A: thread-7; context = Alice; If context isn't cleared: Request B may later execute on: thread-7; and accidentally observe stale context.

Therefore cleanup protects both:

- memory
- correctness/security

Follow up question What is `ThreadLocal`?

Answer: It associates a separate value with each accessing `Thread`. It is useful for thread-confined context, but platform-pool reuse can retain or expose stale context unless `remove()` runs in finally. It does not automatically propagate context across asynchronous task boundaries, and virtual-thread-per-task usage changes its cost model.

`ScopedValue` is an alternative for scoped, fixed context bindings on a Java 25 baseline; see the version boundaries in 9.1. It does not replace every mutable `ThreadLocal` use or implicitly carry context through arbitrary asynchronous work.

3.12 Risk limits atomic reservations and partition ownership

Interview question 1,000 threads update client risk limits. How would you design it?

Start with the invariant.

Suppose: usedExposure + newOrder <= maxExposure; This is a compound operation: read exposure; check limit; update exposure; It must be atomic per relevant risk entity.

A reasonable architecture: `ConcurrentHashMap`<ClientId, RiskState>

```
                ↓
```

per-client synchronization

```
                ↓
```

atomic check + update; Avoid: one global lock

```
if clients are independent.
```

This permits:

```
Client A update ─┐
Client B update ─┼─→ parallel
Client C update ─┘
```

same client

```
 ↓
```

serialized where invariant requires; Senior answer: Choose synchronization granularity from the business invariant.

Interview question Why can't you simply use `AtomicLong` exposure?

Suppose:

```
if (exposure.get() + order <= limit) {
    exposure.addAndGet(order);
}
```

Race: A reads exposure; B reads exposure; A passes check; B passes check; A adds; B adds; The individual reads and additions are atomic.

The business transaction: check + update; isn't.

Possible solutions include:

- CAS loop encoding the complete transition
- lock around check+update
- single-owner/event-loop design
- transactional state architecture

This is a quintessential senior concurrency answer.

Interview question How could CAS solve the risk update?

Conceptually:

```
static boolean tryReserve(AtomicLong exposure, long order, long limit) {
    if (order < 0 || limit < 0) {
        throw new IllegalArgumentException("negative order or limit");
    }
    while (true) {
        long current = exposure.get();
        if (current < 0 || current > limit) {
            throw new IllegalStateException("invalid exposure");
        }
        if (order > limit - current) {
            return false;
        }
        long next = current + order;
        if (exposure.compareAndSet(current, next)) {
            return true;
        }
    }
}
```

The invariant is now included in the retry loop.

But if state involves: exposure; quantity; notional; position; multiple limits; a lock or immutable-state CAS may be clearer than coordinating multiple independent atomics.

Interview question Why are multiple Atomic variables not automatically enough?

Suppose:

```
AtomicLong exposure;
AtomicLong quantity;
```

Business invariant requires both to change together.

`Thread` may observe: new exposure; old quantity; Each variable is individually atomic, but the multi-variable state transition isn't.

This leads to the core rule: Atomic variables protect variables. Business invariants often span state.

Interview question Why might a single-threaded matching/risk engine outperform multithreading?

This is extremely relevant to low-latency trading systems.

Single-thread ownership can eliminate:

- locks
- contention
- cache-line bouncing
- race conditions
- synchronization overhead
- nondeterministic interleavings

Architecture: ordered events

```
    ↓
```

single owner thread

```
    ↓
```

in-memory state

```
    ↓
```

deterministic processing Although only one thread processes that partition, performance can be excellent because each operation is extremely cheap.

Key lesson: Concurrency is not synonymous with parallelism, and parallelism is not automatically lower latency.

Interview question How would you scale a single-threaded engine?

Partition state.

Example: Orders

```
  ↓
```

partition by account/instrument/client

```
  ↓
┌──────────┬──────────┬──────────┐
│Thread 1  │Thread 2  │Thread 3  │
│part A    │part B    │part C    │
└──────────┴──────────┴──────────┘
```

Each partition preserves single ownership.

You gain parallelism between independent partitions without introducing synchronization inside each partition.

But partitioning is only valid if cross-partition invariants are handled correctly.

The tryReserve method fragment requires java.util.concurrent.atomic.`AtomicLong`. Exposure and order use the same nonnegative unit, and limit is stable for the operation. Checking remaining capacity before addition avoids long overflow, including near `Long.MAX_VALUE`. Rejection leaves state unchanged. Every writer, release and limit change must obey the same invariant; a changing limit requires an explicit coordinated policy.

3.13 Testing concurrency and shutdown behavior

Core contract and interview points

Test the invariant, failure paths, and resource lifecycle. Coordinate competing tasks with latches or barriers rather than relying on `Thread.sleep` to produce a race. A deterministic interleaving can reveal a check-then-act bug, but a passing stress run is not a proof that the algorithm is race-free.

For a bounded reservation, assert the accepted total never exceeds the limit, rejected requests do not mutate state, cancellations release a reservation exactly once, and concurrent configuration changes follow a declared policy. Test arithmetic overflow and retry behavior as separate failure triggers.

Test rejection, timeout, interruption, shutdown, and exceptional completion. Give test executors a bounded lifecycle and await termination. Use jcstress when a low-level memory-model outcome needs systematic exploration, and JMH for performance rather than timing correctness tests.

Follow up question How would you demonstrate a compound-action race reliably?

Answer: Use two controlled tasks and latches or barriers to make both observe the precondition before either commits. Assert the violated invariant, then run the same schedule against the corrected transition. Give coordination and executor shutdown bounded waits.

Follow up question What does a passing concurrency stress test establish?

Answer: It did not observe the forbidden outcome under the tested schedules and runtime. It does not prove all interleavings correct. Combine invariant reasoning, happens-before analysis, controlled failure cases and suitable low-level tests where needed.

4 Core JVM memory and performance diagnosis

Connect JVM execution and reachability to observable failures. Choose evidence, test a hypothesis and verify a correction under representative conditions.

4.1 JVM bytecode and execution model

Interview question What is the JVM?

Java source code doesn't directly tell your CPU what to execute.

Java follows roughly: Java source

```
    ↓ javac
```

Bytecode (.class)

```
    ↓
```

JVM

```
    ↓
```

Machine instructions

```
    ↓
```

CPU The JVM is the runtime that understands Java bytecode and executes it on a particular platform.

Answer to rehearse

The Java Virtual Machine is an abstract runtime machine defined by the JVM specification.

It provides:

- class loading
- bytecode verification
- runtime memory management
- execution of bytecode
- garbage collection
- synchronization primitives
- runtime optimization through JIT compilation

The key portability idea is: Java source

```
      ↓
```

platform-independent bytecode

```
      ↓
```

platform-specific JVM

```
      ↓
```

native machine code; Hence: Compile once to bytecode; execute anywhere with a compatible JVM.

Important distinction

JDK

```
├── compiler javac
├── debugger/tools
├── Java APIs
└── JVM/runtime
```

JRE conceptually

```
├── JVM
└── runtime libraries
```

JVM

```
└── executes bytecode
```

Modern Java distributions don't necessarily package a separately branded JRE, but the conceptual distinction remains useful.

Interview question Is Java compiled or interpreted?

Both.

First: Foo.java

```
   ↓ javac
```

Foo.class javac compiles Java source into JVM bytecode.

At runtime, the JVM can initially execute bytecode through an interpreter.

Frequently executed methods become candidates for JIT compilation: bytecode

```
   ↓
interpreter
   ↓
```

profiling detects hot code

```
   ↓
```

JIT

```
   ↓
```

optimized native machine code Therefore Java uses a hybrid execution model.

This is one reason long-running Java applications can become faster after warm-up.

Interview question What exactly is bytecode?

Bytecode is the JVM's platform-independent instruction format stored primarily inside .class files.

For:

```
int c = a + b;
```

the bytecode conceptually performs: load a; load b

```
integer add
```

store c; You can inspect bytecode using: javap -c MyClass; Senior interview point: Java source semantics and JVM execution are not necessarily one-to-one.

The JVM/JIT can transform execution dramatically while preserving observable program semantics.

Interview question Why doesn't Java compile directly to native machine code?

Because JVM bytecode provides portability and enables runtime optimization.

If Java directly generated one machine architecture's instructions:

```
Java → x86
```

the resulting binary wouldn't naturally run on ARM.

Instead: Java

```
 ↓
```

Bytecode

```
 ↓
x86 JVM → x86 machine code
```

Bytecode

```
 ↓
ARM JVM → ARM machine code
```

There's another advantage: the JIT knows runtime information that an ahead-of-time compiler may not know.

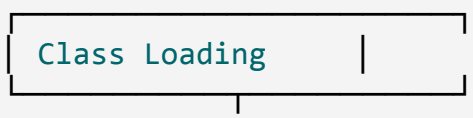
It can discover:

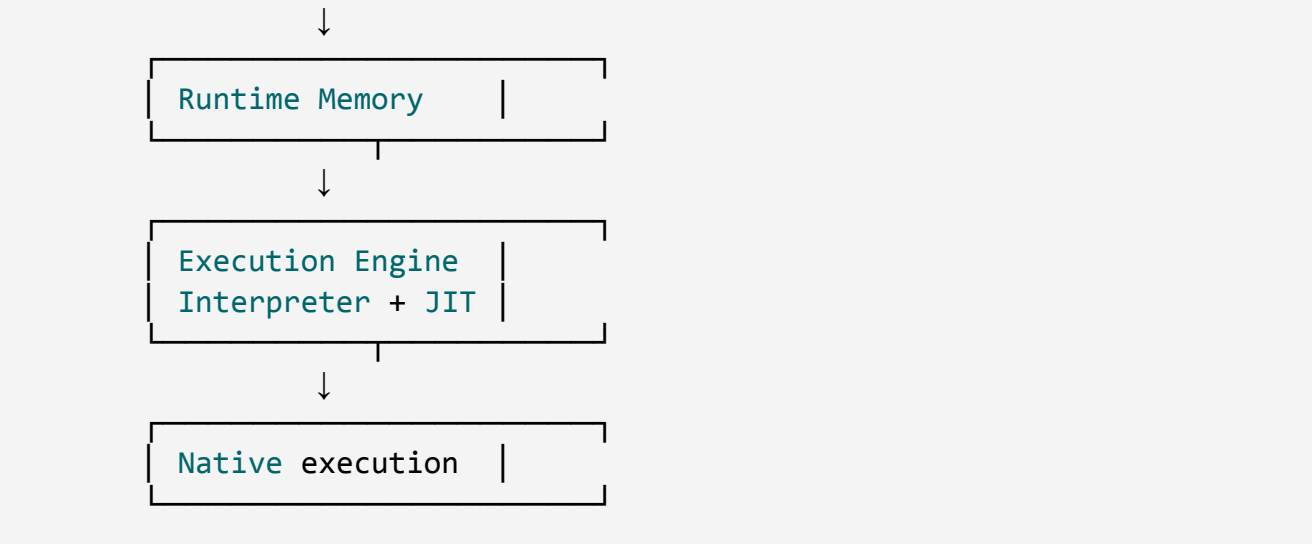
- which methods are actually hot
- common call-site types
- frequently executed branches
- allocation behavior

and optimize accordingly.

Interview question What happens when I run java MyApplication?

A startup overview includes locating and loading classes, linking them through verification, preparation and resolution, initialization when required, and invoking main. Resolution may be lazy. Allocation, interpretation, runtime profiling, JIT compilation and GC occur as execution requires; they are not one fixed global sequence after main.





GC manages heap lifecycle alongside this.

Interview question Interviewer: "What does the JVM actually buy us?"

Strong concise answer: The JVM gives Java a managed execution environment. Java source compiles into portable bytecode; the JVM loads and verifies classes, manages runtime memory, executes bytecode, dynamically optimizes hot code through JIT compilation, provides the Java Memory Model and synchronization semantics, and automatically reclaims unreachable heap objects through GC. The major engineering benefit is that application developers get portability and sophisticated runtime optimization/memory management without directly managing machine code or object deallocation.

Interview question "What does knowing JVM internals change in the code you write?"

Strong answer: It changes how I reason about production behavior rather than making me micro-optimize every statement. I understand allocation and object lifetime, avoid accidental retention, choose concurrency primitives based on visibility/atomicity requirements, avoid unbounded thread and queue growth, understand how blocking interacts with platform or virtual threads, and diagnose CPU, memory, GC and contention problems using evidence such as JFR, thread dumps, heap dumps and GC telemetry.

Excellent senior framing.

Interview question "How deep should a senior Java developer know JVM internals?"

Answer: Deep enough to predict application behavior and diagnose production failures. I should understand memory areas, object lifecycle and reachability, generational GC and our production collector, JMM/happens-before, synchronization, atomics/CAS, thread scheduling and executors, class loading, JIT fundamentals and production diagnostic tools. I don't need to memorize every bytecode opcode or collector implementation detail unless I'm working specifically on JVM/runtime engineering.

4.2 Runtime memory areas and stack frames

Stack vs Heap memory

Further reading

the major difference between Stack memory and heap memory is that the stack is used to store the order of method execution and local variables while the heap memory stores the objects and it uses dynamic memory allocation and deallocation.

Stack Memory

The JVM specifies a private stack for each thread. Physical storage, reservation and growth are implementation details; virtual-thread stacks need not correspond to one fixed native OS stack for their entire lifetime.

When a thread is created, the operating system automatically allocates the stack.

A Java stack stores method frames containing local variables and operand stacks, and unwinds with method calls. It belongs to one thread and is not globally accessible. Excessive stack growth may throw `StackOverflowError`.

The variables are visible only to the owner thread.

Heap Memory

The Java heap is a shared runtime area used for objects and arrays. `Class` metadata is typically stored in native Metaspace in HotSpot, rather than described as all JRE classes being stored in the Java heap.

The JVM manages heap reservation and commitment using platform facilities. The Java language does not specify one direct operating-system allocation call or one physical heap layout.

The heap holds objects and arrays; references can exist in stack frames, other objects, static fields, and runtime structures. Garbage collection reclaims eligible heap storage according to reachability and collector behavior. Heap exhaustion may throw `OutOfMemoryError`.

The heap is shared among threads. A shared location does not itself establish visibility of unsynchronized writes; publication and subsequent access must obey the Java Memory Model.

Q: Heap vs stack?

Heap primarily stores objects and is GC-managed. Each thread has its own JVM stack containing frames for method execution, including local-variable slots and operand-stack state. Don't oversimplify this into "references always live on stack and objects always heap"; JIT optimizations can change physical representation.

Interview question Explain JVM memory areas.

The practical mental model is: JVM PROCESS

```
│
├── Heap
│     └── objects / arrays
│
├── Metaspace
│     └── class metadata
│
├── Thread 1 Stack
│     ├── frame()
│     ├── frame()
│     └── frame()
│
├── Thread 2 Stack
│     └── frames...
│
├── Code Cache
│     └── JIT-compiled code
│
└── Native/off-heap memory
```

Heap

Shared among threads.

Usually contains objects and arrays.

Java stacks

Each thread has its own stack.

Each method invocation creates a stack frame containing things such as:

- local variables
- operand stack
- return information
- references needed by execution

Metaspace

Stores class metadata.

Since Java 8, HotSpot uses native memory for metaspace rather than the old PermGen design.

PC register

Each JVM thread has a program counter associated with execution.

Native method stack

Supports native method execution.

Interview question What does a Java stack frame contain?

Conceptually: `Method` invocation

```
      ↓
```

Stack Frame

```
  ├── Local variable array
  ├── Operand stack
  ├── reference to runtime constant-pool context
  └── execution/return bookkeeping
```

Consider:

```
int add(int a, int b) {
    int c = a + b;
    return c;
}
```

Conceptually: locals:

```
0 → this
1 → a
2 → b
3 → c
```

operand stack: load a; load b; add; store c; Each invocation receives its own frame.

Follow up question Stack vs heap?

Answer: Each JVM thread has its own stack of frames for method execution, including locals and operand-stack state. The heap is the shared runtime area for objects and arrays in the JVM model. Optimizations can eliminate allocations, so do not infer a physical allocation from every source-level new.

Follow up question Heap shared between threads?

Answer: Yes, the JVM heap is shared, but a particular object may be confined to one thread. Sharing the area does not establish safe publication, visibility or atomicity for accesses to an object.

Follow up question Stack shared?

Answer: A JVM stack belongs to one thread. A local reference stored in that stack can still point to a shared heap object. `Thread`-local reference storage therefore does not make the referenced object thread-confined.

4.3 Reachability roots cycles and collection eligibility

When is an object eligible for gc?

The Core Rule

Memorize: An object becomes eligible for garbage collection when it is no longer strongly reachable from any GC root.

Not: when its reference count reaches zero.

Java's mainstream collectors reason about reachability.

Reachability

Imagine: GC ROOT; |; v; A; |; v; B; |; v; C; A, B, C are reachable.

Now:

```
GC ROOT        A → B → C
```

with no path from any root to A.

The entire isolated graph may be collectible.

What Are GC Roots?

High-level examples include references originating from: active thread stacks / local references

```
static/class-associated roots
```

JNI references; active JVM/runtime structures; Interview-level definition: GC roots are starting points from which the JVM determines which objects remain reachable.

Cyclic References

Classic trap:

```
A → B
↑   ↓
└───┘
```

A references B.

B references A.

But neither is reachable from a GC root.

Can Java collect them?

Yes.

Because Java GC does not require reference counting.

Setting Reference to Null

```
Person p = new Person();

p = null;
```

The object may become eligible if no other strong path reaches it.

But:

```
Person a = new Person();
Person b = a;

a = null;
```

`Object` remains reachable through: b

Eligible Does NOT Mean Immediately Collected

Important distinction: unreachable

```
    ↓
```

eligible

```
    ↓
```

GC eventually decides when/how to reclaim; You cannot normally say: object gets deleted immediately.

`System.gc()`

```
System.gc();
```

is generally a request/hint for GC activity, not a portable guarantee that a particular unreachable object is immediately reclaimed.

Application correctness should never depend on it.

What are GC roots?

Answer:
GC doesn't simply ask "is this object old?" It determines whether objects are reachable from GC roots.

Typical roots include: thread stack references

```
static references
```

JNI references; JVM internal references; Anything reachable from roots remains alive.

Unreachable objects become eligible for collection.

Interview question Why doesn't Java primarily use reference counting?

Because cycles are problematic.

```
A → B
↑   ↓
└───┘
```

Suppose nothing outside the cycle references A or B.

Reference counts might remain non-zero even though the entire graph is unreachable.

Tracing collectors solve this naturally: Can a GC root reach A?

No

```
↓
```

A and B are garbage

Follow up question How does Java decide whether an object can be garbage collected?

Answer: For ordinary objects, reachability from GC roots determines liveness. An object with no strong path from a root can become eligible; soft, weak and phantom references have distinct processing rules. Eligibility does not specify the collection time.

Follow up question What are GC roots?

Answer: Examples include live thread execution state, relevant static references of live loaded classes, and JNI references. Heap analysis uses the actual runtime's root categories and paths. Identify the retaining root path rather than treating every class or object as a root.

Follow up question Can GC collect cyclic objects?

Answer: Yes. A cycle with no live root path does not keep itself alive. Reachability-based collection differs from simple reference counting. A cycle that is still reachable from a root remains live.

Follow up question Does setting an object to `null` immediately free it?

Answer: No. It removes that particular reference. Other references may retain the object, and reclamation is scheduled by the collector. Use try-with-resources for deterministic resource closure instead of depending on GC timing.

Follow up question Can calling `System.gc()` guarantee GC?

Answer: No. It requests collection effort; the API does not promise reclamation of a particular object or completion by a deadline. `Runtime` options can affect handling of explicit collection requests.

Follow up question Should production code call `System.gc()`?

Answer: Ordinary business code should not use it to manage object or external-resource lifetimes. Investigate live-set retention and allocation pressure instead. A specialized operational use needs measured evidence and awareness of the deployed collector and explicit-GC configuration.

4.4 Memory leak patterns and retained references

Can Java Have Memory Leaks Despite GC?

Absolutely.

A Java memory leak usually means: `Objects` are no longer useful to the application but remain reachable.

Example:

```
static final List<Object> cache = new ArrayList<>();
```

Continuously:

```
cache.add(new Object());
```

If entries are never removed: GC ROOT

```
  ↓
static cache
  ↓
```

millions of obsolete objects They're still reachable.

Therefore GC correctly doesn't collect them.

Answer to rehearse

Java GC works primarily from reachability. An object becomes eligible for collection when no strong path from any GC root reaches it. Cyclic references are therefore collectible if the cycle itself is unreachable. Eligibility doesn't mean immediate reclamation, and Java can still leak memory when obsolete objects remain reachable through caches, static collections, listeners, ThreadLocals, or similar structures.

Java Memory Leaks and Garbage `Collector`

Further reading

Can Java have memory leaks despite GC?

Answer:
Absolutely.

GC removes unreachable objects, not objects we no longer logically need.

Example:

```
static Map<String, Object> cache = new HashMap<>();
```

If we continuously add entries and never remove them, everything remains reachable.

That's a Java memory leak.

GC-Friendly `Arrays`

Structure

```
class GrowingMessageArray<T>{

    T[] messages;

    int size;

}
```

messages

```
↓
```

[ ][ ][ ][ ][ ][ ]

size=0

Normally, after clearing an array people do:

```
messages[i] = null;
```

This removes references.

Reference stores may require collector-specific write barriers. The cost and treatment of `null` stores depend on the collector and JVM implementation.

These are called write barriers.

Large clearing loops have a cost, but this must be weighed against retaining obsolete objects and increasing live memory.

Traditional reset

```
for(...)

messages[i]=null;
```

Looks harmless.

But internally NULL write

NULL write

NULL write

NULL write

NULL write

...

Millions of writes.

PTR approach

Instead of clearing every slot:

```
size = 0;
```

That's all.

```
Setting size to zero changes the logical view only. Old elements remain
strongly reachable through the backing array while that array is
reachable.
```

GC cannot reclaim those old elements merely because size is zero. Clear obsolete references, overwrite every retained slot within a bounded policy, or replace the backing array when appropriate.

One assignment; size = 0; is much cheaper than; 1 million `null` assignments

Instead of erasing every page in a notebook, you simply start writing again from page 1.

Interview question Common Java memory-leak patterns?

High-value examples:

Unbounded caches

```
Map<Key, Value> cache = new HashMap<>();
```

with no eviction.

Static collections

`Long`-lived root retaining short-lived objects.

Listeners not removed

Publisher

```
   ↓
```

listeners

```
   ↓
```

objects that should have died

`ThreadLocal` misuse

Especially with thread pools.

ClassLoader leaks

Common in containers/plugins/redeployment architectures.

Queues growing without bounds

Producer faster than consumer.

Keeping obsolete references

Custom collection implementations can accidentally retain removed elements.

Interview question Explain the classic obsolete-reference memory leak.

Imagine a custom stack: `Object` pop() {

```
    return elements[--size];
}
```

The logical stack removed the element, but the backing array still references it.

Better:

```
Object result = elements[--size];
elements[size] = null;
return result;
```

Without nulling: array

```
 ↓
```

obsolete object The object remains reachable.

This matters primarily in data structures that manage their own storage.

Don't blindly `null` every local variable.

Interview question Heap continuously increases. Is that automatically a memory leak?

No.

Possible healthy pattern: heap rises

```
↓
```

GC

```
↓
```

heap falls The more useful metric is the post-GC live set.

If: post-GC baseline: 2 GB; 2.3 GB; 2.7 GB; 3.1 GB; 3.8 GB; ...

then retained live data is growing.

That strongly motivates investigation.

Interview question How do you investigate a suspected heap leak?

Typical process: 1. Confirm post-GC live-set growth; 2. Capture heap dump at suitable point; 3. Analyze object histograms; 4. Find dominant object types; 5. Examine dominator tree; 6. Inspect retained size; 7. Trace paths to GC roots; 8. Identify retention owner; 9. Fix lifecycle/eviction; 10. validate under load; Tools include: jcmd; jmap where appropriate; Eclipse MAT; JFR; commercial profilers/APM

Follow up question Can Java have memory leaks even though it has GC?

Answer: Yes. `Objects` that are strongly reachable can remain retained beyond their useful lifetime: unbounded caches, listeners, queues, static collections and uncleared `ThreadLocal` values are common examples. Compare post-GC occupancy and inspect retained-size paths.

Follow up question Can a Java application leak memory without any native bug?

Answer: Yes. Pure Java code can retain an ever-growing graph through live roots. A functioning collector correctly preserves that graph. Fix ownership, boundedness or lifecycle removal, then verify that representative post-GC live-set growth stops.

Follow up question Does GC collect reachable but unused objects?

Answer: Business-level unused is not a collection signal. A strongly reachable object is ordinarily retained. `Runtime` liveness analysis can shorten the lifetime of a local reference, so use Reference.reachabilityFence where the API requires a reachability guarantee for a native-resource interaction.

4.5 Heap dumps retained size and dominators

Interview question Shallow size vs retained size?

Shallow size

Memory occupied directly by the object.

Retained size

Approximate amount of memory that could become collectible if that object became unreachable, considering objects uniquely retained through it.

Example: Cache

```
↓
Map
↓
```

1 million entries The cache object itself may be tiny.

Its retained graph may be gigabytes.

For leak analysis: Retained size is often far more informative than shallow size.

Interview question What is a dominator tree?

`Object` A dominates B if every path from GC roots to B goes through A.

Conceptually: GC Root

```
  ↓
CacheManager
  ↓
cache
  ↓
```

1,000,000 objects If CacheManager dominates that graph, removing its retention may release the entire graph.

Dominator analysis is extremely useful for finding memory-retention owners.

4.6 GC diagnosis allocation rate and heap sizing

Interview question High GC frequency but low pause time - problem?

Not necessarily.

Ask: Is latency affected?

Is CPU spent excessively on GC?

Is allocation rate abnormal?

Is throughput affected?

Is post-GC occupancy healthy?

Frequent cheap collections may be completely acceptable.

Optimize business outcomes, not GC-count vanity metrics.

Interview question Low GC frequency but huge pauses - problem?

Potentially, particularly for latency-sensitive services.

For trading/risk systems: average latency = 2 ms; occasional pause = 500 ms; The average can look excellent while p99.99 is unacceptable.

Therefore: Tail latency matters.

Interview question Why can increasing heap size make GC worse?

Larger heap can: reduce collection frequency; but potentially: increase memory footprint; increase some collection work; increase worst-case pause/recovery costs; The trade-off is collector/workload dependent.

Thus: Bigger heap isn't automatically better.

Interview question Why can too-small heap be bad?

Too-small heap can cause: frequent GC

↓

high GC CPU

↓

reduced application throughput; and eventually: allocation failure / OOM; You want enough headroom for the application's live set and allocation behavior.

Interview question What metrics would you monitor for JVM memory?

High ROI: heap used / committed / max; post-GC occupancy; allocation rate; GC frequency; GC pause duration; GC CPU; old-generation occupancy; metaspace; direct/native memory where relevant; thread count; And correlate these with application: throughput; p95/p99 latency; errors; timeouts

Interview question "GC is slow." True?

Meaningless without workload context.

Modern GC can handle enormous allocation rates efficiently.

The real questions are: Which collector?

Heap size?

Live set?

Allocation rate?

Pause target?

Throughput target?

`Object` lifetime?

Tail-latency requirement?

4.7 Performance regressions CPU waiting and bottlenecks

Batch used to take 15 minutes. now it takes 45.

This is not primarily a Java syntax question.

It's: Can you diagnose production performance without guessing?

First Rule

Never begin with: Increase thread pool size.

Never begin with: GC issue.

Never begin with: Database issue.

Begin with: What changed, and where is the additional 30 minutes being spent?

Investigation Framework

```
15 min → 45 min
```

WHY?

|; +-------+-------+; | | |; CPU WAITING DATA; | | |; GC DB volume; code locks skew; JIT network cardinality; I/O

Step 1 - Establish the Regression

Compare: last known good run; vs; slow run; Ask: Code deployment?

JDK change?

`Configuration`?

Data volume?

Input distribution?

DB changes?

Infrastructure?

Downstream latency?

`Thread`-pool settings?

GC configuration?

Traffic/contention?

A senior engineer establishes a timeline.

Step 2 - CPU Bound or Waiting?

This immediately shrinks the search space.

High CPU

Investigate: hot methods; algorithmic regression; excessive allocation; serialization; compression; busy loops; GC; Use profiling/JFR.

Low CPU + slow application

Likely waiting: database; network; locks; I/O; thread pool; connection pool; downstream service

Step 3 - `Thread` Dumps

Take several, separated in time.

Why several?

One dump is a photograph.

Multiple dumps approximate a movie.

If many threads repeatedly appear: BLOCKED on same monitor

```
→ contention.
```

If: WAITING for connection

```
→ pool pressure.
```

If: socketRead

```
→ downstream/network waiting.
```

Step 4 - GC

Inspect: allocation rate; heap occupancy; GC frequency; pause duration; promotion; old-generation pressure; Important: Don't diagnose GC merely because heap usage looks high.

High heap utilization can be normal.

Look at actual GC behavior and application impact.

Step 5 - Database

Check: query duration; query count; execution plans; indexes; locks; connection pool; N+1 queries; rows scanned; data growth; transaction duration; Example: Previously: 100,000 rows; Now: 10,000,000 rows; Same code can become dramatically slower.

Step 6 - `Thread` Pool Saturation

Suppose: 10 worker threads; All 10 block on slow database calls.

Now: incoming tasks

```
      ↓
```

executor queue

```
      ↓
```

queue

```
      ↓
```

queue Latency explodes even though CPU may remain low.

Monitor: active threads; pool size; queue depth; task duration; rejections

Step 7 - `Connection` Pool

Example: 100 worker threads; 10 DB connections; Ninety workers may spend time waiting for connections.

Adding more threads can make things worse.

This is why: More concurrency ≠ more throughput.

Once the bottleneck is saturated, additional concurrency usually increases waiting and tail latency.

Useful Tools

Know what each is for: jcmd; thread dumps; JFR; GC logs; heap dump; async-profiler; APM/tracing; application metrics; DB slow-query/execution-plan tools; Don't say: I would take a heap dump

```
for every performance issue.
```

Heap dumps are particularly useful for memory/object-retention analysis, not as the default tool for every slowdown.

Answer to rehearse

I'd treat 15-to-45 minutes as a regression investigation, not immediately tune anything. First I'd establish what changed and compare a known-good run against the slow run. Then I'd determine whether we're CPU-bound, GC-bound, blocked, I/O-bound, DB-bound, or queueing. I'd correlate application metrics, traces, thread dumps, JFR/CPU profiles, GC logs, executor and connection-pool metrics, database query performance, and input volume. Once I identify where the extra 30 minutes are spent, I'd optimize that bottleneck and verify the improvement against the same workload.

Excellent phrase:

```
Measure → isolate → explain → change → verify.
```

Interview question Production CPU suddenly hits 100%. What do you do?

Strong answer: 1. Verify process/container CPU; 2. Identify hot threads; 3. Capture multiple thread dumps; 4. Correlate thread IDs/stacks with CPU; 5. Use JFR / profiler; 6. Determine whether CPU is: - application computation; - spin loop; - lock contention; - serialization; - regex; - GC; - excessive retries; 7. Correlate with deployment/traffic changes; Tools: jcmd; JFR; async-profiler; OS/container metrics; APM

Senior signal: Don't restart immediately and destroy evidence unless availability requires it.

Interview question Application latency suddenly rises. JVM investigation?

Don't assume GC.

Check: Latency

```
├── CPU saturation?
├── GC pauses?
├── thread-pool queue?
├── lock contention?
├── DB pool exhausted?
├── downstream latency?
├── network?
├── allocation spike?
├── retry storm?
└── deployment/config change?
```

Then correlate: p50/p95/p99; CPU; GC pause; allocation rate; heap occupancy; thread counts; executor queue depth; DB connection wait; downstream latency; This is much more senior than saying: "Take a heap dump."

4.8 JFR thread dumps and diagnostic evidence

Production debugging rapid fire

Q: CPU 100%. First move?

Find which process/threads consume CPU and profile the hot execution path rather than guessing.

Think: CPU metrics

```
→ hot threads
→ JFR / profiler
→ hot methods
→ root cause
```

Q: Heap continuously increases. What next?

Determine whether it falls after major/old-generation collection.

If retained/live-set usage keeps growing: heap dump

```
→ dominator tree
→ retained size
→ GC roots
→ why objects remain reachable
```

Q: Application latency increases but CPU is low?

Investigate waiting: DB; network; locks; I/O; thread pools; connection pools; downstream latency

Q: Why collect several thread dumps?

One dump shows state at one instant.

Repeated dumps show persistent patterns.

same threads; same stacks; same lock; is much stronger evidence.

Interview question What is JFR?

Java Flight Recorder is a low-overhead event-recording facility integrated with the JVM.

It can provide information about:

- CPU execution
- allocations
- GC
- locks
- threads
- exceptions
- I/O
- JVM events

For production JVM diagnosis, JFR is one of the highest-ROI tools to know.

Interview question JFR vs heap dump vs thread dump?

Memorize:

Problem	**First useful evidence**
Deadlock	`Thread` dump
Threads blocked	`Thread` dump/JFR
CPU hotspot	JFR/profiler
Memory leak	Heap dump
Allocation storm	JFR/profiler
GC issue	GC logs/JFR
`Lock` contention	JFR/thread dumps
`Object` retention	Heap dump
Broad performance issue	Metrics + JFR

Don't collect a 20-GB heap dump for every performance problem.

Virtual-thread diagnostic coverage

For virtual-thread workloads, use the target JDK tools and include jcmd <pid> `Thread.dump_to_file` -format=json <file> or its text format. Traditional thread dumps do not provide the same virtual-thread coverage. The newer dump has different detail, so correlate it with JFR and resource metrics rather than assuming it contains every traditional monitor address or JNI statistic.

Oracle virtual-thread diagnostic commands

4.9 Throughput tail latency and allocation cost

Interview question Throughput vs latency?

Throughput

work / time; Example: 50,000 orders/sec

Latency

Time for one operation.

2 ms/request You can improve throughput while worsening tail latency.

Senior performance engineering requires knowing which objective matters.

Interview question What is tail latency?

Percentiles: p50 = median; p95 = 95% complete below this; p99 = 99%; p99.9 = 99.9%; Example: p50 2 ms; p95 4 ms; p99 15 ms; p99.9 400 ms; Average: 5 ms; might hide serious production behavior.

In financial/trading systems, tails can matter enormously.

Interview question How can GC affect tail latency?

STW events can pause application execution.

Suppose: request normally = 1 ms; GC pause = 50 ms; A request caught by that pause experiences drastically higher latency.

Therefore low-latency applications care about:

- pause distributions
- allocation behavior
- heap sizing
- collector selection
- live-set behavior

not just average GC time.

Interview question Why can adding threads make p99 latency worse?

Because additional concurrency can exceed useful parallelism or downstream capacity.

```
threads ↑
 ↓
contention ↑
queueing ↑
DB waits ↑
context switches ↑
cache locality ↓
 ↓
p99 ↑
```

More concurrency is useful only until a limiting resource saturates.

Interview question Why isn't average latency enough?

Because users and distributed dependencies experience individual requests.

99 requests = 2ms; 1 request = 5 sec; Average can conceal catastrophic tails.

Track: p50; p95; p99; p99.9; according to service objectives.

4.10 Backpressure queue growth timeouts and Little law

Little's Law - Useful Senior Intuition

At steady state: L = λW; where roughly: L = concurrent work in system; λ = throughput; W = average time in system; If downstream latency increases while arrival rate stays high, concurrent work/queues grow.

This explains many production latency cascades.

Interview question `Service` becomes slower as traffic rises. What's your mental model?

Think queueing: arrival rate λ

```
      ↓
```

service

```
      ↓
```

service capacity μ As arrival rate approaches sustainable service capacity:

```
queueing delay ↑ dramatically
```

Eventually: λ > μ

```
↓
```

queue grows

```
↓
```

latency grows

```
↓
```

timeouts

```
↓
```

retries

```
↓
```

more traffic

```
↓
```

collapse Therefore resilient systems need:

- bounded concurrency
- bounded queues
- timeouts

- load shedding
- backpressure
- careful retries

This is beyond JVM trivia and exactly what senior interviews reward.

Interview question Why is an unbounded queue effectively deferred failure?

Because overload doesn't disappear.

arrival > processing means backlog must grow somewhere.

An unbounded queue converts immediate rejection into: memory growth + ever-increasing latency; Eventually the system still fails, often more catastrophically.

Interview question Why does Little's Law matter to Java backend engineers?

At steady state: L = λW; where roughly: L = concurrent work/items in system; λ = throughput/arrival rate; W = time in system; If: 10,000 requests/sec; ×; 0.1 sec; =; ~1,000 concurrent requests; Concurrency is not arbitrary - it emerges from throughput and latency.

This helps reason about:

- connection pools
- thread counts
- queue sizes
- memory
- downstream concurrency

Interview question Why can timeout increases make an outage worse?

If downstream is saturated: requests wait longer

↓

more requests remain in-flight

↓

more threads/connections/memory occupied

↓

queues increase

↓

system saturation worsens Increasing timeout can merely allow more work to accumulate.

Timeouts are part of capacity control, not merely user patience.

Interview question Why can "async everything" be dangerous?

Because async changes where waiting happens, not whether resource limits exist.

1,000,000 async requests

↓

downstream handles 500 You still need:

- bounded concurrency
- queue control
- rate limiting
- cancellation
- deadlines
- backpressure

Otherwise you've created an asynchronous overload machine.

4.11 Optimization workflow and representative verification

Interview question What's your optimization process?

Senior answer: 1. Define objective; 2. Measure baseline; 3. Find bottleneck; 4. Form hypothesis; 5. Change one meaningful thing; 6. Measure again; 7. Verify production-relevant percentiles; Never: guess

```
↓
```

micro-optimize

```
↓
```

declare victory

Interview question What should you optimize first?

Usually algorithmic/systemic problems dominate micro-optimizations.

Example:

```
O(n²) → O(n log n)
```

is often far more important than:

```
stream → loop
```

Likewise:

```
100 DB queries → 1 query
```

usually dominates tiny JVM instruction savings.

ROI hierarchy: Architecture

```
   ↓
```

Algorithm/data structure

```
   ↓
```

I/O/network/database

```
   ↓
```

Concurrency/contention

```
    ↓
```
Allocation/GC
```
    ↓
```
micro-optimization Not universally, but an excellent default.

Interview question "What's the most important thing about JVM performance?"

A mature answer: There isn't one JVM knob. Performance is an emergent property of workload, algorithms, allocation, live-set size, concurrency, contention, I/O, downstream capacity, CPU caches, JIT and GC. I first define the objective - throughput or latency - measure the system, identify the actual bottleneck, and optimize that bottleneck rather than applying JVM folklore.

5 Core Streams and deterministic collection

Reconstruct pipelines with explicit equality, tie, ordering, mutability and side-effect policies; defend when a loop is clearer.

5.1 Functional interfaces lambdas and identity

Java `Function.identity()`

```
static <T> Function<T, T> identity() {
       return t -> t;
   }
```

`f(x)` = x It's useful when the API forces you to pass a function, but all you want is to keep the original value.

For example, let's say you have a `Stream`<Country>, and you want to turn it into a `Map`<`String`, Country>.

You would use

```
stream.collect(Collectors.toMap(Country::getIsoCode, Function.identity()))
```

`Collectors.toMap()` expects a function to turn the Country into a key of the map, and another function to turn it into a value of the map. Since you want to store the country itself as value, you use the identity function.

Function

```
@FunctionalInterface
public interface Function<T,R>
```

Represents a function that accepts one argument and produces a result.

This is a functional interface whose functional method is `apply(Object)`.

```
Function<String, Integer> func = x -> x.length();
        Integer apply = func.apply("mkyong");
```

`Predicate`

`Predicate`<T> is a generic functional interface that represents a single argument function that returns a boolean value (true or false). This interface available in java.util.function package and contains a `test(T t)` method that evaluates the predicate of a given argument.

```
Predicate<String> predicateTest = (s)-> s.length() > 5;
 System.out.println(predicateTest.test("Predicate"));
 Function<String, Boolean> functionTest = str -> str.length()> 5;
System.out.println(functionTest.apply("Function"));
```

A functional interface has one abstract function contract; matching public `Object` methods do not add another abstract function. Default and static methods can coexist with it. Lambdas and method references are target-typed. Captured local variables must be final or

effectively final, but a captured object can still be mutable. A lambda does not create a thread or make captured state thread-safe; execution follows the caller or executor. In a lambda, this denotes the enclosing instance, unlike this inside an anonymous class.

JLS lambda capture and execution semantics

Follow up question Must two equivalent lambda expressions have the same identity?

Answer: No. Do not depend on allocation count, reference identity or synchronization on a lambda object. Depend on its functional behavior; captured local variables must be final or effectively final, but a captured object may still be mutable.

Follow up question How do you recognize a functional interface?

Answer: It has one abstract method after the language's inheritance and Object-method rules are applied. Default and static methods do not add abstract operations. @FunctionalInterface asks the compiler to verify the intended contract.

5.2 Stream pipelines laziness and single use

JAVA STREAMS

SOURCE

```
Collection.stream()
Arrays.stream()
```

`Stream.of()` `IntStream.range()`

```
        ↓
```

FILTER `filter()`

```
        ↓
```

TRANSFORM; `map()`; `mapToInt()`; `flatMap()`

```
        ↓
```

STRUCTURE; `distinct()`; `sorted()`; `limit()`; `skip()`

```
        ↓
```

TERMINAL

SEARCH; `findFirst()`; `findAny()`

BOOLEAN; `anyMatch()`; `allMatch()`; `noneMatch()`

AGGREGATE; `count()`; `min()`; `max()`; `reduce()`

COLLECT; `toList()`; `toSet()`; `toMap()`; `joining()`

GROUP; `groupingBy()`; `partitioningBy()`

DOWNSTREAM; `counting()`; `mapping()`; `filtering()`; `summingInt()`; `averagingInt()`; `maxBy()`; `minBy()`; `collectingAndThen()`; `summarizingInt()`

```
FILTER → filter
TRANSFORM → map, flatMap
ORDER → sorted, distinct, limit, skip
SEARCH → findFirst, anyMatch, max, min
AGGREGATE → count, sum, reduce
COLLECT → toList, toSet, toMap
GROUP → groupingBy, partitioningBy
```

Then you only need to memorize 6 reusable code skeletons.

The master mental model

Imagine a conveyor belt carrying objects.

You can:

```
remove some → change some → arrange them → produce a result
```

SOURCE

```
  ↓
```

FILTER / TRANSFORM / ORDER

```
  ↓
```

TERMINAL OPERATION

```
  ↓
```

RESULT

Example:

```
employees.stream()
    .filter(e -> e.salary() > 100_000)
    .map(Employee::name)
    .toList();
```

Meaning: Employees

```
   ↓
```

Keep salary > 100k

```
   ↓
Employee → Name
   ↓
List<String>
```

A `Stream`<T> is not a collection.

A collection primarily stores objects.

A `Stream` represents a computation over a sequence of objects.

`Collection` = DATA

`Stream` = PROCESSING PIPELINE OVER DATA

A `Stream` pipeline contains: SOURCE + ZERO OR MORE INTERMEDIATE OPERATIONS + ONE TERMINAL OPERATION

Answer to rehearse

Q: What is a Java `Stream`?

A Java `Stream` is a declarative abstraction for processing sequences of elements through a pipeline. It does not itself store data. A pipeline normally consists of a source, lazy intermediate operations such as filter and map, and a terminal operation such as collect, reduce, or findFirst that triggers execution.

`Stream` api - interview floor

You do not need every API.

Remember 7 concepts.

Concept	Methods	Mental meaning
Filter / Transform	filter, map, flatMap	Keep / change
Order / Select	distinct, sorted, limit, skip	Arrange / choose
Search	findFirst, findAny	Find something
Test	anyMatch, allMatch, noneMatch	Ask yes/no
Aggregate	count, min, max, reduce	Many → one
Numeric	mapToInt, sum, average	Numeric processing
Output	toList, collect, forEach	Finish pipeline

Laziness

Building a `Stream` pipeline is like writing instructions.

The work begins only when you finally say: Execute and give me the result.

Intermediate operations are lazy.

```
Stream<Employee> stream =
    employees.stream()
        .filter(Employee::active);
```

No terminal operation yet.

Execution happens when:

```
stream.toList();
```

Answer to rehearse

Intermediate `Stream` operations are lazy. They construct a pipeline but processing starts when a terminal operation is invoked. This enables optimizations and short-circuiting.

Streams are single-use

Invalid:

```
Stream<Employee> stream = employees.stream();

stream.count();

stream.toList();
```

Once a terminal operation consumes the `Stream`, it cannot be reused.

Answer to rehearse

A `Stream` represents a single traversal/computation pipeline rather than reusable storage.

Interview question `Collection` vs `Stream`?

`Collection`: stores data.

`Stream`: represents computation over data.

Interview question Intermediate vs terminal operation?

Intermediate: returns `Stream`; lazy

Examples: filter; map; sorted

Terminal: triggers execution; produces result / side effect

Examples: toList; collect; count; reduce; findFirst

Interview question Why laziness?

Allows Java to:

- delay computation
- fuse operations
- short-circuit
- avoid unnecessary work

Interview question Can a `Stream` be reused?

No.

Official reference Java 25 Stream API

5.3 Filtering mapping and flattening

Convert 2D list to 1D `List`

Further reading

```
List<List<Integer>> listOfLists = Arrays.asList(Arrays.asList(1, 2),
Arrays.asList(3, 4));

List<Integer> faltList = listOfLists.stream().flatMap(s ->
s.stream()).collect(Collectors.toList());
```

```
System.out.println(faltList);
List<Integer> result = list.stream().flatMap(Collection::stream)
.collect(Collectors.toList());
```

Convert 2d `List` to 2D Array

Further reading

```
        List<List<Integer>> list = Arrays.asList(
                Arrays.asList(1, 3, 2),
                Arrays.asList(1, 2, 2),
                Arrays.asList(1, 2)
        );

        int[][] arr = list.stream()
                .map(l ->
l.stream().mapToInt(Integer::intValue).toArray())
                .toArray(int[][]::new);
        List<List<String>> list = Arrays.asList(
                Arrays.asList("A", "B", "C"),
                Arrays.asList("B", "D"),
                Arrays.asList("E", "F")
        );

        String[][] arr = list.stream()
                .map(l -> l.stream().toArray(String[]::new))
                .toArray(String[][]::new);


String[][] arr = list.stream()
                .map(l -> l.toArray(new String[l.size()]))
                .toArray(String[][]::new);
        String[][] arr = new String[list.size()][];

        int i = 0;
        for (List<String> l: list) {
            arr[i++] = l.toArray(new String[l.size()]);
        }
```

Convert 2D Array to Lists

```
List<String> listOfAllChars = Arrays.stream(dataArray)
              .flatMap(x -> Arrays.stream(x))
              .collect(Collectors.toList());
```

flatMap()

```
        List<String> list = Arrays.asList("Geeks", "GFG",
                                  "GeeksforGeeks", "gfg");
```

```
        list.stream().flatMap(str ->
                        Stream.of(str.charAt(2))).
                        forEach(System.out::println);
String[][] array = new String[][]{{"a", "b"}, {"c", "d"}, {"e", "f"}};

  List<String> collect = Stream.of(array)     // Stream<String[]>
          .flatMap(Stream::of)                // Stream<String>
          .filter(x -> !"a".equals(x))
          .collect(Collectors.toList());

  collect.forEach(System.out::println);
```

Independent flattening fragment: list contains objects whose `getBook()` returns a non-`null` collection of non-`null` book titles. `Locale.ROOT` makes this chosen case-folding policy independent of the machine default locale; define a different policy when language-specific matching is required.

```
Set<String> collect2 = list.stream()
                    .flatMap(x -> x.getBook().stream())                //
Stream<String>
                    .filter(x ->
!x.toLowerCase(java.util.Locale.ROOT).contains("python"))
                    .collect(Collectors.toSet());
List<String> names = limitGroups.stream()
    .filter(LimitGroup::enabled)
    .map(LimitGroup::name)
    .collect(Collectors.toList());
```

CORE SOLUTION

```
nested.stream()
     .flatMap(List::stream)
     .collect(Collectors.toList());
```

MUG-UP LINE

```
listOfLists.stream()
          .flatMap(List::stream)
          .collect(toList());
```

`Filter()`

A filter is a sieve.

`Objects` enter

```
    ↓
```

`Condition`

```
    ↓
```

Only matching objects survive

```
employees.stream()
    .filter(e -> e.salary() > 100_000)
    .toList();
```

`filter()` accepts a: `Predicate<T>`

Conceptually:

```
T → boolean
```

It does not transform the object.

It decides whether the object remains in the `Stream`.

Interview question

Q: `filter()` vs `map()`?

filter = Should this element remain?

map = What should this element become?

`Map()`

A transformation machine.

Employee

```
   ↓
```

machine

```
   ↓
```

Name

```
employees.stream()
    .map(Employee::name)
    .toList();
```

`map()` applies a: `Function<T, R>`

Conceptually:

```
T → R
```

Examples:

```
Employee → String
Order → BigDecimal
String → Integer
```

`Map` vs flatmap

This is a must-not-fumble distinction.

`map()`

```
T → R
```

Example:

```
Employee → employeeName
```

`flatMap()`

Used when each input contains multiple values.

```
T → Stream<R>
       ↓
```

flatten

Example:

```
List<List<Integer>> values =
    List.of(
        List.of(1, 2),
        List.of(3, 4)
    );
```

Using map conceptually gives:

```
Stream<Stream<Integer>>
```

Using:

```
values.stream()
    .flatMap(List::stream)
    .toList();
```

gives: [1, 2, 3, 4]

Answer to rehearse

Q: `map()` vs `flatMap()`?

map performs a one-to-one transformation. flatMap is useful when one input produces multiple output elements; it transforms each input into a `Stream` and then flattens those Streams into one `Stream`.

Coding exercise - Filter + Transform

```
employees.stream()
    .filter(e -> e.salary() > 100_000)
    .map(Employee::name)
    .toList();
```

Coding exercise - Flatten

```
lists.stream()
    .flatMap(List::stream)
    .toList();
```

Memory: map

```
T → R
```

flatMap

```
T → Stream<R> → FLATTEN
```

Interview question Active employee names

```
employees.stream()
    .filter(Employee::active)
    .map(Employee::name)
    .toList();
```

Interview question map vs flatMap?

map:

```
T → R
```

flatMap:

```
T → Stream<R> → flatten
```

5.4 Distinct values equality and duplicate detection

Core contract and interview points

distinct relies on element equality; choose a stable equality contract before using it to deduplicate domain objects. Define whether duplicates are detected by the full value, by an identifier, or by a selected field.

The seen.add example below is a deliberately stateful sequential technique. Do not transfer it into a parallel pipeline with a shared `HashSet`. Prefer a clear loop or an aggregation with an explicit ordering and merge policy when the state or duplicate-detection order is part of the result.

```
public static <T> List<T> findDuplicatesByDetectionOrder(
       List<T> values) {

   Set<T> seen = new HashSet<>();

   return values.stream()
           .filter(value -> !seen.add(value))
           .distinct()
           .collect(Collectors.toList());
}

/*
* If ordering does NOT matter.
*/
public static <T> Set<T> findDuplicatesUnordered(
       List<T> values) {

   Set<T> seen = new HashSet<>();

   return values.stream()
           .filter(value -> !seen.add(value))
           .collect(Collectors.toSet());
```

```
}
```

`Distinct()`

```
employees.stream()
    .distinct()
    .toList();
```

For ordinary object Streams, duplicate detection depends on object equality semantics.

Therefore: `distinct()`

```
   ↓
```

`equals()` + `hashCode()`

Interview trap

If custom objects have incorrect equality semantics, `distinct()` may not behave as expected.

Interview question How does `distinct()` work?

Via equality semantics; `equals()` and `hashCode()` therefore matter for ordinary object Streams.

5.5 Sorting ranking limiting and skipping

```
employees.stream()
   .sorted(
       Comparator.comparingLong(Employee::getSalary)
           .reversed()
           .thenComparing(Employee::getName)
           .thenComparingInt(Employee::getId)
)
   .toList();
```

Sorting

Ascending:

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::salary)
)
    .toList();
```

Descending:

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::salary)
                  .reversed()
```

)

```
    .toList();
```

Multiple fields: `Comparator.comparing(Employee::department)`

```
          .thenComparing(Employee::salary)
```

`Comparator.comparing(X::field)`

descending

```
→ .reversed()
```

second field

```
→ .thenComparing(...)
```

`Limit()` and `skip()`

Top 3:

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::salary)
                  .reversed()
)
    .limit(3)
    .toList();
```

Skip first 3: `stream.skip(3)`

Senior-level observation

sorted + `limit(k)`

still commonly sorts the dataset: `O(n log n)`

For huge n and very small k, a heap/top-K algorithm can be better: `O(n log k)`

```
Streams do not remove the need for algorithmic thinking.
```

Coding exercise - Sort Descending

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::salary)
                  .reversed()
)
    .toList();
```

Interview question Top 3 highest-paid

```
employees.stream()
    .sorted(
        Comparator.comparing(Employee::salary)
                  .reversed()
```

)

```
    .limit(3)
    .toList();
```

Interview question Second-highest distinct salary

```
employees.stream()
    .map(Employee::salary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

Follow-up: Sorting costs approximately `O(n log n)`. For extremely large input, a one-pass top-two solution can achieve `O(n)`.

5.6 Finding matching extrema and Optional

```
Optional<LimitGroup> first = limitGroups.stream()
    .filter(LimitGroup::enabled)
    .findFirst();

boolean hasEnabled = limitGroups.stream().anyMatch(LimitGroup::enabled);

employees.stream()
       .max(Comparator.comparingDouble(Employee::getSalary));
```

Find and match

Find

```
employees.stream()
    .findFirst();

employees.stream()
    .findAny();
```

Test

`anyMatch(...); allMatch(...); noneMatch(...)`

Memory: anyMatch = does AT LEAST ONE satisfy?

allMatch = does EVERY element satisfy?

noneMatch = does NO element satisfy?

These are short-circuiting operations.

`Short`-circuiting

Suppose:

```
employees.stream()
    .filter(Employee::active)
    .findFirst();
```

Java does not necessarily process every employee.

Conceptually: Employee 1

```
   ↓
```

filter

```
   ↓
```

not active

Employee 2

```
   ↓
```

filter

```
   ↓
```

active

```
   ↓
```

`findFirst()`

```
   ↓
```

STOP

Why it matters

```
Streams operate as pipelines and can avoid unnecessary computation.
```

`Min()` / `max()`

Highest-paid employee:

```
Optional<Employee> highest =
    employees.stream()
        .max(
            Comparator.comparing(Employee::salary)
        );
```

Q: Why `Optional`<Employee>?

Because the `Stream` may be empty.

empty `Stream`

```
→ no maximum exists
```

Therefore the API cannot safely return an Employee.

`Optional` - minimum you need

Useful methods: `orElse(...)`; `orElseGet(...)`; `orElseThrow()`; `ifPresent(...)`

Important distinction: optional.orElse(`expensive()`)

evaluates `expensive()` eagerly.

optional.orElseGet(() -> `expensive()`)

evaluates lazily only if required.

Coding exercise - Maximum

```
employees.stream()
    .max(
        Comparator.comparing(Employee::salary)
    );
```

Result:

```
Optional<Employee>
```

Interview question Why `Optional` from max / findFirst?

Because a matching/result element may not exist.

5.7 Reduction versus mutable collection

```
long enabledCount =
limitGroups.stream().filter(LimitGroup::enabled).count();

long totalLimit = riskChecks.stream()
    .mapToLong(RiskCheck::limit)
    .sum();
```

`Reduce()`

Take many values and repeatedly combine them until one remains.

1 2 3 4

```
 ↓ ↓ ↓
```

combine

```
   ↓
```

10

```
int total =
    numbers.stream()
        .reduce(0, Integer::sum);
```

Mental model

```
reduce = MANY → ONE
```

Examples: sum; product; maximum; combined value

Reduce vs collect

Critical distinction.

`reduce()`

```
MANY → ONE VALUE
```

`collect()`

```
MANY → RESULT CONTAINER
```

Typical reduce: sum; product; maximum

Typical collect: `List`; `Set`; `Map`; Grouped `Map`

Answer to rehearse

`reduce()` is generally used to combine elements into a single immutable result, while `collect()` is designed for mutable reduction into structures such as Lists, Sets, Maps, or grouped results.

Interview question reduce vs collect?

reduce

```
many → value
```

collect

```
many → result structure
```

A parallel reduction needs an associative accumulator and a valid identity; the three-argument form also needs a compatible combiner. Subtraction and an identity that changes the result are common failures. Floating-point addition can change rounding with grouping. Mutating one shared container through reduce violates the intended reduction discipline; use collect for mutable accumulation.

5.8 Collectors and result mutability

Core contract and interview points

Result mutability is part of the contract. `Stream.toList()` returns an unmodifiable list. `Collectors.toList()` does not guarantee a specific list type or mutability. To require a mutable `ArrayList`, use `Collectors.toCollection(ArrayList::new)`. An unmodifiable container does not make mutable elements immutable.

State the policy for nulls, empty input, ordering, duplicate keys, and the map supplier. Choose `LinkedHashMap` when encounter order must be retained, and `TreeMap` when key order is required. Do not assume toSet or a default grouping map gives a desired iteration order.

`Collectors` - master mental model

```
Streams answer:
```

What should happen to each element?

`Collectors` answer: What final structure/result do I want?

Remember: `List` / `Set`

```
    → toList / toSet
```

one value per key

```
    → toMap
```

many values per key

```
    → groupingBy
```

true / false buckets

```
    → partitioningBy
```

calculation per group

```
    → downstream collector
```

`Collectors` - interview floor

Goal	`Collector`
`List`	`toList()`
`Set`	`toSet()`
Key → Value	`toMap()`
Group	`groupingBy()`
`Boolean` split	`partitioningBy()`
Count	`counting()`
Sum	`summingInt()`
Average	`averagingInt()`
Maximum	`maxBy()`
Minimum	`minBy()`
Transform downstream	`mapping()`
`String`	`joining()`

That is enough for the interview floor.

Official reference Java 25 Stream API

Follow up question How do you require a mutable collected ArrayList?

Answer: Use Collectors.toCollection(ArrayList::new). Stream.toList returns an unmodifiable list; Collectors.toList does not promise its type or mutability. Check element mutability separately from container mutability.

Follow up question What contract should an empty or duplicate-key input obey?

Answer: Declare the intended empty result, null policy, duplicate merge or rejection rule and iteration order before choosing a collector. Test those cases explicitly; a default map supplier is not a promise of business output order.

5.9 Lookup maps duplicate keys and merge policies

```
public static Map<String, Employee> highestPaidPerDepartment(
```

```
        List<Employee> employees) {

    return employees.stream()
            .collect(Collectors.toMap(
                    Employee::getDepartment,
                    Function.identity(),
                    (existing, candidate) ->
                            candidate.getSalary() > existing.getSalary()
                                    ? candidate
                                    : existing
            ));
}
```

Tomap()

Create:

```
employeeId → Employee
```

```
employees.stream()
    .collect(Collectors.toMap(
```

Employee::id,

```
        Function.identity()
    ));
```

Understand: `toMap(; KEY MAPPER,; VALUE MAPPER; )`

Here: Employee::id

```
Employee → ID
```

and: `Function.identity()`

```
Employee → Employee
```

`Function.identity()`; =; x -> x

It returns exactly what it receives.

Duplicate keys in tomap

Suppose:

```
Alice → IT
Bob   → IT
```

Then: toMap(; Employee::department,

```
    Function.identity()
```

)

tries:

```
IT → Alice
IT → Bob
```

Which value should survive?

Java cannot guess.

Without a merge function: `IllegalStateException`

Solution: `Collectors.toMap(; Employee::department,`

```
    Function.identity(),
    (a, b) -> a
```

)

Memory: duplicate key

```
    ↓
```

merge function

```
    ↓
```

decides winner

Tomap vs groupingby

Must be automatic.

toMap

ONE VALUE PER KEY

Example:

```
Employee ID → Employee
```

groupingBy

POTENTIALLY MANY VALUES PER KEY

Example:

```
Department → List<Employee>
```

Memory:

```
ONE / KEY  → toMap
MANY / KEY → groupingBy
```

Coding exercise - Lookup Map

```
employees.stream()
    .collect(Collectors.toMap(
```

Employee::id,

```
        Function.identity()
    ));
```

Remember:

```
identity = x → x
```

Coding exercise - Duplicate-Key `Map`

```
employees.stream()
```

```
    .collect(Collectors.toMap(
```

Employee::department,

```
        Function.identity(),
        (a, b) -> a
    ));
```

Interview question Employee ID → Employee

```
employees.stream()
    .collect(Collectors.toMap(
```

Employee::id,

```
        Function.identity()
    ));
```

Interview question toMap vs groupingBy?

toMap ONE value/key

groupingBy potentially MANY values/key

Interview question Duplicate key in toMap?

Without merge logic: `IllegalStateException`

Checked reconstruction: highest paid with a deterministic tie rule

Java 17+. Place the record and static method in a surrounding class. Salary is integral cents; highest salary wins, then lexicographically smallest ID. IDs are unique within a department, or duplicate IDs must describe the same business row. The returned `LinkedHashMap` is mutable and follows first department encounter order.

```
record PayRow(String id, String department, long salaryCents) {
    PayRow {
        java.util.Objects.requireNonNull(id, "id");
        java.util.Objects.requireNonNull(department, "department");
        if (salaryCents < 0) throw new IllegalArgumentException("salary");
    }
}

static java.util.Map<String, PayRow> highestPaidByDepartment(
        java.util.List<PayRow> rows) {
    java.util.Objects.requireNonNull(rows, "rows");
    java.util.Comparator<PayRow> best = java.util.Comparator
        .comparingLong(PayRow::salaryCents)
        .thenComparing(PayRow::id, java.util.Comparator.reverseOrder());
    return rows.stream().collect(java.util.stream.Collectors.toMap(
        PayRow::department, java.util.function.Function.identity(),
        java.util.function.BinaryOperator.maxBy(best),
        java.util.LinkedHashMap::new));
}
```

Check empty input, one department, multiple departments, salary ties, reordered tied candidates, `null` rows and negative salary. This sequential aggregation has expected `O(n)` work and `O(d)` result storage for d departments with suitable `String` hashing; do not claim an adversarial worst-case hash bound.

5.10 Grouping downstream aggregation and partitioning

```
public static Map<String, List<Employee>> groupByDepartment(
       List<Employee> employees) {

   return employees.stream()
           .collect(Collectors.groupingBy(Employee::getDepartment));
}

Map<String, Long> frequencies =
       values.stream()
               .collect(Collectors.groupingBy(
                       Function.identity(),
                       Collectors.counting()
               ));
public static Map<Boolean, List<Employee>> partitionEmployees(
       List<Employee> employees) {

   return employees.stream()
           .collect(Collectors.partitioningBy(
                   employee -> employee.getSalary() >= 100_000
           ));
}
```

Groupingby()

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department

```
    ));
```

Result:

```
Map<String, List<Employee>>
```

SQL analogy: GROUP BY department

Downstream collectors

This is the most important `Collector` pattern.

Memorize the grammar: `groupingBy(; GROUP KEY,; WHAT TO DO INSIDE EACH GROUP; )`

Example:

```
employees.stream()
```

```
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.counting()
    ));
```

Meaning: Group by department + Count each group

Replace only the downstream operation: `counting()`

`summingInt(...)`

`averagingInt(...)`

`maxBy(...)`

`minBy(...)`

`mapping(...)`

These are not six separate patterns.

They are one composition pattern.

Count by department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.counting()
    ));
```

Mental SQL: SELECT department, `COUNT(*)`; GROUP BY department

Sum by department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.summingInt(Employee::salary)
    ));
```

Average by department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.averagingInt(Employee::salary)
    ));
```

Maximum per department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

```
Employee::department,
        Collectors.maxBy(
            Comparator.comparing(Employee::salary)
)
    ));
```

Do not memorize the entire expression.

Reconstruct: GROUP department

THEN

MAX employee by salary

`Mapping()` downstream

Requirement:

```
Department → employee names
```

Without downstream mapping:

```
Department → List<Employee>
```

With:

```
employees.stream()
    .collect(Collectors.groupingBy(
Employee::department,
        Collectors.mapping(
Employee::name,
            Collectors.toList()
)
    ));
```

Mental model: GROUP

```
 ↓
```

TRANSFORM INSIDE GROUP

```
 ↓
```

COLLECT

`Partitioningby()`

Split active/inactive:

```
employees.stream()
    .collect(Collectors.partitioningBy(
Employee::active
    ));
```

Result:

```
true  → active employees
false → inactive employees
```

Difference: groupingBy

```
→ arbitrary number of groups
```

partitioningBy

```
→ Boolean groups
```

Coding exercise - Group

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department

```
    ));
```

Result:

```
Map<String, List<Employee>>
```

Coding exercise - Group + Aggregate

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.counting()
    ));
```

Replace downstream with: `counting()`

`summingInt(...)`

`averagingInt(...)`

`maxBy(...)`

`minBy(...)`

Interview question Group by department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department

```
    ));
```

Interview question Count per department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.counting()
```

```
    ));
```

Interview question Total salary per department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.summingInt(Employee::salary)
    ));
```

Interview question Highest-paid per department

```
employees.stream()
    .collect(Collectors.groupingBy(
```

Employee::department,

```
        Collectors.maxBy(
            Comparator.comparing(Employee::salary)
```

)

```
    ));
```

Interview question Frequency `Map`

```
words.stream()
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()
    ));
```

Interview question Split active / inactive

```
employees.stream()
    .collect(Collectors.partitioningBy(
```

Employee::active

```
    ));
```

Interview question groupingBy vs partitioningBy?

groupingBy arbitrary groups

partitioningBy true / false

5.11 Side effects peek and encounter order

```
riskChecks.stream().forEach(rc -> LOG.debug("check: {}", rc.name()));
```

`Peek()`

```
employees.stream()
    .peek(System.out::println)
    .filter(Employee::active)
    .toList();
```

Use primarily for: observation; debugging

Avoid: `.peek(database::save)`

Keep required business effects in an explicit operation with declared failure and transaction semantics.

Side effects

Avoid:

```
List<String> names = new ArrayList<>();

employees.stream()
    .forEach(e -> names.add(e.name()));
```

Prefer:

```
List<String> names =
    employees.stream()
        .map(Employee::name)
        .toList();
```

Why?

External mutable state makes pipelines harder to reason about and becomes dangerous with parallelism.

peek is not a guaranteed per-element audit hook: optimization and short-circuiting can avoid executing it. forEach on a parallel stream does not promise encounter order. forEachOrdered honors encounter order where one exists but does not promise that every action runs on the same thread. Keep required business effects out of incidental observation callbacks.

Follow up question Can peek be used to guarantee one database write per element?

Answer: No. Short-circuiting and pipeline optimization can skip observation callbacks. Use an explicit effect operation with ownership, failure and transaction semantics; stream syntax does not provide an exactly-once business guarantee.

Follow up question Does forEachOrdered make arbitrary side effects thread-safe?

Answer: It preserves encounter order where one exists; it does not make shared mutable state safe for unrelated callers or require one worker thread. The stream's ordered traversal and the application's state-ownership protocol are separate contracts.

5.12 Parallel streams and execution constraints

Parallel streams

`employees.parallelStream()`

does not mean: FASTER

Potentially useful when:

- dataset is sufficiently large
- work is CPU-heavy
- elements are independent
- source splits efficiently

Potentially poor when:

- workload is small
- blocking I/O dominates
- shared mutable state exists
- ordering matters
- common `ForkJoinPool` contention exists

Strong interview answer

I do not use `parallelStream()` by default. It can benefit sufficiently large CPU-bound independent workloads, but parallelism introduces splitting, coordination, ordering and common-pool considerations, so I would benchmark rather than assume it is faster.

Interview question Is `parallelStream()` faster?

Not necessarily.

Benchmark.

Interview question When should you use `parallelStream()`?

Potentially useful when:

- large enough dataset
- CPU-heavy independent operations
- easily partitionable work
- associative reductions
- benchmarking proves benefit

Avoid reflexively using it for:

- blocking DB calls
- HTTP calls
- tiny collections
- order-sensitive side effects
- latency-sensitive server paths without careful evaluation

The key question isn't: "Can this run in parallel?"; It is: "Does parallel execution improve the actual workload under production constraints?"

6 Core Exceptions resources and numeric boundaries

Make failure, cleanup, overflow, precision and time contracts explicit in the code you write.

6.1 Throwable exceptions and errors

`Throwable`, error, exception - where does oome sit?

Hierarchy

`Object`; |; `Throwable`; |; +-------------------------+; | |; `Error`
`Exception`; | |; VirtualMachineError `RuntimeException`; |
|; `OutOfMemoryError` `NullPointerException`; `IllegalArgumentException`;
...

`Throwable`

Root superclass for objects that can participate in Java's exception/error mechanism.

Two major branches: `Throwable`

```
├── Error
└── Exception
```

`Exception`

Conditions applications may reasonably deal with.

Checked Exceptions

Example: `IOException`; `SQLException`; Compiler generally requires: catch; OR; declare with throws

Unchecked Exceptions

Subclasses of: `RuntimeException`; Examples: `NullPointerException`; `IllegalArgumentException`; `IllegalStateException`; IndexOutOfBoundsException; Compiler doesn't require explicit handling.

`Error`

`Error` generally represents serious JVM/runtime/environment conditions that ordinary application code is usually not expected to recover from locally.

Examples: `OutOfMemoryError`; `StackOverflowError`; NoClassDefFoundError; AssertionError; Catching `Throwable` indiscriminately is therefore dangerous because it also catches `Error`.

Where Does `OutOfMemoryError` Sit?

`Throwable`

```
   ↓
```

`Error`

```
    ↓
```

VirtualMachineError

```
    ↓
```

`OutOfMemoryError`; Therefore: `OutOfMemoryError`; is not an `Exception`.

It is an `Error`.

What Does OOME Mean?

It does NOT always mean: heap is full.

Examples include messages associated with: Java heap space; GC overhead limit exceeded; Metaspace; unable to create native thread; Direct buffer memory; Different resource exhaustion paths require different investigation.

Should We Catch `OutOfMemoryError`?

Generally: Don't attempt normal business recovery from it.

Why?

The process may already be severely resource constrained.

Even logging, allocating objects, or constructing recovery data may fail.

At infrastructure boundaries, systems sometimes perform minimal emergency handling or diagnostics, but treating OOME like a normal business exception is a red flag.

`StackOverflowError`

Usually caused by exhausting the thread's stack, commonly through runaway recursion.

Example:

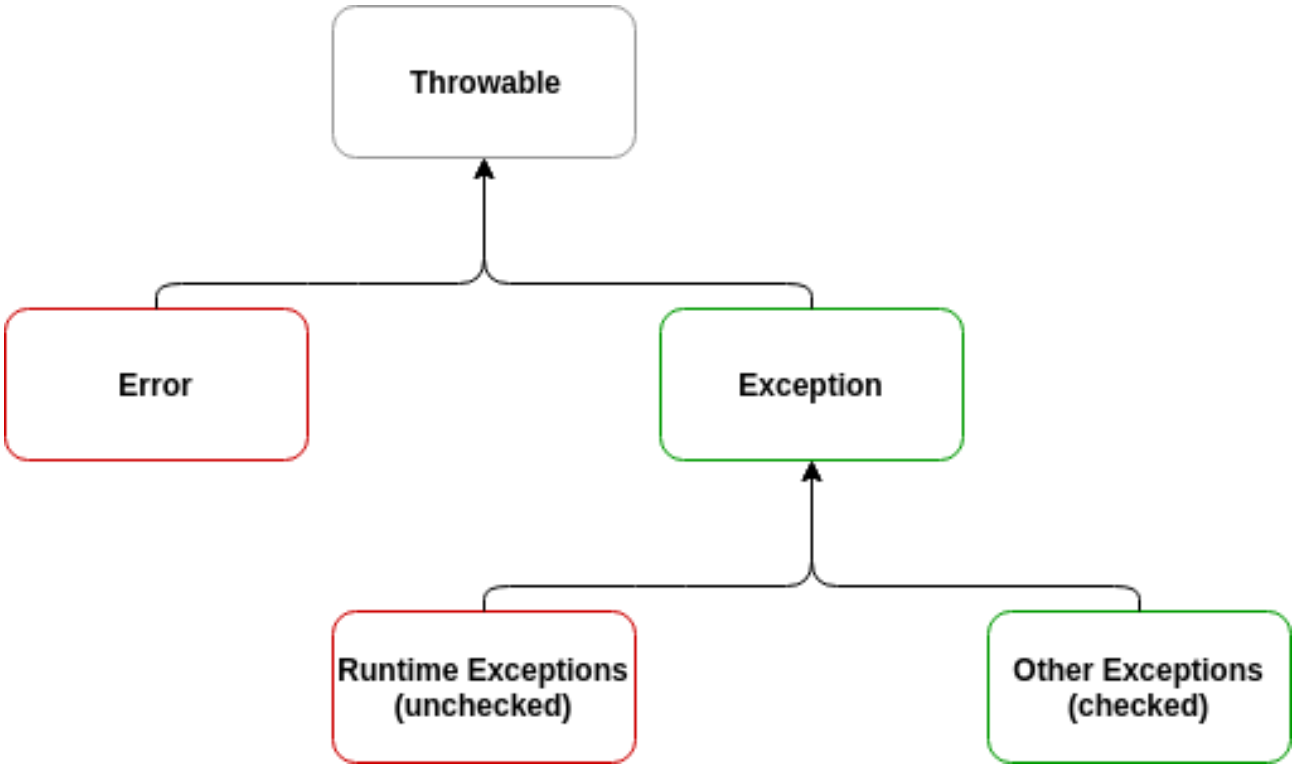
```
void recurse() {
    recurse();
}
```

Not the same as heap OOME.

Answer to rehearse

`Throwable` has two major branches: `Error` and `Exception`. Exceptions represent conditions applications may handle; RuntimeExceptions are unchecked. Errors generally represent serious JVM or runtime problems. `OutOfMemoryError` extends VirtualMachineError, which extends `Error`, so OOME is an `Error`, not an `Exception`.

Java `Exception`



In Java only throwable objects (`Throwable` objects are instances of any subclass of the `Throwable` class) can be thrown as exception. So basic data type can no be thrown at all.

```
class Main {
   public static void main(String args[]) {
      int x = 0;
      int y = 10;
      int z = y/x;
  }
}
```

Compiles fine but throws ArithmeticException exception .ArithmeticException is an unchecked exception, i.e., not checked by the compiler. So the program compiles fine.

When a subclass exception is mentioned after base class exception, then error occurs.

All the errors and exception types are subclasses of the built in class `Throwable` in the java language. `Throwable` class is the superclass of all errors and exceptions in the Java language.

Follow up question What causes `StackOverflowError`?

Answer: Exhaustion of the thread's stack, commonly through unbounded or excessively deep recursion. Inspect repeating frames and the termination condition; use an iterative algorithm when valid. Increasing stack size may move the failure threshold without fixing the recursion.

Follow up question What causes `OutOfMemoryError`?

Answer: An allocation or runtime resource requirement cannot be satisfied. Read the detail message and correlate heap, native memory, metaspace, direct buffers, thread creation and container limits. Heap exhaustion is one cause; a larger heap is not a universal remedy.

6.2 Resource lifetime and deterministic cleanup

Core contract and interview points

Use try-with-resources for `AutoCloseable` resources. Resources are closed in reverse declaration order; a close failure can be recorded as a suppressed exception when the body already threw. Garbage collection determines heap reachability and does not guarantee timely closing of files, sockets, or database connections.

finalize is deprecated for removal and is unsuitable as a resource-lifetime contract. Cleaner can provide a fallback mechanism for some native resource designs, but deterministic close remains the primary path. Clearly define ownership so only the responsible layer closes the resource.

Interview question Should `finalize()` be used for resource cleanup?

No.

Finalization is deprecated for removal and historically problematic because cleanup is nondeterministic and introduces serious lifecycle/performance issues.

For resources use:

```
try (InputStream in = ...) {
}
```

i.e. `AutoCloseable` + try-with-resources.

Interview question Why doesn't GC close files immediately?

GC manages memory reachability.

`Files`/sockets/database connections are external resources with lifecycle requirements.

You want deterministic release: open

```
↓
```

use

```
↓
```

close NOW; not: become unreachable

```
↓
```

maybe GC later

```
↓
```

eventually cleanup?

Hence try-with-resources.

Interview question Does GC manage native resources automatically like heap objects?

Not in the deterministic lifecycle sense required by files/connections.

Official reference Java 25 Object API and cleanup notes

JDBC resource and transaction ownership

For JDBC code you own, scope `Connection`, `PreparedStatement` and `ResultSet` explicitly, commonly with try-with-resources. Closing a pooled connection typically returns it to the pool; it does not replace a transaction decision. With auto-commit disabled, deliberately commit or roll back before releasing the connection. A rollback or close failure must not silently erase the original failure. Do not manually close or commit a framework-managed transaction connection. Keep Spring propagation and rollback rules at 7.3.

JDBC Connection lifetime and transaction contract

6.3 Exception policies catch ordering and translation

Core contract and interview points

Catch specific exception types before a broader type that would already catch them. Related types cannot be alternatives in the same multi-catch. An overriding method cannot broaden the checked exceptions allowed by the overridden contract; it may narrow or omit them. Unchecked exceptions do not require a throws declaration.

Translate a failure when a layer can add useful context or expose its own contract, and retain the cause. Do not return a success-shaped empty result for a failed operation unless that is an explicit domain policy. Avoid logging and rethrowing at every layer; decide who owns the diagnostic record. Keep interruption and cancellation policy at 3.7.

A return or throw from finally can replace a pending return or exception and lose the original failure. For resource failures, use the suppression behavior explained at 6.2. Inspect getSuppressed when the main exception does not explain cleanup failure. A matching exception type alone does not establish whether retry is safe; side effects and idempotency still matter.

Suppression example

Nested helper and method fragment inside a class; import java.io.`IOException`. The method returns the observed failure so a caller can check the primary cause and close order.

```
static final class CloseFailure implements AutoCloseable {
    private final String name;
    CloseFailure(String name) { this.name = name; }
    @Override public void close() throws IOException {
        throw new IOException(name);
    }
```

```
}
static IOException failureWithSuppression() {
    try (CloseFailure first = new CloseFailure("first");
         CloseFailure second = new CloseFailure("second")) {
        throw new IOException("body");
    } catch (IOException error) {
        return error;
    }
}
```

Expected result: the primary message is body; suppressed messages are second, then first. The body failure survives both close failures. Test the case where the body succeeds too, because then a close failure can become the primary exception.

JLS checked exception and catch rules

Follow up question Why should a specific catch precede a general catch?

Answer: A preceding general catch would handle the subtype and make the later subtype catch unreachable. In multi-catch, alternatives cannot be related by subclassing. Catch only failures you can handle or translate meaningfully, and preserve the cause.

Follow up question Should InterruptedException be translated like an ordinary failure?

Answer: Preserve the cancellation signal: propagate it or restore the interrupt flag before a deliberate translation. Do not silently retry indefinitely. Define whether the operation releases resources, returns a partial result or stops.

6.4 Numeric precision overflow and time

Core contract and interview points

For money, define precision and rounding explicitly. Use integer minor units when the scale is fixed and range is adequate, or `BigDecimal` when decimal arithmetic and variable scale are required. `BigDecimal` constructed from a double can capture its binary approximation; construct from a decimal string or use valueOf when appropriate.

`BigDecimal.equals` includes scale while compareTo compares numeric value. This affects sets, map keys, and deduplication. `Integer` arithmetic can overflow silently; use range checks or `Math.addExact` and `Math.multiplyExact` when overflow is an error. `Comparator` subtraction can overflow too.

Use `Instant` for a point on the timeline, an explicit `ZoneId` for local interpretation, and a monotonic clock such as `System.nanoTime` for elapsed duration. Do not mix wall-clock adjustments with timeout measurement.

Interview challenge: design a price comparator, a monetary total, and a timeout budget. State the unit, scale, overflow policy, rounding policy, and clock for each.

Official reference Java 25 BigDecimal API

Wrapper equality and nulls

For wrapper values, == tests reference identity; caching does not make it a general value-equality contract. Use equals or `Objects.equals` as the `null` policy requires. Unboxing a `null Integer` or `Long` throws `NullPointerException`.

Follow up question Why can BigDecimal equality behave differently in a HashSet and TreeSet?

Answer: equals includes scale, while the natural compareTo order compares numeric value. For 1.0 and 1.00 this can produce different deduplication results. Choose a representation and equality policy before using decimal values as keys.

Follow up question How do you measure a timeout budget correctly?

Answer: Use a monotonic elapsed-time source such as System.nanoTime, maintain a remaining budget and propagate it to downstream waits. Use Instant and a ZoneId for wall-clock interpretation. State the time unit and avoid overflow in conversion or deadline arithmetic.

7 Core Spring transactions and shared authority

This bounded backend bridge applies Java guarantees to container-managed objects, proxy call paths, database transitions and shared authority. Prepare injection, scope and state ownership, transaction evidence, retries and idempotency. It is not a complete Spring, persistence or database curriculum; add role-specific depth through 9.25.

7.1 Container bean discovery injection lifecycle and scopes

How does spring create and inject a bean?

This is much deeper than: `@Autowired` injects it.

You write: `@Service`

```
public class RiskService {
}
```

and: `@Service`

```
public class OrderService {

    private final RiskService riskService;

    public OrderService(RiskService riskService) {
        this.riskService = riskService;
    }
}
```

You never write:

```
new RiskService();
```

inside OrderService.

Spring creates and wires these objects.

This is: IoC - Inversion of Control

IoC

Without Spring:

```
RiskService risk = new RiskService();
OrderService order = new OrderService(risk);
```

Your code controls object creation.

With Spring: Spring Container

```
      ↓
```

creates RiskService

```
      ↓
```

creates OrderService

```
      ↓
```

injects RiskService Control over object construction/wiring has moved to the container.

Dependency Injection

DI is one mechanism through which IoC is implemented.

Instead of:

```
class OrderService {

    private RiskService riskService =
            new RiskService();
}
```

we declare dependency:

```
class OrderService {

    private final RiskService riskService;

    OrderService(RiskService riskService) {
        this.riskService = riskService;
    }
}
```

Spring supplies it.

`Bean` Discovery

Spring obtains bean definitions from mechanisms including: `@Component`; `@Service`; `@Repository`; `@Controller`; `@Configuration`; `@Bean`; `Component` scanning discovers annotated classes.

`Configuration` methods can explicitly define beans.

BeanDefinition

Spring first deals with metadata describing how beans should be created.

Conceptually: RiskService.class; scope = singleton; dependencies = ...

qualifiers = ...

lazy = false ...

This metadata is represented through BeanDefinition infrastructure.

Think: BeanDefinition = recipe; `Bean` instance = cooked object

Container

Core abstraction: `BeanFactory`; More feature-rich application container: `ApplicationContext`; The context manages: creation; dependency resolution; lifecycle; post-processing; events; resources; etc.

Simplified `Bean` Lifecycle

High ROI mental model: `Configuration` / scanning

```
        ↓
```

BeanDefinitions registered

```
        ↓
```

`Bean` instantiated

```
        ↓
```

Dependencies populated/resolved

```
        ↓
```

Aware callbacks where applicable

```
        ↓
```

`BeanPostProcessor` before initialization

```
        ↓
```

`@PostConstruct` / initialization callbacks

```
        ↓
```

`BeanPostProcessor` after initialization

```
        ↓
```

possibly wrapped in proxy

```
        ↓
```

bean ready for use This is simplified; actual Spring internals have additional phases and special handling.

`Constructor` Injection

Preferred default: `@Service`

```
public class OrderService {

    private final RiskService riskService;

    public OrderService(RiskService riskService) {
        this.riskService = riskService;
    }
}
```

Benefits: dependency is explicit; supports `final` fields; easy unit testing; object can't exist without required dependency; less hidden wiring; With one constructor, explicit `@Autowired` is generally unnecessary in modern Spring.

Multiple Beans of Same Type

```
interface RiskEngine {
```

```
}
```

Implementations: `@Component`

```
class FastRiskEngine implements RiskEngine {
}
```

`@Component`

```
class SafeRiskEngine implements RiskEngine {
}
```

Now: `OrderService(RiskEngine riskEngine);` Which one?

Ambiguous.

Spring may throw: NoUniqueBeanDefinitionException

`@Primary`

`@Component` `@Primary`

```
class FastRiskEngine implements RiskEngine {
}
```

Means: Prefer this candidate when multiple beans match by type.

`@Qualifier`

`@Component("fastRiskEngine")`

```
class FastRiskEngine implements RiskEngine {
}
```

Injection: OrderService(

```
    @Qualifier("fastRiskEngine")
```

RiskEngine engine) {

```
    this.engine = engine;
}
```

`@Qualifier` narrows candidate selection.

`Bean` Scope

Common scopes include: singleton; prototype; request; session; Default Spring bean scope: singleton; But remember: Spring Singleton normally means one bean instance per container for that bean definition, not globally one JVM object enforced by a private constructor.

Spring first discovers or receives bean definitions through component scanning, configuration classes, `@Bean` methods and related mechanisms. The `ApplicationContext`/`BeanFactory` uses those definitions to instantiate beans, resolve and inject dependencies, invoke lifecycle callbacks and BeanPostProcessors, and potentially wrap beans in proxies for features such as transactions or AOP. I generally prefer constructor injection because dependencies are explicit and can be final. If multiple beans match a dependency I resolve it using `@Qualifier`, `@Primary`, or explicit configuration.

Spring rapid fire

Q: `@Component` vs `@Service` vs `@Repository`?

They are stereotype annotations.

`@Service` communicates service-layer intent.

`@Repository` communicates persistence-layer intent and participates in Spring's persistence exception translation infrastructure when used appropriately.

`@Component` is generic.

Q: `@Bean` vs `@Component`?

`@Component`:

```
class discovered by scanning
```

`@Bean`: method explicitly creates/configures bean; `@Bean` is especially useful for third-party classes or explicit configuration.

Q: Why constructor injection?

explicit dependencies

```
final fields
```

testability required dependencies guaranteed at construction

Q: What happens with two implementations?

Potential ambiguity: NoUniqueBeanDefinitionException; Resolve using: `@Qualifier`; `@Primary`; explicit configuration

Q: Why can `@Transactional` fail on self-invocation?

Because Spring's typical transaction mechanism is proxy-based.

```
proxy → transactional interception → target
```

A direct internal call:

```
this.method();
```

doesn't cross the proxy boundary.

Interview question "Tell me how a Java request travels through a backend application."

A great integrated answer: Network request

```
      ↓
```

server accepts connection

```
      ↓
```

request assigned/executed

```
      ↓
```

Spring/controller/filter chain

↓

application code

↓

objects allocated

↓

JVM executes bytecode/JIT native code

↓

locks/atomics coordinate shared state

↓

DB/HTTP connection pools constrain I/O

↓

blocking / async / virtual-thread behavior

↓

response

↓

temporary objects become unreachable

↓

GC eventually reclaims memory; Meanwhile: JIT optimizes hot paths; GC manages heap; JMM defines thread semantics; OS schedules platform/carrier threads; CPU executes native instructions; That's the whole system.

7.2 AOP proxies and self invocation

`@Transactional` - The Proxy Question

Suppose: `@Service`

```
class PaymentService {

    @Transactional
    public void pay() {
        ...
    }
}
```

Spring commonly exposes a proxy: Caller

↓

Spring Proxy

↓

transaction starts

```
   ↓
PaymentService.pay()
   ↓
```

commit / rollback The annotation itself isn't magical bytecode that automatically opens a transaction merely because the method contains it.

Spring's infrastructure intercepts the invocation.

Self-Invocation Trap

```
public void outer() {
    inner();
}
@Transactional
public void inner() {
}
```

If `outer()` calls `inner()` directly on this, the invocation may not pass through the transactional proxy.

Conceptually: External caller

```
     ↓
Proxy
     ↓
outer() |
     └── this.inner()
           ↑
```

proxy bypassed Therefore expected transactional behavior may not be applied to `inner()` as assumed.

This is an extremely common Spring interview follow-up.

JDK Proxy vs CGLIB

High-level:

JDK Dynamic Proxy

Interface-oriented proxying.

```
interface
   ↑
proxy
```

CGLIB-style subclass proxy

Proxy subclasses the target class.

`Target`

```
↑
```

Proxy subclass Don't over-memorize internals unless interviewer drills further.

Understand why proxy limitations matter.

Follow up question How does Spring `@Transactional` relate to JVM method calls?

Answer: The annotation is metadata. In default proxy mode, a call through the Spring proxy is intercepted to apply transaction behavior; an ordinary direct call or self-invocation does not become transactional merely because the method has an annotation. Check propagation, rollback policy, proxy type and the actual call path.

Follow up question How does Spring AOP work conceptually?

Answer: A proxy wraps the target and applies an interceptor chain around eligible calls. JDK proxies expose interfaces; class proxies rely on overridable methods. Advice can manage transactions or other cross-cutting behavior. In default proxy mode, a target's direct self-call bypasses the proxy chain.

Official reference Spring declarative transaction annotation behavior

JDK proxies intercept calls through implemented interfaces. `Class`-based proxies cannot advise methods they cannot override, including final or private methods. `Method` visibility, the Spring version and configuration affect transactional interception. Test the actual proxy call path; a direct self-call in default proxy mode still bypasses it.

Spring proxy limitations

7.3 Transaction propagation isolation and rollback

Core contract and interview points

A transaction boundary defines which operations commit or roll back together. REQUIRED joins an existing transaction or creates one. REQUIRES_NEW creates an independent transaction and commonly requires an additional connection while the outer transaction is suspended. NESTED depends on transaction-manager and resource support, often using savepoints.

Default declarative rollback commonly applies to unchecked exceptions and `Error`, while checked-exception policy must be configured deliberately. A caught exception does not automatically communicate the same rollback decision as one crossing the proxy; a joined transaction can also become rollback-only and cause UnexpectedRollbackException at the outer boundary.

Isolation and optimistic concurrency address different anomalies. State the database and isolation level, then reason about dirty reads, non-repeatable reads, phantoms, lost updates, and write skew. A version column detects conflicting updates only when every writer obeys the versioned transition.

A typical imperative Spring transaction is thread-bound and does not automatically propagate into a newly started async task. Keep transaction scope short and define how remote calls, messages, and database state become consistent. For a database update plus an event, discuss an outbox and idempotent consumers rather than assuming one annotation covers both resources.

Official reference Spring declarative transaction annotation behavior

Follow up question Why can REQUIRES_NEW exhaust a connection pool?

Answer: The outer transaction can retain its connection while a nested independent transaction needs another. Concurrent callers may consume all available connections and wait for more. Keep scopes short, measure the actual manager behavior and size or redesign the resource demand.

Follow up question Does catching an inner failure guarantee the outer transaction can commit?

Answer: No. A participating inner transaction may already have marked the shared transaction rollback-only. The outer boundary can then throw UnexpectedRollbackException. Verify propagation, rollback rules and the actual proxy call path against the configured manager.

7.4 Local coordination versus shared authority

Concurrency control in java - now across three servers

This question moves from Java process-local coordination to shared authoritative state.

Inside One JVM

Suppose:

```
synchronized void updateBalance() {
    ...
}
```

Threads inside the same JVM coordinate using the same monitor.

JVM

```
Thread A ──┐
Thread B ──┼──> SAME LOCK
Thread C ──┘
```

Now Deploy Three Instances

Server 1 Server 2 Server 3

JVM A JVM B JVM C; Lock A Lock B Lock C; Each JVM owns its own memory.

Therefore: `synchronized`; on server A does nothing to prevent server B from entering its own critical section.

Same applies to ordinary: `ReentrantLock`; `AtomicInteger`; `ConcurrentHashMap`; They coordinate state inside a process, not across independent machines.

Example - Inventory

Database: stock = 1; Server A: read stock = 1; Server B: read stock = 1; Both think they can sell.

Both decrement.

Potential overselling/lost-update problem.

A local JVM lock cannot solve it because: A's lock ≠ B's lock

Message `Queue` / Serialization

Sometimes avoid concurrent mutation entirely.

requests

```
    ↓
```

partition by accountId

```
    ↓
```

one ordered processing stream per key

```
    ↓
```

state update This can simplify concurrency dramatically.

But now you must reason about: retries; duplicates; ordering; partitioning; consumer failure; idempotency

The "three servers" pattern

This deserves memorization because it generalizes far beyond locks.

Whenever interviewer says: It works on one server. Now deploy three.

Immediately ask yourself: WHAT WAS LOCAL STATE?

Examples:

```
synchronized lock
```

local cache; session; rate limiter; counter; scheduler; in-memory queue; idempotency map; On one server: JVM; |; shared memory; On three: JVM A JVM B JVM C; | | |; memory A memory B memory C; Now ask: Where does coordination/state ownership live?

Possible answers: database; Redis/shared store; message broker; partition owner; coordination service; load-balancer affinity; external durable store; This mental transformation solves a large family of system-design questions.

Follow up question Why doesn’t synchronizing each of three servers protect one stock count?

Answer: Each process has a different monitor and memory. Put the invariant at one shared authority, such as a conditional database transition or a partition owner with defined failover; all writers must obey the same protocol.

Follow up question Does sticky routing supply durable distributed correctness?

Answer: No. Affinity can affect where requests arrive but does not by itself survive failover or prevent another writer from changing authoritative state. Explain the storage transition, ownership and retry behavior separately from routing.

7.5 Database atomicity optimistic and pessimistic locking

Solution 1 - Database Atomicity / Constraints

Often the database should enforce the invariant.

Example: UPDATE product; SET stock = stock - 1; WHERE id = ?

```
  AND stock > 0;
```

Then inspect affected-row count.

This can be much safer than: SELECT stock; application check; UPDATE stock; because the check and update aren't separated.

General principle: Put critical invariants as close as practical to the authoritative state.

Optimistic Locking

Record: id; balance; version; Read: balance = 100; version = 7; Update: UPDATE account; SET balance = ?,

```
    version = 8
```

WHERE id = ?

```
AND version = 7;
```

If affected rows:

```
1 → success
0 → somebody changed it
```

Then retry or reject according to business semantics.

Great when: conflicts are relatively uncommon

Pessimistic Locking

Conceptually: SELECT ...

```
FOR UPDATE;
```

`Lock` the relevant database row during the transaction.

Useful when conflicts are expected and serialization is necessary.

Costs: waiting; contention; deadlock risk; reduced concurrency

Database concurrency rapid fire

Q: Lost update?

Initial balance = 100

A reads 100 B reads 100

A writes 110 B writes 120

Expected = 130; Actual = 120; A's update was lost.

Q: How prevent it?

Depending on requirements: atomic SQL; optimistic locking; pessimistic locking; appropriate transaction isolation; serialization/message partitioning

Q: Optimistic locking good when?

Conflicts are relatively uncommon.

Advantages: less blocking; higher concurrency; Cost:

```
conflict → retry/failure handling
```

Q: Pessimistic locking good when?

When conflicts are sufficiently likely or the operation requires exclusive access during a transaction.

Cost: blocking; deadlocks; lower concurrency

7.6 Idempotency retries and effect guarantees

Idempotency

Suppose payment request: PAY ₹100; Client times out and retries.

Without protection: ₹100; ₹100; could be processed twice.

Use an idempotency key: requestId = abc123; Server remembers/atomically records:

```
abc123 → already processed
```

Repeated request produces the same business effect rather than another charge.

Exactly Once?

Be cautious saying: Kafka gives exactly once, therefore my entire distributed business operation happens exactly once.

Broker guarantees do not automatically extend across arbitrary external systems.

For practical distributed workflows, think: at-least-once delivery + idempotent processing + deduplication + transactional boundaries; depending on architecture.

Answer to rehearse

Inside one JVM I can coordinate threads using `synchronized` blocks, locks, atomics, concurrent collections, or message-passing patterns. Once the application runs on three servers, those locks are process-local and don't coordinate the other JVMs. I therefore move the concurrency guarantee to shared authoritative infrastructure - for example atomic database updates, unique constraints, optimistic versioning, pessimistic row locking, serialized message processing, or when justified a distributed lock. The exact choice depends on contention, consistency requirements, failure semantics and throughput. I also design operations to be idempotent because retries are unavoidable in distributed systems.

Choose the authority and transition that enforce the business invariant, then explain what happens under retry, process failure and stale ownership.

Interview question Why are retries dangerous?

Suppose downstream is overloaded.

Clients retry.

original traffic = 10k/sec; failures = 50%; retries = 5k/sec; Now downstream receives: 15k/sec; which creates more failures.

This is retry amplification.

Mitigate with:

- exponential backoff
- jitter
- retry limits
- idempotency
- circuit breaking
- deadlines
- load shedding

8 Core Timed reconstruction and defense

Finish the Core pass with cold coding, debugging and interviewer challenges. Record the exact gap before proceeding to additional depth.

8.1 Timed coding debugging and defense practice

Core contract and interview points

Attempt before reading. For each topic, give a direct answer, explain the mechanism, write a small example, identify a failure trigger, and defend a tradeoff. Use the numbered topic as the one place to repair a gap.

Coding round 1 10 minutes: write an immutable class containing a list, then defend mutable elements and construction leakage. Coding round 2 10 minutes: build a frequency map and highest-paid-per-department result with an explicit tie policy. Coding round 3 15 minutes: implement a bounded concurrent reservation and test rejection without state mutation.

Debugging round 15 minutes: diagnose a frozen batch from thread states and lock ownership, then distinguish CPU saturation, queue delay, connection starvation, and GC pressure. Explain which evidence would invalidate the first hypothesis.

Defense round 10 minutes: explain why one `AtomicLong` does not protect a multi-field invariant, what changes across three JVMs, when virtual threads help, and why a timeout does not guarantee cancellation.

Assess the results against the opening readiness check and record the next repair at its numbered topic.

Hands-on completion checks

Use these as reconstruction tasks after studying their linked mechanisms. Attempt from a blank file, compile on the stated JDK, and check the failure trigger. Reading a solution does not complete a task. Eight selected examples were checked on Java 21: immutable profile, department winner, bounded reservation, exception suppression, `UTF-8` lines, Externalizable data validation, Future lifecycle and primitive/boxed array conversion. The other tasks below are practice acceptance criteria, not a claim that every fragment in the book was executed.

Topic	Reconstruct	Acceptance check
2.4	Immutable profile	Change the caller list; reject accessor mutation and `null` elements; explain shallow versus deep copying. Defend why an unmodifiable list still permits mutation of mutable elements.
2.8	Mutable map key	Demonstrate a failed lookup after mutating a key, then use a stable value key; prove equality/hash consistency. Explain why resizing or synchronizing the map cannot repair a broken key contract.

Topic	Reconstruct	Acceptance check
2.9	Priority queue	Use an overflow-safe comparator, poll in order, and explain why iterator order is different. Defend poll complexity and a deterministic tie policy.
5.9	Department winner	Test empty input, salary ties and reordered tied rows; state the deterministic tie and output-order policies. Defend the comparator and merge function when input order changes.
5.10	Grouping collector	Test missing categories, repeated keys and `null` policy; explain result mutability and downstream accumulation. Explain which accumulator state is shared in sequential and parallel collection.
6.3	Resource suppression	Throw from the body and both closes; assert primary failure and reverse closure/suppression order. Explain why a close failure does not replace a body failure.
10.4	UTF-8 and buffer	Close `Files.lines`; test blank/non-ASCII lines; demonstrate flip/clear and partial-write handling. Explain resource ownership and why a read/write may transfer fewer bytes than requested.
3.3	Lost update	Explain the permitted read/read/write/write interleaving; fix it and check the invariant after all futures complete. Name the linearization point and show why `volatile` alone is insufficient.
9.14	Timed lock	Handle failed acquisition without unlock; release acquired locks in finally; preserve interruption policy. Defend one lock versus CAS for a multi-field invariant.
3.8	Bounded executor	Force saturation with latches; observe the chosen rejection policy, record failures and terminate workers. Explain how worker, queue and downstream limits affect rejection and latency.
3.10	`Future` composition	Test success, upstream failure and timeout; distinguish completion from stopping the underlying operation. Defend where cancellation, deadline propagation and resource cleanup belong.
9.17	Admission semaphore	Release only after successful acquisition; verify peak admitted work stays within the limit. Explain why virtual threads do not increase downstream capacity.
9.18	Frozen latch	Force one task to fail; arrange finally/countDown and observed failures; distinguish timed wait from completion. Explain why a zero latch count does not mean every task succeeded.
3.12	Concurrent reservation	Test Long.MAX_VALUE boundaries, negative input and competing callers; assert total and rejection without mutation. Identify the successful CAS as the linearization point; defend overflow checks and release ownership.
7.3	Transaction call path	Compare an external proxy call with self-invocation; verify actual commit/rollback and propagation behavior in an integration test. Explain the proxy boundary and evidence for the actual transaction.

Topic	Reconstruct	Acceptance check
7.6	Idempotency boundary	Send duplicate and concurrent requests; atomically claim the key and business effect; test failure and retry recovery. Defend crash recovery when the database commit and external effect cannot be atomic.
4.8	Diagnostic defense	Use repeated thread evidence and resource metrics to separate deadlock, queueing and connection starvation. State a falsifiable hypothesis and evidence that would change the diagnosis.

Interviewer challenges and defensible answers

2.4 Immutability

Interviewer challenge: The interviewer says `List.copyOf` makes the entire object graph immutable. Do you agree?

Defensible answer: It protects the list structure and rejects `null` elements, but does not deep-copy mutable elements. With immutable `String` elements that is sufficient for this profile; with mutable positions, copy each position or expose immutable values. `final` protects the reference assignment, not the referenced object. Demonstrate both caller-list mutation and element mutation before choosing the copy strategy.

5.9 Deterministic aggregation

Interviewer challenge: The interviewer asks whether keeping the first salary tie is deterministic.

Defensible answer: It is tied to encounter order, so reordered input can choose a different person. Define a business tie rule, such as salary descending then a stable unique employee ID, and use the same total comparator in the merge function. Separate winner selection from department output order. Shuffle input and assert the same winners and prescribed output order; document the policy for duplicate IDs.

3.3 and 3.12 Shared invariant

Interviewer challenge: The interviewer proposes `volatile` consumed and says visibility makes check-then-add safe.

Defensible answer: Two callers can both observe room and both commit, so visibility does not make the transition atomic. For one bounded counter, validate inputs and overflow-safe remaining capacity, then CAS from the observed value to the proposed value and retry on failure. The successful CAS is the linearization point. If reservation changes several fields, protect the whole invariant with one lock or one immutable state in an `AtomicReference`. Across JVMs, the authority must move to shared storage or an owner with a defined failure protocol.

3.8 and 3.10 Async capacity

Interviewer challenge: The interviewer says a timed-out `CompletableFuture` has freed the worker and downstream connection.

Defensible answer: Timeout or exceptional completion reports a result boundary; it does not prove the underlying operation stopped. `CompletableFuture` cancellation does not generally interrupt its computation. Bound admission, propagate a deadline to the operation, arrange cooperative cancellation where supported, and close resources at the actual operation lifecycle. Prove cleanup with a controlled blocked operation and observed termination. Virtual threads reduce the cost of waiting, but still need downstream capacity limits.

7.3 Transactions

Interviewer challenge: The interviewer shows a self-call to a transactional method and expects a new transaction.

Defensible answer: In Spring default proxy mode, self-invocation bypasses the proxy advice. An existing outer transaction may still be active, but the inner annotation does not independently apply its requested propagation. Use a call through a separate injected bean or a deliberately chosen transaction mechanism. Verify commit, rollback and propagation using an integration test against the actual transaction manager; the presence of an annotation is insufficient evidence.

7.6 Idempotency

Interviewer challenge: The interviewer says inserting a deduplication key before an external charge guarantees one charge.

Defensible answer: A crash between recording the key and charging, or between charging and recording its outcome, leaves ambiguity. Define request identity, payload conflicts and a durable state machine. Atomically record database-owned effects with their claim; for an external effect use a provider idempotency key or a recoverable outbox and reconciliation protocol. A local unique constraint prevents duplicate claims, but cannot atomically commit an unrelated external system. Test crashes at each boundary and explain how retry discovers the prior outcome.

4.8 Diagnosis

Interviewer challenge: The interviewer says a large number of WAITING threads proves deadlock.

Defensible answer: Waiting is a state, not a deadlock diagnosis. Take repeated thread evidence, identify owned and requested locks and look for a cycle; also inspect executor queues, connection-pool usage, CPU and allocation/GC evidence. A batch waiting on a latch whose failed worker never counted down is different from a monitor cycle. State the hypothesis, choose evidence that could disprove it, then validate the smallest fix under the same failure trigger and representative load.

After each answer, change one assumption: mutable elements, unordered input, several fields, multiple JVMs, a slow dependency or a crash after an external effect. Explain which guarantee breaks, revise the design and state the test that would distinguish the alternatives. Defend the decision from evidence and constraints; revise it when the interviewer supplies a new fact.

9 Applied Java depth and production scenarios

After the Core pass, extend the mechanisms through selected implementations, runtime versions and production scenarios.

9.1 Modern Java features and version boundaries

Core contract and interview points

Use Java 17 and Java 21 as explicit interview baselines, and name the version when discussing newer features. Explain a feature through the problem it solves and its constraints rather than listing release trivia.

Records express data carriers with generated accessors, equality, hashing, and a canonical constructor. They are shallowly immutable: a record component referring to a mutable list still requires a defensive copy. Sealed hierarchies restrict permitted subtypes and help model a closed set of outcomes. Pattern matching can make type-based branches clearer.

Java 21 finalizes virtual threads and pattern matching for switch. Java 25 supports compact source files and eligible instance or no-argument main methods. Traditional public static void `main(String[] args)` remains a useful portable entry point.

Do not claim that preview features are stable APIs. State the target JDK, required flags, and whether the feature is finalized before suggesting it for a production codebase.

Official reference Java 25 compact source files and instance main methods | Java Language Specification execution and memory model | JDK 24 changes including virtual thread synchronization and ZGC

Records and closed hierarchies

Records are final data carriers and may implement interfaces. Generated equality delegates reference components to their equals contract: two int[] components with equal contents can still compare unequal because arrays use identity equality. Validate invariants in the canonical or compact constructor. Keep deep-copy policy with topic 2.4; a generated accessor does not copy a mutable component.

A direct subtype of a sealed class must be final, sealed or non-sealed, subject to the language rules. Pattern switches require valid coverage and branch ordering; decide `null` handling explicitly rather than assuming default catches `null`. State the target JDK before using record patterns or pattern switch.

Version addresses

Java 21 adds sequenced collection interfaces for ordered first, last and reversed access. A reversed view is backed by the original collection; supported mutations are visible through both views. An unmodifiable collection remains unmodifiable, and these APIs are unavailable on a Java 17 baseline.

Java 25 finalizes `ScopedValue` for lexically scoped bindings. The binding is fixed within a scope, but a bound mutable object is not thereby made immutable; this is not automatic context propagation into every arbitrary executor task. `StructuredTaskScope` remains a preview API in Java 25, with version-specific signatures and preview flags. Recheck its status when updating the target JDK.

Java 25 Record contract

SequencedCollection contract

JDK 25 finalized and preview feature status

9.2 Bounded caches FIFO LRU and deduplication

Core contract and interview points

Insertion-order `LinkedHashMap` evicts by arrival order when a bounded removeEldestEntry policy is used: that is FIFO. Access-order `LinkedHashMap` uses new `LinkedHashMap`<>(capacity, loadFactor, true) for LRU ordering. In access-order mode, get may modify order, which matters for concurrency.

A bounded in-memory deduplication cache gives a limited recent-history window. An old duplicate may be processed after eviction or restart. Durable idempotency requires a shared stable key and an atomic record of the effect; cache size alone does not establish that guarantee.

Bounded FIFO deduplication cache

The example maintains a small cache of recently seen messages to prevent duplicate processing.

It uses: `LinkedHashMap`; with a maximum size of 100 entries.

When entry 101 arrives:

- The earliest inserted entry is evicted when the bounded insertion-order map exceeds its limit. This is FIFO, not access-order LRU.
- The entry count is bounded. Actual memory use also depends on the retained objects, backing storage, and object sizes.
- the cache never grows indefinitely.

Why insertion order?

PTR evicts the oldest message by arrival time, not by how recently it was accessed.

That's appropriate for deduplication, where "oldest seen" is the right eviction policy.

A security guard keeps the last 100 visitor IDs.

When the 101st visitor arrives, the very first ID is discarded to make room for the new one.

Bounded `LinkedHashMap` cache implementation

The notes describe this implementation:

```
private final Set<RxRejectAll> cache =
    Collections.newSetFromMap(
        new LinkedHashMap<>() {
            @Override
            protected boolean removeEldestEntry(...) {
                return size() > 100;
            }
        });
```

This cache is used to deduplicate recently seen RxRejectAll messages and keeps only the latest 100 entries.

Why do we need a cache?

Imagine the system receives: RxRejectAll #101

RxRejectAll #101

RxRejectAll #101

RxRejectAll #101; Without a cache: Reject; Reject; Reject; Reject; The same work is repeated.

Instead: Receive

```
↓
```

Already seen?

```
↓
```

YES

```
↓
```

Ignore

Official reference Java 25 LinkedHashMap API

9.3 Sorted concurrent maps and skip lists

`ConcurrentSkipListMap` vs `ConcurrentHashMap`

`ConcurrentSkipListMap` has sorted keys while `ConcurrentHashMap` does not sort.

`ConcurrentSkipListMap` is not fast as compared to `ConcurrentHashMap`.

`ConcurrentSkipListMap` keeps keys in their natural or comparator order and supports concurrent sorted navigation with expected logarithmic operations. `HashMap` normally has expected constant-time key operations with suitable hashes; measured throughput depends on workload and synchronization requirements.

`ConcurrentHashMap`

- ❌ Sorted

- ❌ Navigable
- ✅ Concurrent

`ConcurrentSkipListMap`

- ✅ Sorted
- ✅ Navigable
- ✅ Concurrent

Further reading

`LinkedHashMap` - original insertion order `TreeMap` - sorted using comparator or natural order

`ConcurrentSkipListMap` (`Thread`-safe Sorted `Map`)

In many applications, you need a map that multiple threads can safely read and update at the same time.

A regular `TreeMap` is sorted, but it is not thread-safe.

A `ConcurrentHashMap` is thread-safe, but it does not keep keys sorted.

The example needs both:

- multiple threads can access the map simultaneously
- PTLGs should appear in alphabetical order in the UI

So it uses: `ConcurrentSkipListMap`<`String`, ParticipantRow>; This provides:

- `Thread`-safe access
- Keys remain sorted automatically
- `ConcurrentSkipListMap` provides expected average logarithmic lookup and update costs, with sorted navigation.
- It can be appropriate for concurrent sorted access; throughput relative to a `synchronized TreeMap` depends on the workload.

Why not `ConcurrentHashMap`?

Suppose the UI displays: BANK_C; BANK_A; BANK_D; BANK_B; Users would find this confusing.

A `ConcurrentSkipListMap` automatically displays: BANK_A; BANK_B; BANK_C; BANK_D; without additional sorting.

Why Rotations are Bad for Concurrency?

Imagine; `Thread` A; Rotate Tree; Meanwhile; `Thread` B; Searching; During rotation; Pointers temporarily become; A

```
↓
```

?

```
↓
```

B Tree is temporarily inconsistent.

Mutating an ordinary `TreeMap` concurrently requires synchronization. Concurrent tree algorithms exist; rotations do not prove that every tree implementation needs one global lock.

Often One global lock is a simple synchronization strategy, not an unavoidable property of all tree designs.

Thousands of threads

↓

Waiting

↓

Bad scalability.

Skip `List` Idea

Instead of rotating trees; Use; Multiple linked lists.

Imagine city roads.

Local roads; A-B-C-D-E-F-G-H-I-J-K-L; Slow.

Express highway; A------F-------L; Very fast.

Super highway; A----------L; Even faster.

Searching Top Level

↓

Too far?

↓

Drop one level

↓

Continue; Eventually; Reach target.

Structure

Level 3; A----------Z; Level 2; A------G-------Z; Level 1; A---C---F---J---Z; Level 0; A-B-C-D-E-F-G-H-I-J-K...

Every node exists at Level 0.

Higher levels are shortcuts.

9.4 Primitive streams boxing and array results

Core contract and interview points

`String.chars()` emits UTF-16 code units as ints. Casting each value to char does not support every Unicode character as one element. For code-point semantics, use

`codePoints()` and keep the result as int code points; grapheme clusters require a further text boundary policy.

The first non-repeating `Character` example therefore uses UTF-16 char semantics. State that constraint or change the return type and frequency keys when the input contract requires supplementary Unicode code points.

Convert 1D `List` to 1D Array

Further reading

```
List<Integer> list = List.of(1, 2, 3);
```

For a List<Integer>, use these independent primitive and boxed results. Mapping to int unboxes each element and throws NullPointerException for a `null` element; the boxed array conversion preserves `null` elements.

```
int[] primitive = list.stream().mapToInt(Integer::intValue).toArray();
Integer[] boxed = list.toArray(Integer[]::new);
```

Everything becomes interesting when it is a 2D array. In some problems, it is expected to return a int[][] but things will be easy if we use `List<List>` , In this case below will be helpful.

- Create `List` of int[] rather `List<List>` eg: `List`<int []> myList= new `ArrayList`<>();
- add or remove items using add method eg: `myList.add(new int[] {1,2});`
- finally while returning, rather looping over list and dumping it into new array of int[][], return myList.toArray(new int[myList.size()][]);

Intentional compile-failure comparison. The stream2 assignment is deliberately invalid: `IntStream` is a primitive stream, not a `Stream<Object>`. Compile the valid alternatives separately to demonstrate boxing and unboxing bridges.

```
Stream<Object> stream1 = Stream.of(1, 2, 3);
IntStream intStream1 = IntStream.of(4, 5, 6);

Stream<Object> stream2 = IntStream.of(4, 5, 6);
Stream<Object> stream3 = IntStream.of(4, 5, 6).mapToObj(e -> e);
IntStream intStream2 = Stream.of(4, 5, 6).mapToInt(e -> e);
```

```
public static Optional<Character> firstNonRepeatingCharacter(String input)
{

   if (input == null || input.isEmpty()) {
       return Optional.empty();
   }

   Map<Character, Long> frequency =
           input.chars()
                   .mapToObj(c -> (char) c)
                   .collect(Collectors.groupingBy(
```

```
                            Function.identity(),
                            LinkedHashMap::new,
                            Collectors.counting()
                    ));

    return frequency.entrySet()
            .stream()
            .filter(entry -> entry.getValue() == 1)
            .map(Map.Entry::getKey)
            .findFirst();
}
```

Primitive streams

Java provides: `IntStream`; `LongStream`; `DoubleStream`

Example:

```
int total =
    employees.stream()
        .mapToInt(Employee::salary)
        .sum();
```

Average:

```
double avg =
    employees.stream()
        .mapToInt(Employee::salary)
        .average()
        .orElse(0);
```

Convenient numeric operations and reduced boxing/unboxing compared with:

```
Stream<Integer>
```

9.5 Streams loops and algorithmic complexity

Algorithmic complexity

```
Streams do not fix bad algorithms.
```

Bad:

```
list.stream()
    .filter(otherList::contains)
```

If otherList is an `ArrayList`: contains = `O(m)`

overall ≈ `O(n × m)`

Better:

```
Set<T> lookup = new HashSet<>(otherList);

list.stream()
    .filter(lookup::contains)
```

Approximately: `O(n + m)`

Senior principle: Data structure and algorithm choice matter more than `Stream` vs loop.

Streams vs loops

```
Streams are excellent for:
```

- filtering
- transformation
- grouping
- aggregation
- readable data pipelines

Loops can be clearer for:

- complex mutation
- state machines
- intricate branching
- multiple forms of early exit
- certain hot performance paths

Strong answer

I prefer Streams when they make a data transformation clearer. I do not force Streams into algorithms where explicit state or complex control flow makes an ordinary loop easier to understand.

Red-flag statements

Do not say:

```
Streams are always faster.
```

Do not say: `parallelStream()` makes code faster.

Do not say:

```
Streams can be reused.
```

Do not say: `toMap()` automatically handles duplicate keys.

Do not hide important mutations inside: peek()

Do not forget:

```
max/min/find → may produce Optional
distinct → equality matters
parallelism + mutable shared state → dangerous
```

Interview question "Streams are slower than loops." True?

Too simplistic.

Performance depends on:

- workload

- allocations
- pipeline structure
- JIT optimization
- data size
- boxing
- terminal operation
- parallelism

For example: `IntStream`; may avoid boxing compared with:

```
Stream<Integer>
```

But don't choose syntax based on folklore.

For ordinary business code: Prefer clarity, then benchmark genuinely hot paths.

9.6 Stream coding practice and reconstruction

```
// 1. FILTER + MAP
list.stream()
    .filter(x -> condition)
    .map(x -> transformation)
    .toList();
// 2. SORT
list.stream()
    .sorted(Comparator.comparing(X::field))
    .toList();
// 3. MAX / MIN
list.stream()
    .max(Comparator.comparing(X::field));
// 4. TO MAP
list.stream()
    .collect(Collectors.toMap(
```

X::key,

```
        Function.identity()
    ));
// 5. GROUP
list.stream()
    .collect(Collectors.groupingBy(
```

X::field

```
    ));
// 6. GROUP + AGGREGATE
list.stream()
    .collect(Collectors.groupingBy(
```

X::field,

```
        Collectors.counting()
```

```
    ));
```

If these six become reflexes, most other questions are merely substitutions.

The reconstruction algorithm

Whenever you receive a `Stream` coding problem, ask: 1. INPUT; What am I streaming?

2. KEEP Do I need `filter()`?

3. CHANGE Do I need `map()` or `flatMap()`?

4. ORDER / SELECT `sorted()`? `distinct()`? `limit()`? `skip()`?

5. OUTPUT One value?

`Boolean`?

`Number`?

`List`?

`Map`?

Groups?

Memory formula

```
INPUT → KEEP → CHANGE → ORDER → OUTPUT
```

Example question: Find names of the top 3 highest-paid active IT employees.

Decompose:

```
INPUT   → Employee
KEEP    → active
          department = IT
ORDER   → salary DESC
SELECT  → first 3
CHANGE  → Employee → name
OUTPUT  → List<String>
```

Then reconstruct:

```
employees.stream()
    .filter(Employee::active)
    .filter(e -> e.department().equals("IT"))
    .sorted(
        Comparator.comparing(Employee::salary)
                  .reversed()
)
    .limit(3)
    .map(Employee::name)
    .toList();
```

Don't-confuse table

A	B	Difference
filter	map	keep vs transform
map	flatMap	transform vs transform + flatten
toMap	groupingBy	one/key vs many/key
groupingBy	partitioningBy	arbitrary keys vs `Boolean`
reduce	collect	value vs container
map	mapToInt	object `Stream` vs primitive numeric `Stream`
findFirst	findAny	encounter-order preference vs greater freedom

9.7 Allocation TLABs escape analysis and boxing

Primitive `Collections` (Avoid Boxing)

A normal Java map like: `HashMap<Long`, RiskCheck>; stores `Long` objects, not primitive long values.

Whenever you do:

```
map.get(tradeableId);
```

Java automatically converts long

```
into
```

`Long` This process is called boxing.

Boxing may allocate a wrapper, but cached values and JIT optimizations mean it is not correct to say every boxing operation creates a new object.

On a busy hot path, boxing and wrapper storage can increase allocation and footprint. Measure the actual allocation rate before claiming a throughput improvement.

A primitive-key collection such as Eclipse `Collections` LongObjectHashMap can avoid wrapper keys. Treat the PTR naming in this example as teaching context, not a verified claim about a production deployment.

Instead of; `HashMap<Long`, RiskCheck>; it uses; LongObjectHashMap<RiskCheck>; Now the key stays a primitive long.

Primitive keys remove key boxing, but the map still allocates storage and may resize. This is not a promise of zero allocation for the entire operation.

No boxing.

Less GC.

Potential throughput improvement must be demonstrated by measurements on the actual workload.

Interview question Are objects always allocated on the heap?

The safe conceptual answer is: Java objects are conceptually heap objects, but the JIT may optimize allocation away when observable semantics allow it.

For example:

```
Point p = new Point(1, 2);
return p.x() + p.y();
```

If p never escapes, JIT optimization may make the physical allocation unnecessary.

Possible optimizations include:

- scalar replacement
- allocation elimination

So avoid the simplistic claim: "new always means an actual heap allocation."; At the language level you should reason as though an object exists. Physical runtime implementation may differ.

Interview question Isn't allocating objects expensive?

Not necessarily.

Modern JVM allocation is often very fast.

One optimization is TLAB - `Thread` Local Allocation `Buffer`.

Conceptually: Heap

```
  ├── Thread A TLAB
  ├── Thread B TLAB
  └── Thread C TLAB
```

`Thread` A can often allocate by advancing a pointer: current

```
    ↓
```

[used][used][free][free]

```
                ↑
```

allocate here This avoids global synchronization on every small allocation.

The larger performance question is often: What is the application's allocation rate and resulting GC/memory pressure?

Interview question What is escape analysis?

Escape analysis determines whether an object's reference escapes a certain scope.

Example:

```
int calculate() {
    Point p = new Point(1, 2);
```

```
    return p.x + p.y;
}
```

p doesn't escape the method.

This information may enable optimizations such as:

- scalar replacement
- allocation elimination
- lock elimination in suitable cases

Important correction: Don't simply say: "Escape analysis puts objects on the stack."; That's an oversimplification.

A better statement is: Escape analysis provides information enabling optimizations that can eliminate allocations or replace object fields with scalar values.

Interview question "`Object` creation is expensive, so reuse everything." True?

Usually bad advice.

Modern allocation is cheap.

`Object` pooling can introduce:

- synchronization
- stale-state bugs
- lifecycle complexity
- cache effects
- memory retention

Pooling is justified for specific expensive/scarce resources or measured extreme performance requirements.

Examples:

```
DB connections → pool
threads → historically pool platform threads
ordinary DTOs → usually don't pool
```

Low-latency systems can have specialized reasons for object reuse, but that's a measured engineering decision.

Interview question Why might lowering allocation improve latency even if GC pauses are already small?

Because allocation affects more than pauses:

- memory bandwidth
- cache pressure
- GC CPU
- young collection frequency
- copying work

But allocation elimination has diminishing returns.

Don't turn clean domain code into object-pooling machinery unless profiling justifies it.

9.8 Class loading initialization and identity

Interview question Explain class loading.

Major lifecycle: Loading

```
  ↓
```

Linking

```
├── Verification
├── Preparation
└── Resolution
  ↓
```

Initialization

Loading

`Class` bytes are located and a runtime class representation is created.

Verification

Checks bytecode validity/safety constraints.

Preparation

Creates/initializes class-level runtime structures and static field storage to default values as specified.

Resolution

Symbolic references may be converted to direct runtime references.

Initialization

Static initialization logic runs.

Interview question What is the ClassLoader delegation model?

Simplified: Application ClassLoader

```
      ↓ asks parent
```

Platform ClassLoader

```
      ↓ asks parent
```

Bootstrap ClassLoader The typical idea is parent-first delegation.

Why?

It helps avoid applications accidentally replacing foundational platform classes and promotes consistent class identity.

Interview question What are the main class loaders?

Conceptually in modern Java: Bootstrap

```
    ↓
```

Platform

```
    ↓
```

Application/`System`

```
    ↓
```

custom loaders if needed The old "extension class loader" terminology belongs to older Java designs; modern Java uses the Platform ClassLoader.

Interview question When are two classes considered the same class?

A very important advanced question.

`Class` identity depends on both: binary class name + defining ClassLoader; Therefore: loader A loads com.foo.User; loader B loads com.foo.User; These can be distinct runtime types.

This explains surprising ClassCastExceptions in:

- plugin systems
- application servers
- hot reload
- custom classloader architectures

Interview question How can ClassLoaders cause memory leaks?

Suppose: Application ClassLoader

```
      ↓
```

classes

```
      ↓
static fields
      ↓
```

large object graph If something long-lived retains the ClassLoader, everything reachable through that loader may remain alive.

Potential sources include:

- threads
- ThreadLocals
- static registries
- callbacks/listeners
- native references

This is particularly relevant to redeployment systems.

Interview question Can two classes with the same name be different types?

Yes, if defined by different ClassLoaders.

9.9 JIT inlining deoptimization and code cache

Interview question What is JIT compilation?

The JVM observes execution and compiles frequently executed bytecode into optimized native machine code.

bytecode

```
   ↓
interpreted execution
   ↓
```

profiling

```
   ↓
```

hot method detected

```
   ↓
```

JIT compilation

```
   ↓
```

optimized native code This allows optimization based on actual runtime behavior.

Interview question Why not JIT compile everything immediately?

Compilation itself costs:

- CPU
- memory
- startup time

And much code executes rarely.

Suppose: 10,000 methods loaded; 100 methods consume 90% CPU; Optimizing all 10,000 aggressively would waste resources.

So the JVM focuses expensive optimization on hot paths.

Interview question What is tiered compilation?

Conceptually, HotSpot can use multiple compilation levels.

```
interpret / profile
       ↓
```

quickly compiled code

```
       ↓
```

more profiling

```
        ↓
```

highly optimized code; The goal is to balance: startup speed + profiling quality + peak performance; You generally don't need compiler-tier numbers unless interviewing for JVM internals.

Interview question What is method inlining?

Instead of:

```
int result = add(a, b);
```

requiring an actual call boundary, the JIT may effectively incorporate add's operations into the caller.

Why important?

Inlining eliminates call overhead, but even more importantly: It exposes more code to subsequent optimizations.

For example: Inlining

```
  ↓
```

constant propagation

```
  ↓
```

dead-code elimination

```
  ↓
```

escape analysis

```
  ↓
```

further optimization Inlining is therefore one of the JIT's most important enabling optimizations.

Interview question What is deoptimization?

JIT optimizations may depend on assumptions inferred from runtime profiling.

Example:

```
interface PaymentProcessor
        ↑
```

only CardProcessor observed The JIT may optimize aggressively around the observed type.

Later another implementation appears.

The JVM must preserve correctness.

It can invalidate assumptions and fall back to less optimized execution.

That process is broadly called deoptimization.

This demonstrates a crucial property: JIT optimization can be speculative while remaining semantically correct.

Interview question What is the Code Cache?

JIT-generated native machine code must be stored somewhere.

HotSpot maintains memory areas for compiled code, commonly discussed as the code cache.

Therefore JVM memory isn't simply: heap + stack; A real JVM process consumes: heap; metaspace; thread stacks; code cache; direct buffers; GC structures; native libraries; JVM internal memory; ...

9.10 Generational collection and pause semantics

Interview question Why does generational GC exist?

Because many applications exhibit the weak generational hypothesis: Most objects die young.

Example: HTTP request

```
↓
```

DTO

```
↓
```

temporary validation objects

```
↓
```

response

```
↓
```

most become garbage quickly Therefore collectors can optimize around object age.

Conceptually: Young Generation

```
├── Eden
└── Survivor regions/spaces
```

Old Generation

```
└── longer-lived objects
```

Exact organization depends on collector implementation.

Interview question What happens when an object is allocated?

Conceptually: new `Object()`

```
    ↓
```

allocation

```
    ↓
```

usually young-generation region

```
    ↓
```

survives collections?

```
    ↓ yes repeatedly
```

eventually treated as older/long-lived This is simplified because modern collectors such as G1 operate using regions.

Interview question What is a minor GC?

Historically, a minor GC refers to collection focused on the young generation.

Because many young objects are dead, these collections can reclaim large amounts of memory efficiently.

Terminology varies somewhat between collectors, so in modern interviews it is better to explain the actual collector behavior rather than obsess over labels.

Interview question What is a Full GC?

Broadly, a Full GC is a heavyweight collection involving substantially more of the heap/runtime state than an ordinary young collection.

It is often undesirable because it can produce significant pauses.

But: Full GC does not automatically mean "GC bug."; It can result from:

- heap sizing
- allocation pressure
- retained live set
- promotion pressure
- explicit `System.gc()`
- metaspace pressure
- collector-specific fallback situations

Production investigation requires GC logs and runtime telemetry.

Interview question What does Stop-The-World mean?

A STW pause is a period during which application Java threads are paused so the JVM can safely perform certain operations.

Not every GC phase must necessarily stop all application work.

Modern collectors attempt to perform significant work concurrently.

Important distinction: Concurrent ≠ zero pause; Even concurrent collectors require some synchronization pauses.

9.11 G1 regions phases and humongous allocations

Interview question What is G1 GC?

G1 is a region-based, generational garbage collector designed to balance throughput with predictable pause-time goals.

Instead of thinking of the heap as simply contiguous: Young | Old; think: Heap

[R][R][R][R][R][R][R][R] Y O Y F O Y O ...

where regions can take different roles.

G1 can prioritize regions expected to yield significant reclaimable memory.

Hence the name: Garbage First.

Interview question Why is G1 region based?

Regions provide flexibility.

The collector can choose subsets of heap regions for collection rather than requiring a simplistic fixed contiguous generational layout.

Benefits include:

- incremental evacuation
- adaptive generation sizing
- reclaiming garbage-rich regions
- easier management of large heaps

Interview question What are G1's major conceptual phases?

You don't need every internal phase for ordinary backend interviews.

Know: Young collections + Concurrent marking of old-generation liveness

```
        ↓
```

identify reclaim opportunities

```
        ↓
```

mixed collections

```
        ↓
```

young + selected old regions; Key idea: G1 incrementally collects selected regions rather than routinely compacting the entire heap.

Interview question What is a mixed collection in G1?

A mixed collection collects:

- young regions
- plus selected old-generation regions

after G1 has gathered information about old-generation liveness.

The goal is to gradually reclaim old-generation garbage without constantly resorting to full-heap collection.

Interview question What are humongous objects in G1?

Large objects exceeding a collector-specific threshold relative to region size receive special treatment.

They can occupy one or more contiguous regions and may create memory-management challenges.

Senior practical point: If GC behavior is strange and the application allocates very large: byte[]; char[]; large arrays; investigate humongous allocation patterns.

9.12 Collector choice G1 Parallel and ZGC

Core contract and interview points

Choose a collector from service goals and measurements. Parallel GC emphasizes throughput; G1 balances regional evacuation and pause goals; ZGC emphasizes low pauses using substantial concurrent work. Concurrent collectors still have pauses and consume CPU and memory headroom.

State the JDK, distribution, supported collector, heap, allocation rate, live set, CPU limits, and latency objective. A pause target is a goal rather than a guarantee. JDK 24 removes the non-generational ZGC mode; avoid applying older ZGC descriptions to every version.

Validate a collector change under comparable request load and container limits. Compare p95 and p99 latency, throughput, GC pauses, concurrent GC CPU, allocation, and process footprint rather than comparing only one pause statistic.

Official reference Java 25 garbage collection tuning guide | JDK 24 changes including virtual thread synchronization and ZGC

Follow up question Is the collector with the smallest GC pauses always best for p99?

Answer: No. Concurrent GC consumes CPU and memory headroom, while application queueing, locks and I/O may dominate latency. Compare total request latency and throughput under equivalent load, allocation, live set and container limits.

Follow up question What would make you reject a collector change?

Answer: Reject a change that misses the service objective or worsens sustainable throughput, footprint or stability under representative load, even if one pause metric improves. Record the JDK, settings and rollback condition.

9.13 Explicit locks read write locks and StampedLock

Locks and Atomic Variables

Instead of using an intrinsic lock via the `synchronized` keyword, you can also use various Locking classes provided by Java's Concurrency API to have more fine-grained control over the locking mechanism.

Locks

`ReentrantLock`

`ReentrantLock` is a mutually exclusive lock with the same behavior as the intrinsic/implicit lock accessed via the `synchronized` keyword.

`ReentrantLock`, as the name suggests, possesses reentrant characteristics. That means a thread that currently owns the lock can acquire it more than once without any problem.

```
import java.util.concurrent.locks.ReentrantLock;

class ReentrantLockCounter {
    private final ReentrantLock lock = new ReentrantLock();

    private int count = 0;

    public void increment() {
        lock.lock();
        try {
            count = count + 1;
        } finally {
            lock.unlock();
        }
    }
}
```

ReadWriteLock

`ReadWriteLock` consists of a pair of locks - one for read access and one for write access. The read lock may be held by multiple threads simultaneously as long as the write lock is not held by any thread.

A read-write lock can allow concurrent readers, but read-heavy traffic alone does not guarantee better performance. Measure critical-section cost, contention, writer progress and lock overhead against simpler alternatives.

```
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;

class ReadWriteCounter {
    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    private int count = 0;

    public int incrementAndGetCount() {
        lock.writeLock().lock();
        try {
            count = count + 1;
            return count;
        } finally {
```

```
            lock.writeLock().unlock();
        }
    }

    public int getCount() {
        lock.readLock().lock();
        try {
            return count;
        } finally {
            lock.readLock().unlock();
        }
    }
}
```

Q: `synchronized` vs `ReentrantLock`?

Both provide mutual exclusion and memory-visibility guarantees when used correctly.

`ReentrantLock` additionally offers capabilities such as: tryLock

```
interruptible lock acquisition
```

multiple Conditions; timed acquisition; optional fairness; Use the simplest mechanism satisfying requirements.

Interview question `synchronized` vs `ReentrantLock`?

Both provide mutual exclusion and memory-synchronization semantics when used correctly.

`ReentrantLock` provides additional capabilities such as: `tryLock()`; `lockInterruptibly()`; fairness option; multiple `Condition` objects; Typical pattern:

```
lock.lock();
try {
    update();
} finally {
    lock.unlock();
}
```

Always emphasize the finally.

Decision rule

Use `synchronized` when basic mutual exclusion is enough.

Use explicit locks when you specifically need their capabilities.

Don't automatically replace `synchronized` because "locks are faster." Modern JVM synchronization is highly optimized and workload-dependent.

Interview question What is `tryLock()` useful for?

It allows an attempt without waiting indefinitely.

```
if (lock.tryLock()) {
    try {
```

```
        update();
    } finally {
        lock.unlock();
    }
}
```

Useful when you need:

- timeout behavior
- fallback paths
- deadlock-avoidance strategies
- latency bounds

Interview question What is `ReadWriteLock`?

It distinguishes: read lock; write lock; Multiple readers may coexist when no writer holds the lock.

A writer requires exclusive access.

Potentially useful for: many reads; few writes; sufficiently expensive protected operations; But synchronization overhead and contention patterns matter.

Don't assume read/write locks always outperform a simple mutex.

Interview question What is `StampedLock`?

`StampedLock` supports modes including:

- write locking
- read locking
- optimistic reads

Optimistic pattern conceptually:

```
long stamp = lock.tryOptimisticRead();

int x = this.x;
int y = this.y;

if (!lock.validate(stamp)) {
    // retry under read lock
}
```

Potentially useful for read-heavy structures.

But it is more complex and not reentrant.

High ROI answer: Know why optimistic reads exist; don't reach for `StampedLock` by default.

9.14 CAS ABA and compound state transitions

Java.util.concurrent.atomic

- AtomicBoolean

- `AtomicInteger`
- `AtomicIntegerArray`
- AtomicIntegerFieldUpdater
- `AtomicLong`
- AtomicLongArray
- AtomicLongFieldUpdater
- AtomicMarkableReference
- `AtomicReference`
- AtomicReferenceArray
- AtomicReferenceFieldUpdater
- `AtomicStampedReference`
- DoubleAccumulator
- `DoubleAdder`
- `LongAccumulator`
- `LongAdder`

What is CAS?

Answer:

CAS means Compare-And-Swap.

Conceptually:

```
if (currentValue == expectedValue)
    currentValue = newValue
```

The comparison and update happen atomically.

Example:

```
AtomicInteger counter = new AtomicInteger();

counter.incrementAndGet();
```

Atomics commonly use CAS internally.

Benefit: avoids blocking locks for many simple concurrent operations.

Trade-off: under heavy contention, repeated CAS failures cause retries and CPU consumption.

What is the ABA problem?

Answer:

Suppose CAS expects value A.

Another thread changes:

```
A → B → A
```

Our thread sees A again and CAS succeeds, even though the state changed in between.

That's the ABA problem.

A common solution is attaching a version: `A(version 1)`

```
→ B(version 2)
→ A(version 3)
```

Java provides: `AtomicStampedReference`

What would atomic mean?

Atomic means the entire:

```
READ → MODIFY → WRITE
```

behaves as one indivisible operation.

No other thread can interfere in the middle.

That's why this works:

```
AtomicInteger count = new AtomicInteger(10);

count.incrementAndGet();
```

`volatile` makes individual reads/writes visible, but it doesn't make a sequence of operations atomic. `count++` is read-modify-write, so multiple threads can cause lost updates.

And this gives you the crucial distinction:

```
volatile       → "Can other threads SEE my update?"
atomic/lock    → "Can other threads INTERFERE with my update?"
```

Q: `AtomicInteger` vs `synchronized`?

Atomic operations often use CAS for operations such as:

```
incrementAndGet();
```

Good for relatively simple atomic state transitions.

For compound invariants involving multiple pieces of state, a lock or different design may be clearer.

Q: What is CAS?

Compare current value with expected value

IF equal: replace with new value

ELSE: fail/retry; Conceptually: `CAS(memory, expected, newValue);`
Hardware-supported atomic primitive underlying many concurrent algorithms.

Interview question What is CAS?

CAS = Compare-And-`Set`/Swap.

Conceptually:

```
if currentValue == expectedValue:
    currentValue = newValue
```

success; else: failure; Atomics use hardware/JVM primitives to implement operations without traditional mutual-exclusion locking for many use cases.

Example:

```
AtomicInteger count = new AtomicInteger();

count.incrementAndGet();
```

Interview question How can CAS implement an increment?

Conceptually: loop:

```
    old = value
    new = old + 1

    if CAS(value, old, new):
        return new
```

retry Suppose another thread modifies the value between read and CAS.

The CAS fails.

We retry with the new value.

Interview question `Lock` vs CAS?

`Lock`

Acquire

```
 ↓
```

critical section

```
 ↓
```

release Under contention threads may block/park.

Cas

read

```
 ↓
```

attempt atomic update

```
 ↓
```

failure?

```
 └── retry
```

CAS can perform very well for short operations and moderate contention.

But under extreme contention: many failed CAS attempts

```
↓
```

CPU waste; There is no universal: "`Lock`-free is always faster."

Interview question What is the ABA problem?

Suppose CAS expects: A; Another thread changes:

```
A → B → A
```

The first thread observes A again and CAS may succeed, even though state changed in between.

That's ABA.

Solutions in algorithms can include versioning/stamping.

Java provides tools such as: `AtomicStampedReference`; when that semantic problem matters.

Interview question "`Lock`-free = faster." True?

No.

CAS under contention can cause: failed CAS

```
↓
```

retry

```
↓
```

CPU burn Locks can sometimes behave better under contention by parking waiters.

Choose based on semantics and measured workload.

Interview question Can `AtomicInteger` prevent all races?

No.

Only operations/state correctly expressed through it.

9.15 ForkJoinPool and work stealing

What is `ForkJoinPool`?

Answer:
It's a thread pool designed around work stealing.

Each worker maintains work, and idle workers can steal tasks from busy workers.

Worker 1: [A B C D] Worker 2: []

Worker 2 steals

```
          ↓
```

Worker 1: [A B]; Worker 2: [C D]; Useful for recursively decomposable parallel computation.

Parallel streams commonly use the common `ForkJoinPool`.

Interview question What is `ForkJoinPool`?

Designed primarily around divide-and-conquer parallelism and work stealing.

big task

```
   ↓ fork
```

task A task B

```
 ↓              ↓
```

smaller smaller; \ /; join; Workers have task queues and idle workers can steal work from others.

Interview question What is work stealing?

Instead of one centralized task queue:

```
Worker A → deque A
Worker B → deque B
Worker C → deque C
```

If C becomes idle, it can steal work from another worker.

This helps balance irregular recursive workloads.

Interview question Why can blocking inside `ForkJoinPool` be problematic?

Its parallelism assumptions are oriented around active computation and cooperative task execution.

`Long` uncontrolled blocking can occupy workers and reduce useful parallelism.

This is one reason blindly putting blocking I/O into: `parallelStream()`; can be problematic.

9.16 Virtual threads carrier threads and version specific pinning

Core contract and interview points

Virtual threads are finalized in Java 21. They improve scalability for supported blocking workloads by allowing many tasks to share carrier threads, but do not increase CPU cores, database connections, or the useful concurrency of a downstream service.

Java 21 can pin a carrier while blocking in a `synchronized` region. Java 24 changes monitor behavior so `synchronized` blocking no longer causes that monitor-related pinning. Native interactions and other version-specific behavior still require investigation. Assess the deployed JDK rather than replacing every `synchronized` block automatically.

Keep admission limits at the resource boundary, for example a `Semaphore` around expensive downstream operations. Avoid pooling virtual threads as scarce reusable workers; manage task lifetime and dependency limits instead.

Interview question What are virtual threads?

Virtual threads are lightweight Java threads managed by the JVM rather than mapping one-to-one to expensive OS threads for their entire lifetime.

Conceptually: many virtual threads

```
        ↓
```

JVM scheduler

```
        ↓
```

smaller set of carrier/platform threads

```
        ↓
```

OS They make thread-per-task/request programming practical for many blocking I/O workloads.

Interview question Do virtual threads make code execute faster?

Not automatically.

They primarily improve scalability of concurrency, especially when tasks spend significant time blocking.

For CPU-bound work: 100,000 virtual threads; doesn't create: 100,000 CPUs; CPU throughput remains bounded by available compute.

Interview question Do virtual threads remove the need for connection pools?

No.

This is an important interview question.

Virtual threads make blocked threads cheaper.

They do not make downstream resources infinite.

Example: 10,000 virtual threads

```
        ↓
```

database

```
        ↓
```

50 useful concurrent queries You still need:

- DB connection limits
- rate limits
- semaphores
- bounded downstream concurrency
- timeouts

Virtual threads remove one scarcity: expensive platform-thread occupancy.

They don't remove every scarcity.

Interview question What is virtual-thread pinning?

Normally a virtual thread can unmount from its carrier when blocking in supported ways.

Some operations can prevent that temporarily, leaving the virtual thread attached to the carrier.

This is called pinning.

Modern Java releases have improved pinning behavior, so don't repeat outdated blanket claims such as "`synchronized` always pins virtual threads." Interview-safe principle: Understand that virtual-thread scalability depends on whether blocking can release carrier resources; evaluate pinning with the behavior of the Java version you're actually running.

Follow up question Are virtual threads OS threads?

Answer: No. A virtual thread is scheduled by the Java runtime onto carrier platform threads. It preserves the Java `Thread` programming model without dedicating one OS thread to every virtual thread for its lifetime.

Follow up question Do virtual threads eliminate race conditions?

Answer: No. Shared mutable state still obeys the same Java Memory Model and needs correct ownership, atomic operations or synchronization. More concurrent tasks can expose races and downstream capacity limits more readily.

Follow up question Do virtual threads make CPU work parallel beyond core count?

Answer: No. They help scale tasks that spend substantial time waiting; runnable CPU work still competes for available processors. Bound expensive CPU work and downstream admission separately rather than equating cheap threads with unlimited capacity.

Official reference JDK 24 changes including virtual thread synchronization and ZGC

9.17 Semaphores and downstream admission limits

What is a `Semaphore`?

Answer:
A semaphore controls how many threads may access something concurrently.

```
Semaphore semaphore = new Semaphore(10);
```

Only 10 permits can be held concurrently.

Useful for protecting a limited downstream resource: 1000 requests

```
    ↓
Semaphore(20)
    ↓
```

maximum 20 concurrent downstream calls Unlike a mutex, it isn't fundamentally about exactly one owner.

Interview question What is a `Semaphore`?

A semaphore controls access through a fixed number of permits.

Example:

```
Semaphore s = new Semaphore(10);
```

Conceptually: 10 permits

```
   ↓
```

maximum 10 concurrent users Typical use:

```
s.acquire();
try {
    callExpensiveService();
} finally {
    s.release();
}
```

Interview question Why use a `Semaphore` with virtual threads?

Excellent modern question.

Virtual threads may allow: 100,000 concurrent tasks; But downstream may tolerate: 100 concurrent calls; Use: virtual threads

```
      ↓
```

`Semaphore(100)`

```
      ↓
```

downstream; This separates: cheap application concurrency; from: limited downstream capacity

Release a permit only after acquisition succeeded, and release it in finally when the protected work finishes. An interrupted or timed-out acquire must not trigger an unmatched release; extra releases can increase the available permits and break the intended admission bound. Choose a timeout and overload policy, and do not confuse admission control with protection of a multi-field state transition.

9.18 CountDownLatch CyclicBarrier and coordination

Interview question What is `CountDownLatch`?

One or more threads wait until a count reaches zero.

```
CountDownLatch latch = new CountDownLatch(3);
```

Workers:

```
latch.countDown();
```

Coordinator:

```
latch.await();
```

Conceptually:

```
Worker A ─┐
```

```
Worker B ─┬─→ countdown → 0 → coordinator proceeds
Worker C ─┘
```

A latch is one-shot.

Interview question `CountDownLatch` vs `CyclicBarrier`?

`CountDownLatch`

N events happen

```
↓
```

waiter proceeds One-shot.

`CyclicBarrier`

N participating threads

```
↓
```

all reach barrier

```
↓
```

all continue Can be reused across cycles.

Completion and failure contracts

`CountDownLatch` is one-shot; it cannot be reset. Actions before countDown are visible after a successful await return through its memory-consistency contract. A timed await returning false does not establish that all work completed. Use finally for completion signaling where appropriate, and separately observe task failure rather than treating a zero count as success.

`CyclicBarrier` coordinates a fixed group at repeated rendezvous points. Interruption or timeout can break a barrier; peers may receive BrokenBarrierException and need a recovery policy. A barrier action and its ordering are distinct from merely decrementing a latch. Do not reset a barrier while assuming it cancels all participating tasks. Use `Phaser` when dynamic party registration is a real requirement.

9.19 Singleton initialization and ways to break assumptions

Write a thread-safe singleton. now break it with reflection.

What is a Singleton?

Normally:

```
new Service();
new Service();
new Service();
```

creates different objects.

A Singleton tries to enforce: Application; |; +------> ONE Singleton instance; |; +------> same instance; |; +------> same instance; No matter how many callers request it, they receive the same instance.

Basic Singleton

```
public final class Singleton {

    private static final Singleton INSTANCE = new Singleton();

    private Singleton() {
    }

    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

Why is the constructor private?

Otherwise:

```
Singleton a = Singleton.getInstance();
Singleton b = new Singleton();
```

would create another instance.

Is this thread-safe?

Yes.

```
private static final Singleton INSTANCE = new Singleton();
```

Static initialization is performed safely by the JVM's class-initialization mechanism.

The class is initialized before INSTANCE is used.

This is one of the simplest thread-safe Singleton implementations.

Lazy Initialization - Naive Version

Suppose creation is expensive and we want to create it only when needed.

```
public final class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {

        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```

Problem

Two threads: `Thread` A `Thread` B

instance == null instance == null

new `Singleton()` new `Singleton()`

instance = A instance = B Two objects may be created.

Therefore: This lazy Singleton is not thread-safe.

Synchronized Singleton

```
public final class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static synchronized Singleton getInstance() {

        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }
}
```

Correct.

But synchronization occurs on every invocation.

Usually unnecessary after initialization.

`Double`-Checked Locking

```
public final class Singleton {

    private static volatile Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {

        if (instance == null) {

            synchronized (Singleton.class) {

                if (instance == null) {
                    instance = new Singleton();
```

```
                }
            }
        }

        return instance;
    }
}
```

Why TWO `null` checks?

First:

```
if (instance == null)
```

avoids synchronization after initialization.

Second:

```
if (instance == null)
```

prevents two threads that raced past the first check from both creating the object.

Example: A sees `null`; B sees `null`

A gets lock B waits

A creates Singleton A releases lock

B gets lock

Without second check: B creates another Singleton

Why MUST instance be `volatile`?

This is the important interview question.

`Object` construction can conceptually be viewed as: 1. allocate memory; 2. initialize object; 3. assign reference to instance; Without the required memory-ordering guarantees, another thread must not be allowed to observe publication as though it happened before initialization was safely visible.

The dangerous conceptual outcome is: 1. allocate memory; 2. publish reference; 3. initialization not yet safely visible to another thread; `Thread` B could observe: instance `!= null`

```
while seeing incompletely initialized state.
```

`volatile` provides the required visibility and ordering guarantees.

Answer to rehearse

`Double`-checked locking requires `volatile`. A `volatile` write safely publishes the initialized object, and a subsequent `volatile` read establishes the necessary happens-before relationship, preventing another thread from observing improperly published state.

Better Solution - Initialization-on-Demand Holder

```
public final class Singleton {

    private Singleton() {
    }

    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

Advantages: lazy; thread-safe; no explicit synchronization; simple; Why?

Holder isn't initialized until it is actually referenced.

JVM class initialization provides synchronization guarantees.

Strong Simple Solution - Enum Singleton

```
public enum Singleton {
    INSTANCE;

    public void execute() {
        System.out.println("running");
    }
}
```

Usage:

```
Singleton.INSTANCE.execute();
```

Advantages: thread-safe initialization; serialization-safe by design; strong protection against ordinary reflective construction; very little code; Joshua Bloch famously popularized this approach.

Break the Ordinary Singleton Using Reflection

Consider:

```
public final class Singleton {

    private static final Singleton INSTANCE = new Singleton();

    private Singleton() {
    }

    public static Singleton getInstance() {
        return INSTANCE;
    }
```

```
}
```

Reflection can bypass normal Java access checks: `Constructor`<Singleton> constructor =

```
        Singleton.class.getDeclaredConstructor();

constructor.setAccessible(true);

Singleton s1 = Singleton.getInstance();
Singleton s2 = constructor.newInstance();

System.out.println(s1 == s2);
```

Potential result: false; The private constructor alone does not establish an absolute security boundary against reflective access in environments where such access is permitted.

Other Ways Singleton Assumptions Can Break

Serialization

An ordinary serializable Singleton may deserialize into another instance.

Traditional mitigation:

```
private Object readResolve() {
    return INSTANCE;
}
```

Cloning

If cloning is exposed, another object could potentially be produced.

Multiple ClassLoaders

Singleton means roughly: one instance per defining ClassLoader.

Two independent class loaders can load separate copies of the class.

So "one object in the entire universe" is too strong.

Answer to rehearse

For a simple Singleton I prefer an enum Singleton or the initialization-on-demand holder idiom. If I use double-checked locking, the instance must be `volatile` because safe publication and visibility matter. A normal private-constructor Singleton can potentially be broken using reflection, serialization, cloning, or separate class loaders. Enum Singleton handles several of these problems cleanly.

Follow-up questions

Why not synchronize the entire getter?

Correct but unnecessary synchronization after initialization.

Why `volatile`?

Visibility + ordering + safe publication.

Does `volatile` make an object thread-safe?

No.

It safely publishes the reference here. Mutable fields inside the object still require proper synchronization if concurrently modified.

Is Spring Singleton the same thing?

No.

Spring Singleton normally means: one bean instance per Spring `ApplicationContext`/bean factory scope.

It isn't the Gang-of-Four Singleton pattern enforcing construction globally.

Follow up question Explain double-checked locking.

Answer: Read the `volatile` instance into a local, return it if present, otherwise acquire the class monitor and check again before constructing and publishing exactly once. The second check handles a competing initializer; `volatile` supplies the publication relationship. Avoid leaking this from the constructor.

Follow up question Why must the instance be `volatile`?

Answer: In this double-checked-locking protocol, readers can return without taking the initializer's monitor. Volatile publication prevents them from observing an improperly published constructed instance. A protocol that always uses the same lock is different and does not need this unlocked-read pattern.

Follow up question Better singleton alternatives?

Answer: Use a static `final` field for eager initialization or an initialization-on-demand holder for lazy class-initialization-based publication. An enum can be appropriate when its construction and serialization semantics fit. A container-managed single bean is per container, not automatically one instance across JVMs.

9.20 Object pools ownership and reference counting

Core contract and interview points

The reference-counting example here concerns pool ownership, not the JVM garbage collector. The object may return to the pool only after every legitimate consumer releases its claim and no consumer can retain an untracked reference.

A counter alone does not establish a safe pooling protocol. Define who may acquire a claim, prevent resurrection after the final release, make claim and release thread-safe when shared, handle exceptional paths, and ensure a reused object is not observed by an old consumer. Generation tokens or strict ownership can be necessary.

Prefer ordinary allocation until measurements justify pooling. Pooling can retain a larger live set, increase complexity, and introduce use-after-return style bugs even though Java references remain memory-safe.

Solution: Reference Counting

Instead of owned

or

not owned; store; owners = 3; Initially; `from()`

```
↓
```

refCount = 1 Risk Engine owns it.

Audit needs it.

`claim()`

```
↓
```

`refCount++`

```
1 → 2
```

Persistence needs it.

`claim()`

```
↓
2 → 3
```

Current state Exposure

refCount = 3

```
↙      ↓       ↘
```

Risk Audit Persist

Risk Engine finishes `release()`

```
↓
3 → 2
```

`Object` NOT returned.

Audit finishes `release()`

```
↓
2 → 1
```

Still not returned.

Persistence finishes `release()`

```
↓
1 → 0
```

NOW safe.

```
clear()
↓
return to pool
```

9.21 Price time ordering and partial execution

Core contract and interview points

The priority-queue sketch illustrates price priority and deterministic time or sequence tie breakers. It is a partial execution sketch, so a complete matching transition must also decrement both quantities, retain or reinsert a partially filled order, publish the trade once, and handle cancellation.

Use an exact price representation such as integer ticks when the product requires decimal precision. Define ownership of the book and mutable order quantities. A thread-safe queue alone does not make the multi-step matching transaction atomic.

```
private final PriorityQueue<Order> buyOrders =
       new PriorityQueue<>(
Comparator
                       .comparingDouble(Order::getPrice)
                       .reversed()
                       .thenComparingLong(Order::getArrivalTime)
                       .thenComparingLong(Order::getOrderId)
       );

private final PriorityQueue<Order> sellOrders =
       new PriorityQueue<>(
Comparator
                       .comparingDouble(Order::getPrice)
                       .thenComparingLong(Order::getArrivalTime)
                       .thenComparingLong(Order::getOrderId)
       );

public boolean canExecute() {

   if (buyOrders.isEmpty() || sellOrders.isEmpty()) {
       return false;
   }

   return buyOrders.peek().getPrice()
           >= sellOrders.peek().getPrice();
}

public void executeNext() {
```

```
    if (!canExecute()) {
        System.out.println("No executable pair.");
        return;
    }

    Order buy = buyOrders.poll();
    Order sell = sellOrders.poll();

    int executedQuantity =
            Math.min(buy.getQuantity(), sell.getQuantity());
```

Follow up question What must happen after matching only part of an order?

Answer: Decrement both remaining quantities, preserve the residual order's declared priority, and record the trade once in the book owner's transition. Define cancellation and replay behavior. The stored sketch is intentionally incomplete and is not a runnable matching engine.

Follow up question Why is double a weak default price representation?

Answer: Binary floating point may not represent decimal ticks exactly. Use validated integer ticks or a declared decimal policy, with overflow checks and product scale. The legacy comparator sketch demonstrates ordering only; do not use it as the monetary-value contract.

9.22 Distributed locks leases and fencing

Distributed Lock

External coordination service:

```
Server A ─┐
Server B ─┼──> shared lock service
Server C ─┘
```

Examples can be built using systems such as Redis or coordination services.

But this introduces distributed-systems problems: server crashes; network partition; lock holder pauses; lease expires; stale owner continues; Hence: TTL / leases; unique ownership tokens; fencing tokens; failure semantics; may matter.

Why Fencing Tokens?

Imagine: A acquires lock; token = 41

A pauses

lease expires

B acquires lock token = 42

A wakes up A may incorrectly believe it can still write.

If downstream storage accepts only monotonically newer tokens: 42 accepted

later 41 rejected the stale owner can be fenced out.

This is a deeper distributed-lock insight.

Follow up question What makes a fencing token effective?

Answer: The resource performing the write must validate the ownership epoch or token under its atomic write protocol and reject stale owners. Merely generating a higher token at a lock service does not protect a storage system that ignores it.

Follow up question Is a unique ownership token the same as a fencing token?

Answer: No. A unique owner token helps ensure only the matching owner releases a lease. An ordered fencing token lets an authoritative resource reject stale-owner writes. Define expiry, renewal, failover and resource-side enforcement.

9.23 Lock contention and reducing shared mutation

Interview question How does lock contention affect performance?

Suppose:

```
synchronized (globalLock) {
    process();
}
```

100 threads arrive.

Actual execution: 1 running; 99 waiting; More threads don't improve throughput.

They can worsen:

- latency
- scheduling overhead
- memory usage

This is why reducing shared mutable state can be more powerful than tuning locks.

Interview question How would you reduce contention?

Potential approaches: 1. Reduce critical-section duration; 2. Partition state; 3. Use per-key locking; 4. Use concurrent structures; 5. Use immutable snapshots; 6. Use atomics for suitable state; 7. Avoid unnecessary shared state; 8. Redesign ownership; Example: Instead of: one global RiskState lock; use:

```
Client A → state A
Client B → state B
Client C → state C

if business invariants permit independent partitioning.
```

Interview question Why can a `synchronized` single-thread design beat lock-free concurrent code?

Single ownership can reduce CAS retries, lock contention and cache-coherence costs. It can improve throughput and predictability, but does not guarantee deterministic end-to-end latency; scheduling, GC, I/O and handoff queues still matter.

9.24 JVM metrics and service observability

Interview question What JVM dashboard would you want in production?

At minimum: APPLICATION

```
├── request rate
├── error rate
├── p50/p95/p99 latency
└── saturation
```

JVM

```
├── CPU
├── heap
├── post-GC heap
├── allocation rate
├── GC pauses
├── GC CPU
├── thread count
├── thread states
└── metaspace
```

RESOURCES

```
├── executor queue depth
├── active workers
├── DB pool
├── HTTP pool
└── downstream latency/errors
```

The power comes from correlation.

Example:

```
p99 ↑
executor queue ↑
DB pool wait ↑
```

GC unchanged Don't blame GC.

Interview question Why is allocation rate useful?

Two applications both use: 4 GB heap; Application A: allocates 100 MB/sec; Application B: allocates 5 GB/sec; Their GC behavior can be radically different.

Heap occupancy tells you: how much exists; Allocation rate tells you: how quickly garbage/liveness is being produced; Both matter.

9.25 Maintaining and extending these notes

Stable topic locations

Maintain one canonical home for each topic and preserve its internal bookmark when its title or position changes. Correct the explanation, source and example together. Place a new topic after its prerequisites in the appropriate preparation depth. Renumber visible sections and topics sequentially to match the physical hierarchy, then update cross-references, category links, reading ranks and the Word contents. Give a new topic a unique internal bookmark; do not reuse a retired bookmark.

Before adding an explanation, locate the mechanism that already owns it. Extend that topic if the new material adds a distinct contract, example, failure case or version difference. Create a separate topic when it introduces a different mechanism with its own useful question. A scenario can link to several mechanisms while keeping its business invariant and failure analysis in one place.

Reusable topic shape

- Question and purpose followed by the exact guarantees and important non-guarantees.
- One reconstructible example with imports or surrounding types stated, plus a boundary or failure case.
- At the assigned depth, include a useful tradeoff and distinct follow-ups. Core coding topics need reconstruction and a boundary check; diagnostic topics need an evidence-driven investigation. Awareness and Lookup do not need artificial code examples or repeated answers.
- Applicable Java or framework versions, primary references, and the validation actually performed.

Version and evidence updates

Keep Java 17 and 21 as the current baseline until the target role or deployed runtime changes. For a newer JDK, check official release notes, specifications and API documentation. Label finalized, preview and incubator features correctly. Keep a historical claim only when it teaches a relevant version difference; do not leave an obsolete claim beside its correction.

For a code change, compile the complete example on its stated JDK and test a relevant boundary, failure or concurrent invariant. A successful example is evidence for that example, not every fragment or deployment. Update internal links and priorities, refresh the Word contents, and render changed pages and their following page boundaries. A broad reorganization needs a broad layout review.

Scope and admission for future material

The syllabus is fixed at 96 canonical topics for this edition. Accept corrections, relevant JDK or framework contract changes, demonstrated cold-practice gaps and explicit role requirements. Extend the existing owner first. Spring and SQL are bounded backend bridges; deeper JPA/Hibernate, reactive programming, security, networking/FIX, cloud or trading-domain coverage belongs in linked role-specific material when needed. An Awareness or Lookup update need not enlarge Core.

Change record

Date	Topics	Reason	Evidence
13 Sep 2026	2.6 and 3.1 to 3.2	Correct runtime interning and happens-before claims	Oracle `String` API and JLS 17
13 Sep 2026	3.12	Prevent reservation arithmetic overflow	Java 21 boundary and concurrent checks
13 Sep 2026	10.1 and 6.3 to 10.4	Add bounded coverage and examples	Official annotation, reflection, exception and NIO references; selected Java 21 checks
13 Sep 2026	Follow-ups, ranks and 8.1	Answer prompts; order all topics; add reconstruction checks	Direct answer audit; 96 unique ranks; seven selected Java 21 examples
13 Sep 2026	10.3 and 3.9	Fix checked exceptions, data validation and executor cleanup	Java 21 serialization and interruption/lifecycle checks
13 Sep 2026	Inline code and 2.7	Add native inline code style; qualify iterator claim	Word render; unchanged code-text checks; Oracle `ConcurrentHashMap` API
13 Sep 2026	Reading sequence code and defense	Match physical order and ranks; Java colours; remove redundant comments; repair array conversion; add defenses	Source checks; eight selected Java 21 examples; labelled inheritance failure rejected; primary APIs; Word render
14 Sep 2026	Default interview flow	Order Core before Applied, Awareness and Lookup; align nested section numbers, references, routes and Word contents with physical reading order	All topic blocks, source questions, code and media retained; rank/body/contents checks; Word render
14 Sep 2026	Final editorial pass	Restore depth symbols; merge repetition; remove misleading visuals; bound	Java 21 composition checks; exact source and unchanged-owner audit; refreshed Word contents and render

Date	Topics	Reason	Evidence
		backend scope; add distinct follow-ups	
Future entry	Topic ID	Gap or version change	Source plus validation performed

Code formatting and copyability

Use native Word Inline Code for API identifiers and expressions in prose, and Code paragraphs for separate examples. Java keywords are dark blue, type names teal, strings green, numbers brown and comments gray; ordinary identifiers and punctuation stay black. Colour is an aid to scanning, not a correctness signal. Paste plain source with straight quotes and preserve indentation, brackets and operators. Keep comments that explain a contract, non-obvious step, failure or required placeholder; remove jokes, commented-out alternatives and narration that merely repeats the code. Label complete examples, context-dependent fragments and intentional failures. Compare stored source with compiled source and check rendered wrapping and page boundaries after changes.

Ranking and answer maintenance

Keep nested section numbers, reading ranks and contents in exactly the physical sequence. All Core topics precede Applied, Awareness and Lookup. Preserve topic bookmarks and move a complete owner block once when necessary. Retain repetition only when it adds a distinct example, diagnostic consequence, business invariant or deliberate recall task; merge equivalent local questions and explanations. Use the same 🔥 / ★ / ○ / △ symbols throughout navigation.

10 Awareness Java internals and version details

Use this section for deeper implementation and reference detail. Earlier Core topics supply the main interview contracts.

10.1 Annotations reflection and module boundaries

Core contract and interview points

Annotations supply metadata; they do not execute behavior by themselves. `Retention` determines whether metadata remains in source, the class file or at runtime; the default retention is CLASS. `Runtime` inspection requires RUNTIME retention. `Target` constrains legal annotation locations. An annotation processor, compiler or framework must interpret the metadata to produce behavior.

Reflection inspects or invokes runtime types and members and therefore still needs correct type, access and exception handling. A reflective `Method.invoke` wraps a thrown target exception in `InvocationTargetException`; inspect its cause when translating the failure. Access suppression is not a universal permission to cross module boundaries.

A named module distinguishes exported packages for ordinary accessible APIs from opened packages for deep reflection. setAccessible or trySetAccessible can be restricted by module openness and access rules. An opens change is a deployment decision, not evidence that the reflected target contract is safe. Keep Spring proxy behavior at 7.2 and class identity at 9.8.

Interface default methods are implementation-bearing methods, while static interface methods belong to the interface. Private interface helpers do not become inherited public contracts. Resolve conflicting inherited defaults explicitly where the language requires it; sharing multiple contracts does not give a class multiple superclass state layouts.

Follow-up answers

Why is an annotation invisible to reflection? `Runtime` reflection normally needs RUNTIME retention; the default retention is CLASS. Confirm the element actually carries the annotation and whether inheritance or repeatable-annotation rules apply.

Why can reflection fail after a module migration? Readability, exported packages and opened packages affect access. setAccessible does not universally bypass module encapsulation; open the specific package deliberately when the integration requires deep reflection.

Why does a transaction annotation need an interceptor? It declares metadata and does not alter the JVM invocation instruction. Spring proxy or weaving infrastructure must interpret it and apply transaction behavior on the intercepted call path.

JLS interface and annotation rules

Reflective access and module openness

10.2 Collector internals and advanced composition

Advanced - understand, don't memorize

Recognition is enough for: `Collectors.filtering()`; `Collectors.flatMapping()`; `Collectors.collectingAndThen()`; `Collectors.teeing()`

`summarizingInt()`; `groupingByConcurrent()`; `toConcurrentMap()`

`Collector.of()` `Collector.Characteristics`

`Stream.iterate()`; `Stream.generate()`; `takeWhile()`; `dropWhile()`

Strong response if exact syntax is forgotten: I understand the operation conceptually, but it is not an API overload I use frequently enough to memorize exactly. I would verify the signature rather than guess.

`Collector` internals - senior awareness

Conceptually: `Collector`<T, A, R>

Where: T = stream element

A = accumulation container

R = final result

A `Collector` conceptually defines: supplier; accumulator; combiner; finisher; characteristics

Example `List` collector: supplier

```
→ create List
```

accumulator

```
→ add element
```

combiner

```
→ combine partial Lists
```

finisher

```
→ produce final result
```

Why a combiner?

Parallel execution may create multiple partial results which must be merged.

You should understand this concept.

You do not need to memorize custom `Collector` implementations for ordinary interviews.

10.3 Serializable Externalizable and safe wire formats

Core contract and interview points

`Externalizable` requires a public no-argument constructor for reconstruction. writeExternal and readExternal must agree on field order and types; include a schema version and validate bounds before allocating from untrusted lengths. Java object-stream protocol overhead still exists around the custom payload.

Do not deserialize untrusted object graphs without an explicit trust boundary, filtering, and validation. A data format with a defined schema is often easier to evolve across services. Defend compatibility, missing fields, malicious sizes, and the distinction between a wire representation and an in-memory domain object.

`Externalizable` (Custom Serialization)

Sometimes risk data has to travel between servers.

Before sending over the network, Java objects must become bytes.

This process is called serialization.

`Serializable`

Java's default serialization: implements `Serializable` Default Java serialization is convenient for supported object graphs, but cost and compatibility depend on the graph, class descriptors, references, and stream reuse.

It automatically writes:

- class metadata
- field names
- object information

That means more bytes and more CPU work.

`Externalizable`

The example instead uses; implements `Externalizable`; The programmer explicitly decides:

- what fields to write
- in what order
- how many bytes

This produces a compact binary format.

Custom field serialization can reduce payload and processing costs for a suitable schema; benchmark rather than promising that it is always faster.

Smaller network messages.

`Externalizable` Deep Dive

First, what is Serialization?

Serialization means converting an object into bytes.

Java `Object`

↓

Serialize

↓

10101010101010010101010

↓

Network The receiver reconstructs the object.

Bytes

↓

Deserialize

↓

Java `Object` The example needs this because risk state must travel between JVMs (rx-srv ↔ rx-wsgwy / ring communication).

Why not simply send Java `Objects`?

`Objects` only exist inside one JVM.

JVM A

RiskCheck `Object`

❌

JVM B cannot understand it Need a common language.

Bytes.

`Externalizable`

Instead, you write exactly what goes over the network.

Example

Complete Java 17+ example. Save in RiskCheck.java. The custom version and bounds are validated before state assignment; the surrounding object-stream protocol still carries its own descriptors. This is an owned data-format example, not permission to deserialize arbitrary untrusted object graphs.

```
import java.io.*;

public class RiskCheck implements Externalizable {
    private static final long serialVersionUID = 1L;
    private long limit;
```

```
    private long consumed;
    public RiskCheck() {} // Required public no-argument constructor.

    @Override
    public void writeExternal(ObjectOutput out) throws IOException {
        out.writeInt(1); // Custom payload version, separate from serial
UID.
        out.writeLong(limit);
        out.writeLong(consumed);
    }

    @Override
    public void readExternal(ObjectInput in) throws IOException {
        int version = in.readInt();
        if (version != 1) throw new InvalidObjectException("version");
        long newLimit = in.readLong();
        long newConsumed = in.readLong();
        if (newLimit < 0 || newConsumed < 0 || newConsumed > newLimit) {
            throw new InvalidObjectException("invalid risk state");
        }
        limit = newLimit;
        consumed = newConsumed;
    }
}
```

`Externalizable` controls the object payload written by writeExternal, but `ObjectOutputStream` still has stream and object protocol overhead.

`Externalizable` does not imply that all reflection or construction overhead disappears.

`Externalizable` does not eliminate all stream metadata.

For a standalone compact wire format, define explicit framing, field types, schema versions, bounds, and compatibility rules.

Official reference Java 25 Externalizable API

Default `Serializable` field handling omits static and transient instance fields; custom writeObject can deliberately write additional data. Declare serialVersionUID when compatible class evolution matters. During `Serializable` reconstruction, the first non-serializable superclass constructor has a special role; Externalizable instead requires its public no-argument constructor. A matching UID does not make every schema change semantically compatible.

Java serialization contract

Object Serialization Specification class and field contracts

10.4 Files paths character encodings and NIO

Core contract and interview points

Use `Path` to describe a filesystem location and `Files` for file operations. State the charset at the byte-to-text boundary, for example `StandardCharsets.UTF_8`. A Reader consumes text units while an InputStream consumes bytes; Unicode character policy stays at 9.4. For a large file, avoid readAllBytes or readAllLines when materializing everything would exceed the memory budget.

`Files.lines` is lazy and retains an open file until its `Stream` is closed. A terminal operation does not replace resource ownership. Opening can throw `IOException`; an I/O failure during lazy traversal can be reported as UncheckedIOException. Decide the failure policy at the caller boundary.

`Method` fragment requiring java.nio.file.`Files`, `Path`, java.nio.charset.`StandardCharsets`, java.util.stream.`Stream` and java.io.`IOException`. It counts nonblank lines without retaining all lines; memory still depends on individual line size and the pipeline.

```
static long countNonBlankLines(Path path) throws IOException {
    try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8))
{
        return lines.filter(line -> !line.isBlank()).count();
    }
}
```

Buffers and channel behavior

A `Buffer` has capacity, limit and position. After filling it, flip sets limit to the written position and position to zero for reading. clear resets the cursor range for reuse but does not erase bytes. compact preserves unread data and prepares space for further writes. Check remaining rather than assuming the buffer is full or empty.

Channel reads and writes may be partial; handle framing, remaining data, EOF and zero progress. NIO does not mean every operation is nonblocking. `FileChannel` is not a selectable network channel; selectors coordinate readiness for selectable channels, and asynchronous channels use a different completion model. Blocking, timeout and admission policies remain part of the application contract.

Follow-up answers

Who closes a `Files.lines` stream? The caller owns the returned resource-backed stream and should close it with try-with-resources. A terminal operation does not replace closure.

Why does clear not erase sensitive bytes? `Buffer.clear` resets position and limit; it does not overwrite storage. Deliberate overwriting and the surrounding secret-handling lifecycle are separate concerns.

Why is one channel write insufficient? A write may transfer fewer bytes than remain, or zero in a non-blocking mode. Continue according to buffer state, readiness, deadline and error handling; avoid a busy loop on repeated zero writes.

Files and lazy line streams

Buffer state transitions

Channel and selector models

10.5 Object layout compressed references and footprint

Interview question What does a Java object look like in memory?

Conceptually: `Object`

```
├── Header
│    ├── mark word
│    └── class metadata reference
├── instance fields
└── padding/alignment
```

For arrays there is also length information.

Exact sizes depend on:

- JVM implementation
- architecture
- compressed references
- alignment
- field layout

Don't memorize one universal object size.

Interview question What is the object header used for?

HotSpot object headers contain runtime metadata required for operations such as:

- identity hash-related state
- GC-related information
- synchronization/locking-related state
- class identification

Implementation details evolve between JVM versions.

Therefore interview-safe wording is: The object header contains JVM metadata needed for runtime object management; exact bit layouts are implementation/version dependent.

Interview question What are compressed ordinary object pointers?

On 64-bit systems, raw 64-bit references increase memory footprint.

HotSpot can represent heap references more compactly under suitable heap/configuration constraints.

Benefits: smaller references

```
↓
```

smaller objects

↓

better cache utilization

↓

potentially lower memory bandwidth This is why heap sizing can occasionally affect more than simply "available memory."

10.6 Native memory JNI and process limits

Interview question What is off-heap memory?

Memory outside the Java heap.

Examples include:

- direct byte buffers
- native libraries
- JVM internal native structures
- metaspace

Advantage for direct buffers: Native I/O can sometimes operate efficiently without extra copying through ordinary heap buffers.

But: Off-heap doesn't mean free or automatically safe.

It still consumes process/container memory.

Interview question Why can application OOM even when Java heap looks fine?

Because: Process memory

```
├── Java heap
├── metaspace
├── thread stacks
├── direct buffers
├── code cache
├── native libraries
└── JVM internals
```

Container: memory limit = 4 GB; heap = 3.5 GB; leaves little room for everything else.

The OS/container may kill the process even though heap itself hasn't hit its configured maximum.

Excellent production interview answer.

Interview question Why can too many threads cause OOM?

Each platform thread requires native resources, including stack memory.

Suppose: 10,000 platform threads; ×; significant stack reservation/resources; This can exhaust native memory or OS thread limits.

Potential symptom: `OutOfMemoryError`: unable to create native thread; This isn't necessarily a heap problem.

Interview question Can heap be healthy while process memory is exhausted?

Yes.

10.7 Reference strengths and weak maps

Reference Strength

Know the vocabulary: Strong; Soft; Weak; Phantom; Ordinary:

```
Object o = new Object();
```

is a strong reference.

WeakReference allows an object to be reclaimed once it is only weakly reachable, subject to JVM reference-processing semantics.

Interview question Strong vs weak references?

Strong

Normal:

```
Object x = new Object();
```

As long as strongly reachable, object isn't collectible.

Weak

A weakly referenced object can be collected when no strong reachability keeps it alive.

Used in specialized structures such as certain canonicalization/metadata patterns.

Interview question What is WeakHashMap?

Its keys are weakly referenced.

If a key is no longer strongly reachable elsewhere, its mapping may eventually disappear following GC processing.

It is not a universal cache implementation.

For production caching you usually need explicit policies such as: maximum size; expiry; refresh; metrics

`WeakHashMap` values are strongly referenced. If a value refers directly or indirectly to its own key, that path can keep the key alive and defeat the intended weak-key cleanup. It is not an expiry clock or a thread-safe cache. Do not write a test that assumes `System.gc` will remove an entry immediately.

WeakHashMap key and value reachability

10.8 False sharing and contention awareness

Interview question What is false sharing?

CPUs cache memory in cache lines.

Suppose: Cache line: [counterA][counterB]; `Thread` A updates A.

`Thread` B updates B.

They are logically independent, but because they share a cache line, cache-coherence traffic may bounce that line between cores.

Result: high cache contention; without logical lock contention; That's false sharing.

This is advanced but useful in very high-throughput systems.

Interview question Should ordinary application developers manually pad every field?

No.

False sharing matters primarily in carefully measured high-performance concurrent code.

Prematurely padding structures can:

- waste memory
- increase complexity
- hurt cache density

Measure first.

10.9 Memory fences VarHandle and AQS

What is AQS?

Answer:
AQS = AbstractQueuedSynchronizer.

It's a framework used to build synchronization primitives such as: `ReentrantLock`; `Semaphore`; `CountDownLatch`; `ReentrantReadWriteLock`; Conceptually: CAS on state

```
                ↓
acquisition succeeds?
/        \ yes          no
     ↓                      ↓
continue      queue thread
                            ↓
park
                            ↓
predecessor releases
```

↓

unpark It maintains a synchronization state and a queue of waiting threads.

You generally use the abstractions built on AQS rather than implementing AQS yourself.

Interview question What is a memory barrier/fence?

At a high level, memory barriers constrain certain memory-operation reorderings and help establish required visibility/order behavior.

Java programmers normally use high-level constructs: `volatile`; `synchronized`; locks; atomics; `VarHandle`; The JVM maps their semantics onto compiler and hardware mechanisms appropriate to the platform.

Interview answer: Don't reason directly in terms of a specific CPU fence unless you're working at JVM/`VarHandle` level. Reason primarily using JMM happens-before guarantees.

Interview question What is `VarHandle`?

`VarHandle` provides controlled low-level access to variables with different memory-ordering semantics.

It supports operations such as:

- plain accesses
- opaque accesses
- acquire/release accesses
- `volatile` semantics
- CAS-like atomic operations

It is useful for libraries and advanced concurrent structures.

For ordinary application code: Prefer higher-level concurrency abstractions unless you have a demonstrated reason to use `VarHandle`.

Interview question What is AQS?

AQS = AbstractQueuedSynchronizer.

It's a framework used to build synchronization primitives around:

- synchronization state
- atomic state transitions
- queues of waiting threads
- park/unpark coordination

Important Java utilities are built using AQS-style machinery.

Examples commonly associated with AQS include: `ReentrantLock`; `Semaphore`; `CountDownLatch`

Interview question Explain AQS intuitively.

Think: synchronizer state

```
                        ↓
Thread A → acquire → success → runs
Thread B → acquire → fails
                        ↓
```

queue

```
Thread C → acquire → fails
                        ↓
```

queue

A releases

```
    ↓
```

eligible waiter proceeds AQS handles much of the difficult queueing/waiting infrastructure.

Concrete synchronizers define what acquiring/releasing state means.

10.10 JMH and microbenchmark traps

Interview question Why can microbenchmarks lie?

Because JVM execution includes:

- warmup
- JIT
- dead-code elimination
- constant folding
- escape analysis
- GC
- CPU caches
- branch prediction

Naive:

```
long start = System.nanoTime();

for (...) {
    operation();
}

long time = System.nanoTime() - start;
```

may measure something very different from what you intended.

Use JMH for serious Java microbenchmarks.

Interview question Why JMH?

JMH is designed around JVM benchmarking realities.

JMH provides warmup, measurement iterations, forks, state setup and benchmark modes, and helps address dead-code elimination when the benchmark is designed correctly.

Still: A correct microbenchmark doesn't prove production impact.

Production workload validation remains necessary.

11 Lookup Java and SQL recipes

Keep startup, array conversion and joining recipes here for targeted repair. The SQL topic is a bounded backend reference for row counts, joins and operation semantics; SQL reasoning used in the Core transaction and invariant topics remains required.

11.1 Java startup and main methods

main method in Java

What happens if the `main()` method is written without `String` args[]?

For the traditional Java 17 and Java 21 launcher entry point, use public static void `main(String[] args)`, or `String`... args. A class can compile without that entry point but cannot be launched through the traditional class entry point. Java 25 also supports eligible instance main methods and no-argument main methods; state the Java version when answering.

Execution Process

The launcher loads the initial class, links it, initializes it when required, and invokes the selected main method. Static initializers execute during class initialization in textual order after required superclass initialization. Ordinary static methods and instance methods execute only when called; the JVM does not automatically execute every method.

Is main method compulsory in Java?

Historical static-initializer-only launch tricks are not a useful modern entry-point contract. A class without a launcher-eligible main method can still be compiled and used as a library.

- A static initializer performs class initialization work; it is not a substitute for an application entry point.
- `Class` loading and class initialization are distinct. A class is initialized on specified active uses; initialization executes static field initializers and static blocks in order.
- Do not use `System.exit` to hide a missing entry point. Compile-time acceptance and successful application launch are different checks.

Compilation does not generally require a main method. Missing traditional entry points are reported by the launcher, not as a general Java compilation rule or a blanket bytecode verification failure.

Official reference Java 25 compact source files and instance main methods | Java Language Specification execution and memory model

11.2 Arrays lists copying and conversions

Convert `ArrayList` of `String` to `String` Array in Java

```
        ArrayList<String> al = new ArrayList<String>();
String[] answer = Arrays.copyOf(
```

```
            al.toArray(), al.size(), String[].class);
        String k[] = al.toArray(new String[al.size()]);
```

ArrayCopy

```
public static void arraycopy(Object source_arr, int sourcePos,
                             Object dest_arr, int destPos, int len)
```

Parameters : source_arr : array to be copied from; sourcePos : starting position in source array from where to copy; dest_arr : array to be copied in; destPos : starting position in destination array, where to copy in; len : total no. of components to be copied.

```
System.arraycopy(source_arr, sourcePos, dest_arr,
                                              destPos, len);
```

Arrays and lists in practice

ArrayList and LinkedList selection

Further reading

`ArrayList` gives constant-time indexed access and amortized constant-time append. Indexed insertion or removal shifts a suffix of references; removing the last element needs no suffix shift.

`LinkedList` gives constant-time relinking after an iterator has located the node. Finding an index or value can require linear traversal. End operations can be constant time, but nodes add allocation and pointer overhead; deletion is not universally faster.

An ArrayList has a contiguous backing array of references, not necessarily contiguous element objects. A LinkedList uses separately allocated linked nodes. Choose ArrayList for most general list work and ArrayDeque for a queue or stack; choose LinkedList only for a justified access pattern.

11.3 Joining strings

`Joining()`

```
employees.stream()
    .map(Employee::name)
    .collect(Collectors.joining(", "));
```

Result: Alice, Bob, Charlie

11.4 Joins row counts and data definition operations

Sql: table1 has 10 rows, table2 has 20

Question: SELECT *

```
FROM table1, table2;
```

How many rows?

Cartesian Product

Without a join condition, this is conceptually: SELECT *; FROM table1

```
CROSS JOIN table2;
```

Every table1 row combines with every table2 row.

10 × 20 = 200 Answer:

rows

Visual

table1 table2

A X; B Y; Z; Result: A X; A Y; A Z

B X; B Y; B Z; Therefore: 2 × 3 = 6; Same principle: 10 × 20 = 200

Delete All Rows From table1

```
DELETE FROM table1;
```

Now: table1 = 0; table2 = 20; Run: SELECT *

```
FROM table1, table2;
```

Result: 0 × 20 = 0; Therefore:

rows.

Inner join

SELECT *; FROM employee e; INNER JOIN department d

```
    ON e.department_id = d.id;
```

Returns only matching combinations.

Conceptually: LEFT ∩ RIGHT; according to the join predicate.

Left join

SELECT *; FROM employee e; LEFT JOIN department d

```
    ON e.department_id = d.id;
```

Returns: all left rows + matching right rows; No right-side match: right columns = NULL

Cross join

SELECT * FROM A

```
CROSS JOIN B;
```

If: |A| = m; |B| = n; then: |result| = m × n

DELETE vs TRUNCATE vs DROP

Interview-level distinction:

Operation	Meaning
DELETE	Deletes rows
TRUNCATE	Removes all rows using DB-specific DDL/optimized semantics
DROP	Removes the table object

DELETE can generally: DELETE FROM employee

```
WHERE id = 10;
```

TRUNCATE doesn't use a row-filtering WHERE clause.

DROP removes the table definition itself.

Be careful with claims such as: TRUNCATE cannot rollback; because transactional behavior varies by database.

12 Official references

Use the specification and APIs to check exact contracts; distinguish language guarantees from HotSpot implementation details and measurements on one deployment. The references below support the version and API distinctions used throughout the notes.

Java 25 compact source files and instance main methods

Java Language Specification execution and memory model

Java 25 String API

Java 25 HashMap API

Java 25 ConcurrentHashMap API

Java 25 LinkedHashMap API

Java 25 Stream API

Java 25 CompletableFuture API

Java 25 ThreadPoolExecutor API

Java 25 LongAdder API

Java 25 Object API and cleanup notes

Java 25 garbage collection tuning guide

JDK 24 changes including virtual thread synchronization and ZGC

Spring declarative transaction annotation behavior

Java 25 BigDecimal API

Java language types values variables and erasure

Java language classes inheritance and records

Java language threads locks and memory model

Java 25 Externalizable API

Java 25 thread states